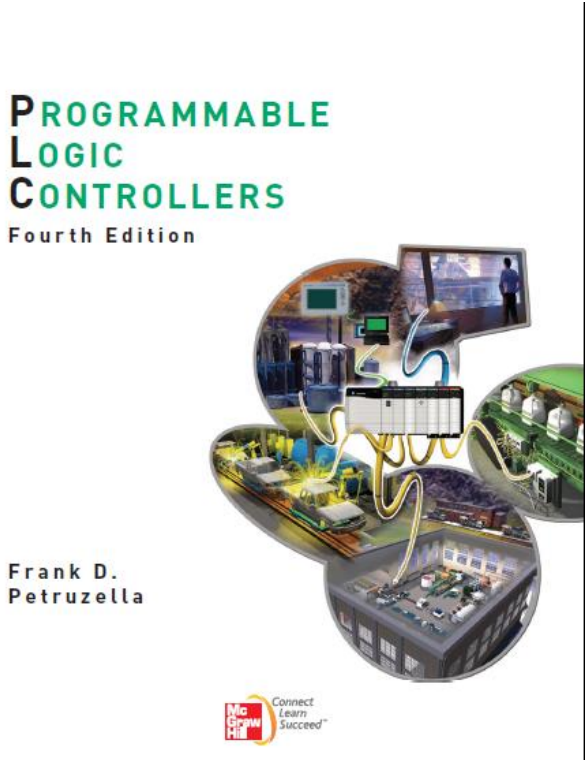
# PROGRAMMABLE LOGIC CONTROLLERS

**Text Book**


PROGRAMMABLE
LOGIC
CONTROLLERS
Fourth Edition
Frank D.
Petruzella
Connect
Learn
Succeed

# Chapter 1

## Chapter Objectives

- Define what a programmable logic controller (PLC) is and list its advantages over relay systems
- Identify the main parts of a PLC and describe their functions
- Outline the basic sequence of operation for a PLC
- Identify the general classifications of PLCs

# Introduction to Programmable Logic Controllers (PLC's)

# The First Programmable Logic Controllers(PLCs)

- Introduced in the late 1960's
- Developed to offer the same functionality as the existing relay logic systems
- Programmable, reusable and reliable
  - Could withstand a harsh industrial environment
  - They had no hard drive, they had battery backup
  - Could start in seconds
  - Used Ladder Logic for programming

# Programmable Logic Controller

- A programmable logic controller (PLC) is a specialized computer used to control machines and process.
- It uses a programmable memory to store instructions and specific functions that include On/Off control, timing, counting, sequencing, arithmetic, and data handling

Relay based
control panel



PLC-based control panel

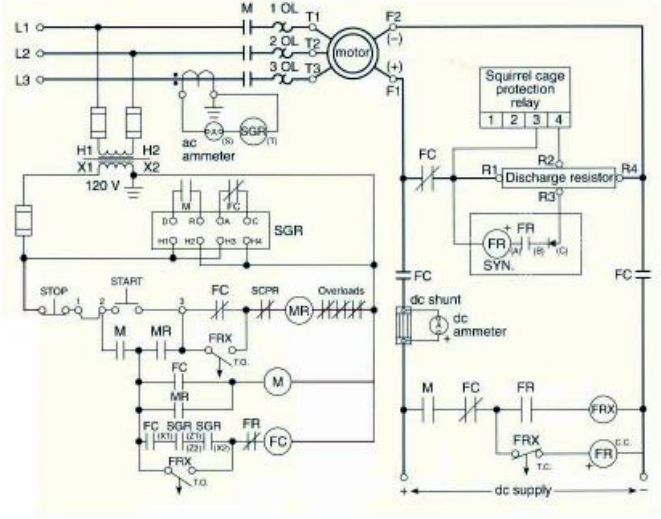


# Advantages of PLC Control Systems

- Flexible
- Faster response time
- Less and simpler wiring
- Solid-state - no moving parts
- Modular design - easy to repair and expand
- Handles much more complicated systems
- Sophisticated instruction sets available
- Allows for diagnostics “easy to troubleshoot”
- Less expensive

# Advantages of a PLC Control System

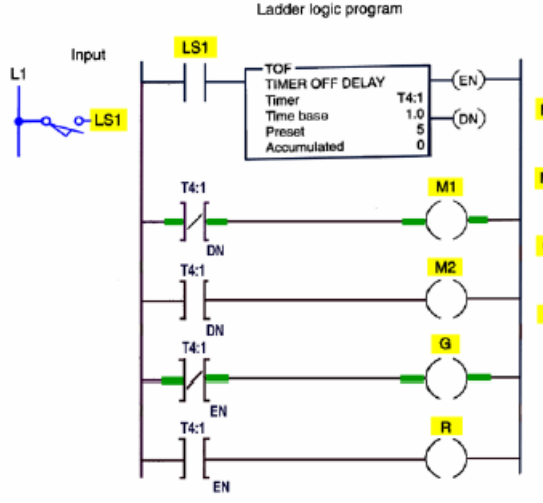
Eliminates much of the hard wiring that was associated with conventional relay control circuits.

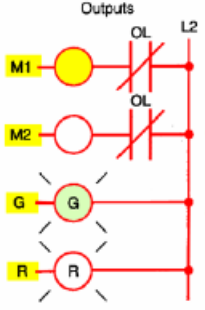



The program takes the place of much of the external wiring that would be required for control of a process.

# Advantages of a PLC Control System

*Increased Reliability:*
Once a program has been written and tested it can be downloaded to other PLCs.



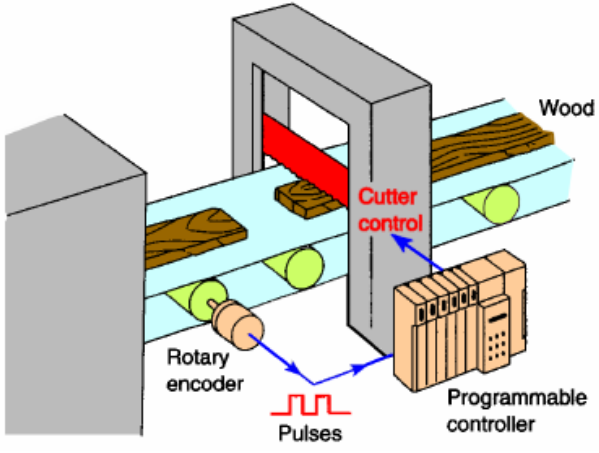




Since all the logic is contained in the PLC's memory, there is no chance of making a logic wiring error.

Conversely .....

# Advantages of a PLC Control System

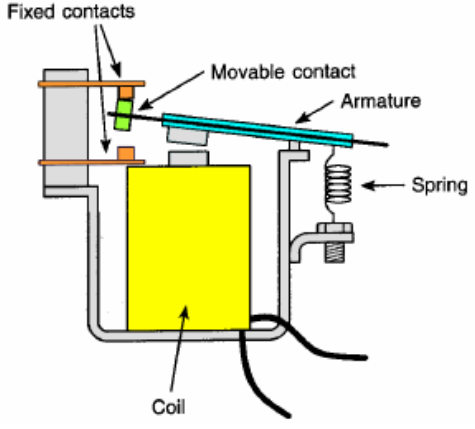
*More Flexibility:*
Original equipment manufacturers (OEMs) can provide system updates for a process by simply sending out a new program.




It is easier to create and change a program in a PLC than to wire and rewire a circuit. End-users can modify the program in the field.

# Advantages of a PLC Control System

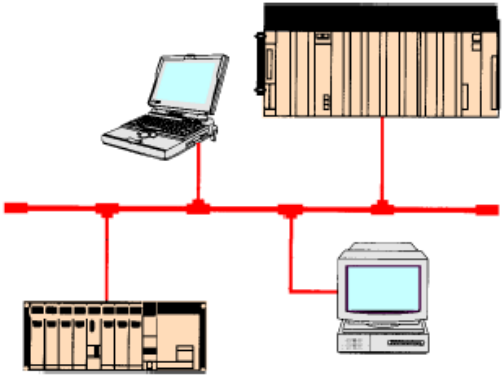
*Lower Costs:*
Originally PLCs were designed to replace relay control logic. The cost savings using PLCs have been so significant that relay control is becoming obsolete, except for power applications.




Generally, if an application requires more than about 6 control relays, it will usually be less expensive to install a PLC.

# Advantages of a PLC Control System

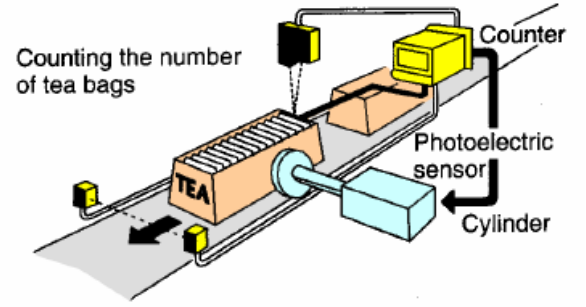
*Communications Capability:*
A PLC can communicate with other controllers or computer equipment.



They can be networked to perform such functions as: supervisory control, data gathering, monitoring devices and process parameters, and downloading and uploading of programs.

# Advantages of a PLC Control System

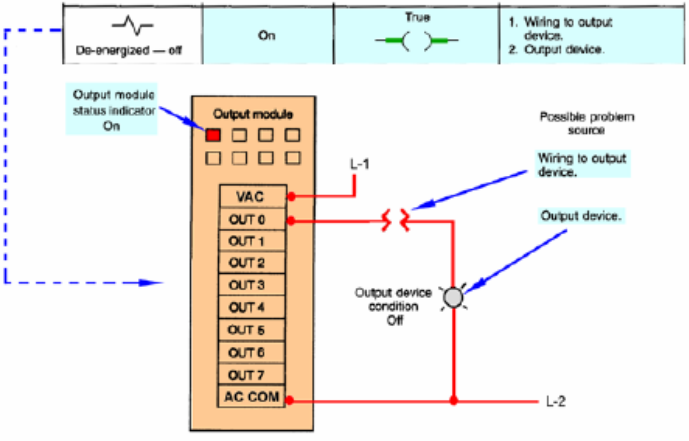
*Faster Response Time:*
PLCs operate in real-time which means that an event taking place in the field will result in an operation or output taking place.




Machines that process thousands of items per second and objects that spend only a fraction of a second in front of a sensor require the PLC's quick response capability.

# Advantages of a PLC Control System

*Easier To Troubleshoot:*
PLCs have resident diagnostic and override functions allowing users to easily trace and correct software and hardware problems.




The control program can be watched in real-time as it executes to find and fix problems

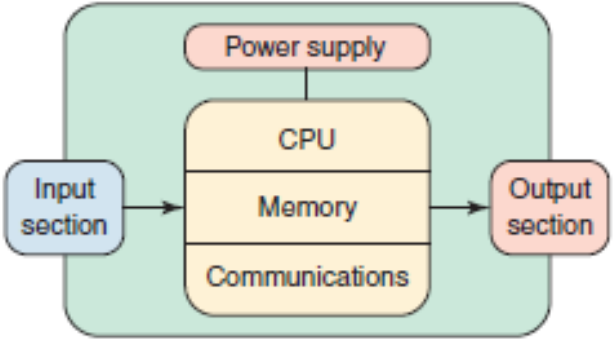
# Parts of a PLC

A.central processing unit (CPU)

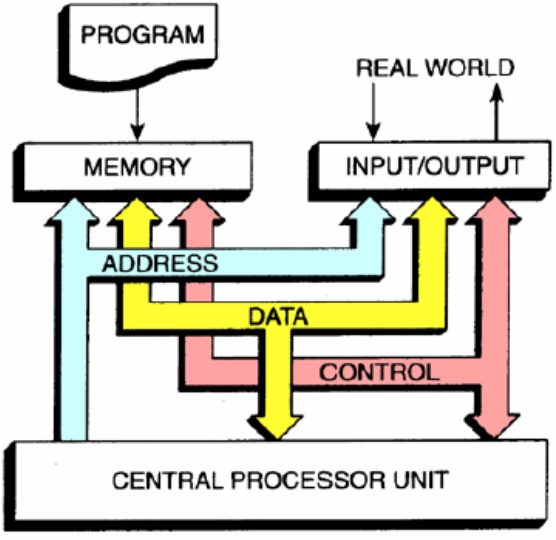
B. input/output (I/O) section

C. The *power supply*

D. The programming device.



# PLC Architecture




The structure of a PLC is based on the same principles as those employed in computer architecture.

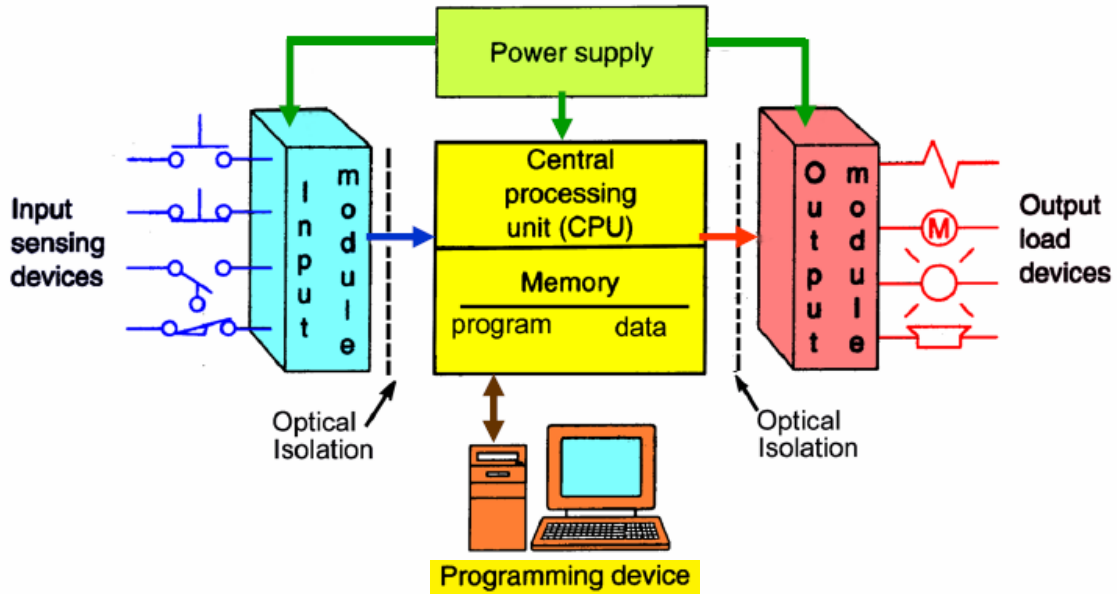
# PLC Architecture

- An *open architecture* design allows the system to be connected easily to devices and programs made by other manufacturers.

- A *closed architecture* or *proprietary* system, is one whose design makes it more difficult to connect devices and programs made by other manufacturers.

NOTE: When working with PLC systems that are proprietary in nature you must be sure that any generic hardware or software you use is compatible with your particular PLC.

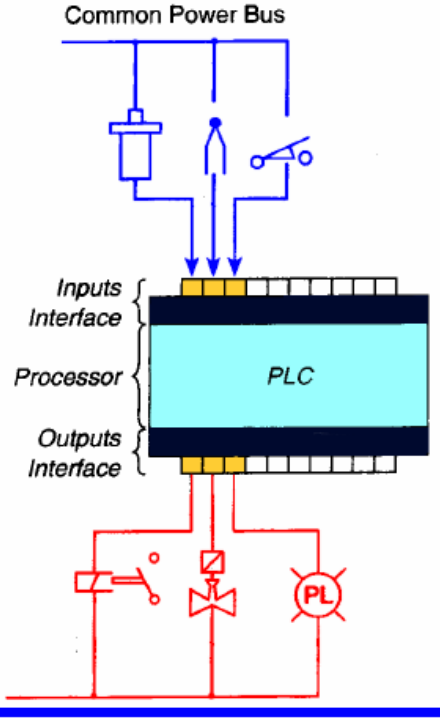
# PLC System

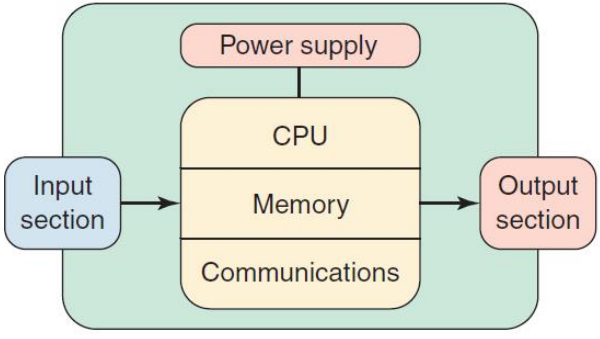

Power supply
Central processing unit (CPU)
Memory
program
data
Input module
Output module
Input sensing devices
Output load devices
Optical Isolation
Optical Isolation
M
Programming device

# I/O Configurations

## Fixed I/O

- Is typical of small PLCs
- Comes in one package, with no separate removable units.
- The processor and I/O are packaged together.
- Lower in cost – but lacks flexibility.




Power supply
CPU
Input section
Memory
Output section
Communications

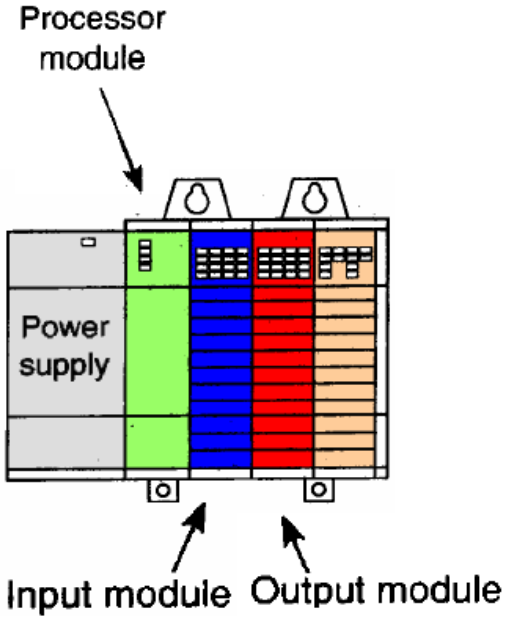


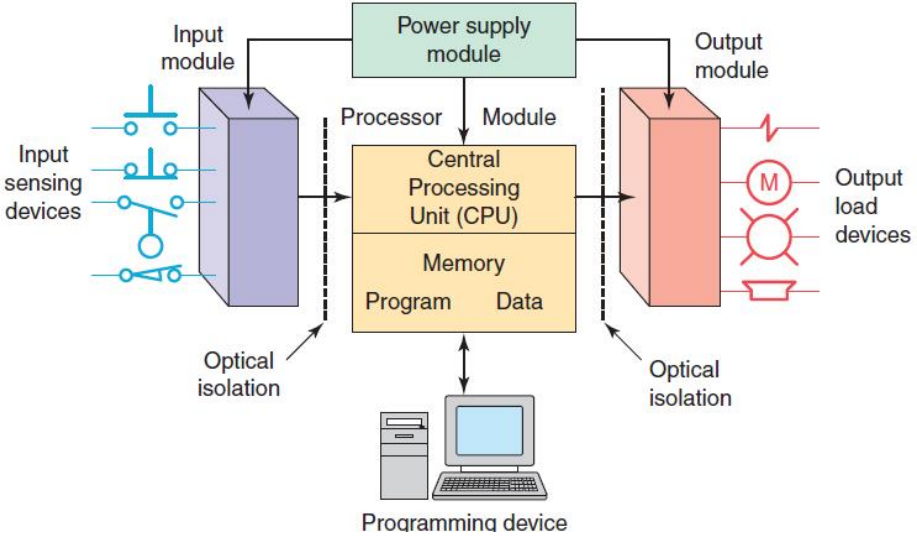
# I/O Configurations

## Modular I/O

Is divided by compartments into which separate modules can be plugged.

This feature greatly increases your options and the unit's flexibility. You can choose from all the modules available and mix them in any way you desire.

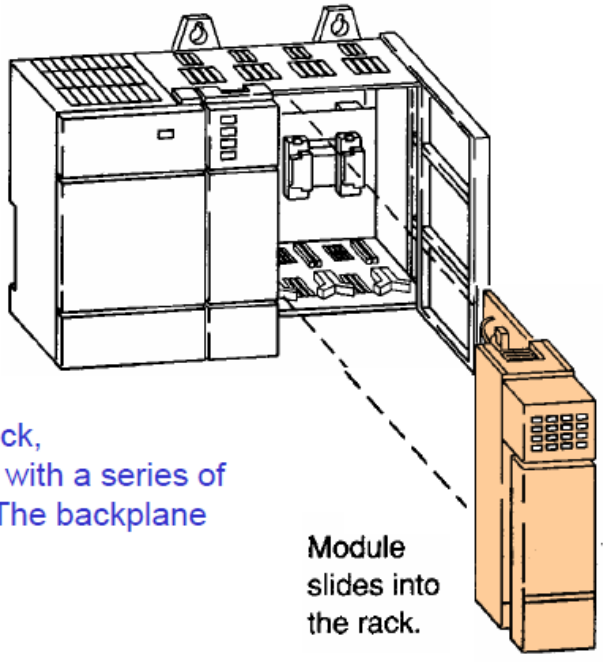



Input module
Power supply module
Output module
Processor Module
Input sensing devices
Central Processing Unit (CPU)
Memory
Program Data
M
Output load devices
Optical isolation
Optical isolation
Programming device



# I/O Configurations

## Modular I/O




When a module slides into the rack, it makes an electrical connection with a series of contacts - called the *backplane.* The backplane is located at the rear of the rack.

Features

- separate modules can be plugged

- The basic modular controller consists of a rack, power supply, processor module (CPU), input/output (I/O modules), and an operator interface for programming and monitoring.

- The modules plug into a rack.

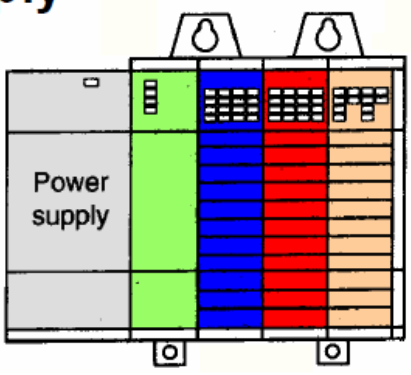
- It has an electrical connection with a series of contacts called the backplane, located at the rear of the rack.

- The PLC processor is also connected to the backplane and can communicate with all the modules in the rack.

# PLC parts in details

# Power Supply

- Supplies DC power to other modules that plug into the rack.
- In large PLC systems, this power supply does not normally supply power to the field devices.

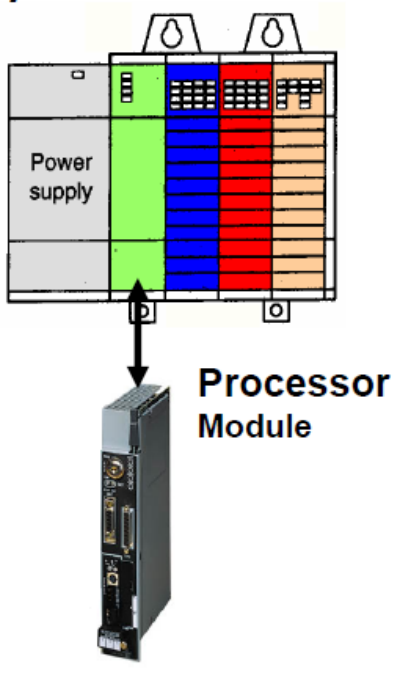



- In small and micro PLC systems, the power supply is also used to power field devices.


DDI1602
Dig 16I
24 Vdc Sink
M
3
4
0
Ethernet
Reset
24V
0V
PE
N
L
Power supply

# Processor (CPU)

- Is the “brain” of the PLC.
- Consists of a microprocessor for implementing the logic, and controlling the communications among the modules.
- Designed so the desired circuit can be entered in relay ladder logic form.
- The processor accepts input data from various sensing devices, executes the stored user program, and sends appropriate output commands to control devices.

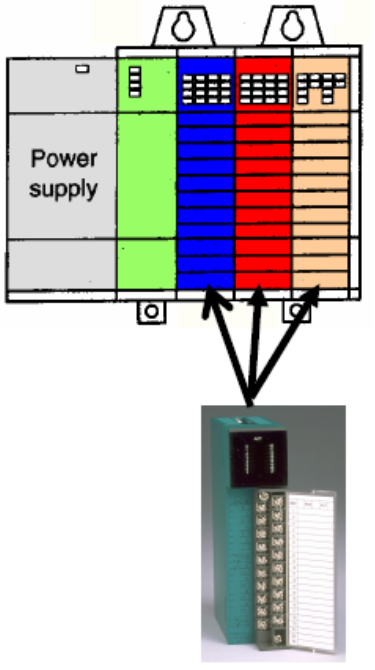



SLC 5/03 CPU
RUN
FORCE
FLT
DH485
BATT
RS232
RUN
REM
PROG
SLC 5/04 CPU
RUN
FORCE
FLT
DH+
BATT
RS232
RUN
REM
PROG
SLC 5/05 CPU
RUN
FORCE
FLT
ENET
BATT
RS232
RUN
REM
PROG

# I/O Section

**Consists of:**

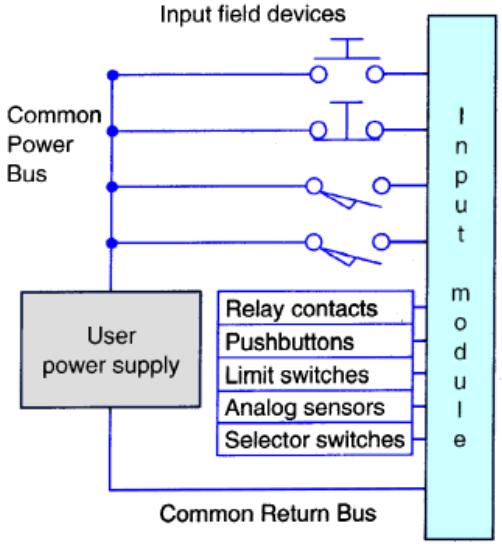
- Input modules
- Output modules.



# I/O Section

## Input Module

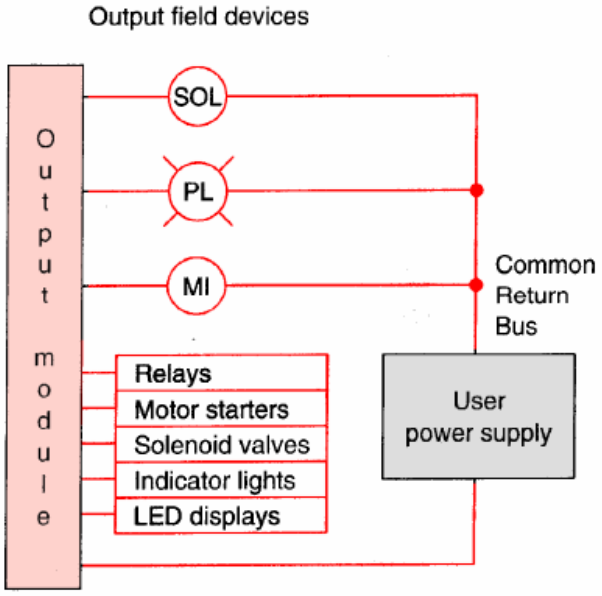
- Forms the interface by which input field devices are connected to the controller.
- The terms “field” and “real world”are used to distinguish actual external devices that exist and must be physically wired into the system.



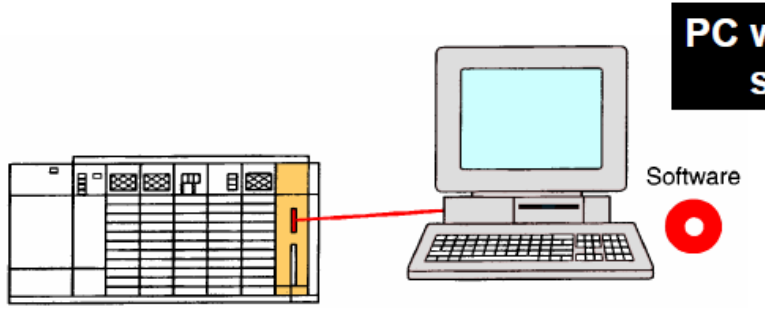
# I/O Section

## Output Module

- Forms the interface by which output field devices are connected to the controller.
- PLCs employ an optical isolator which uses light to electrically isolate the internal components from the input and output terminals.



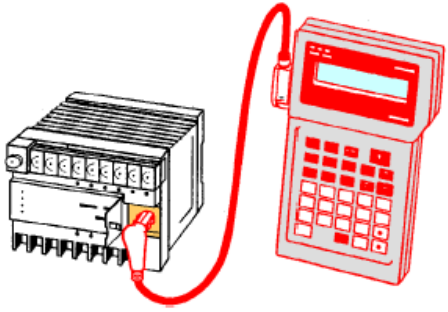
# Programming Device



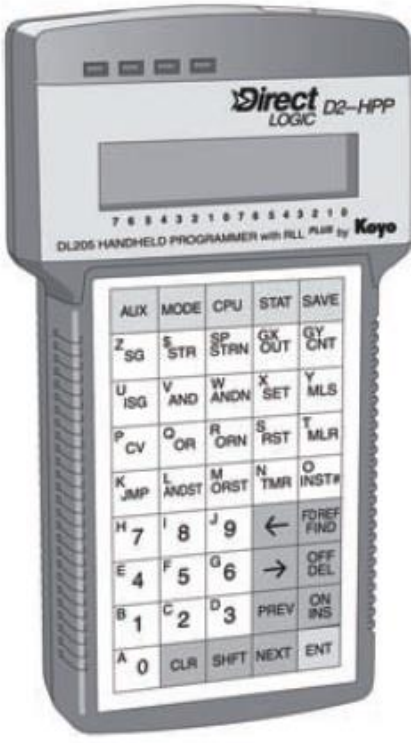

- A personal computer (PC) is the most commonly used programming device
- The software allows users to create, edit, document, store and troubleshoot programs
- The personal computer communicates with the PLC processor via a serial or parallel data communications link

# Programming Device

**Hand-held unit with display**



- Hand-held programming devices are sometimes used to program small PLCs
- They are compact, inexpensive, and easy to use, but are not able to display as much logic on screen as a computer monitor


Direct LOGIC D2-HPP
7 6 5 4 3 2 1 0 7 6 5 4 3 2 1 0
DL205 HANDHELD PROGRAMMER with RLL PLUS by Koyo
AUX
MODE
CPU
STAT
SAVE
Z SG
$ STR
SP STRIN
GX OUT
GY CNT
U ISG
V AND
W ANDN
X SET
Y MLS
P CV
Q OR
R ORN
S RST
T MLR
K JMP
L ANDST
M ORST
N TMR
O INST#
H 7
I 8
J 9
←
FDREF FIND
E 4
F 5
G 6
→
OFF DEL
B 1
C 2
D 3
PREV
ON INS
A 0
CLR
SHFT
NEXT
ENT

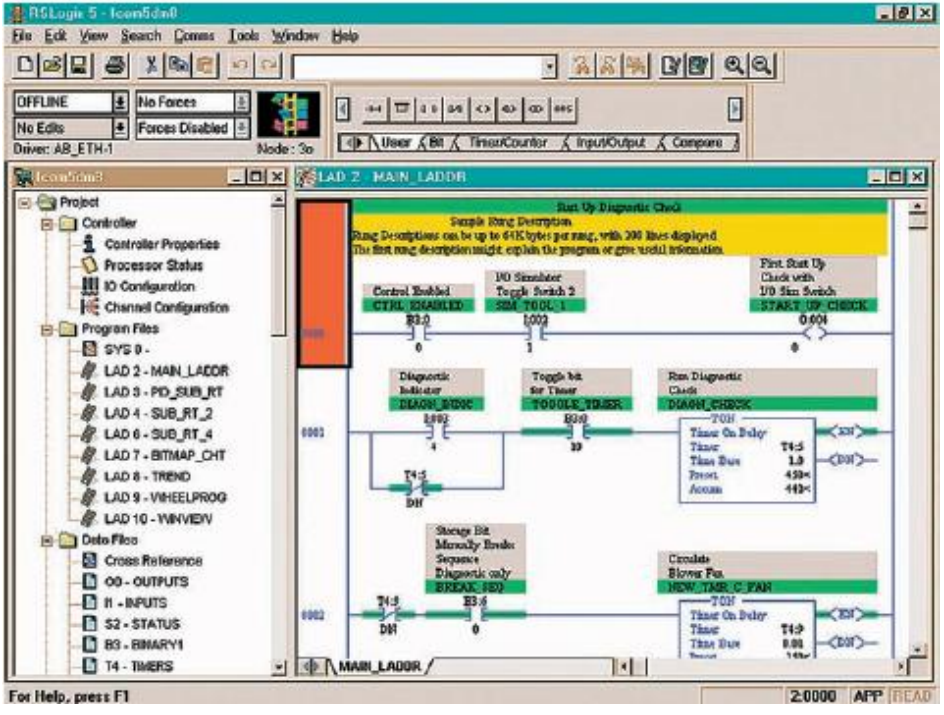
# A PLC *program*

- A *program* is a user-developed series of instructions that directs the PLC to execute an action.

- A *programming language* provides rules for combining the instructions so that they produce the desired action.

- *Relay ladder logic (RLL)* is the standard programming language us with PLCs.

- The relay ladder logic program graphically represents rungs of contacts, coils, and special instruction blocks.

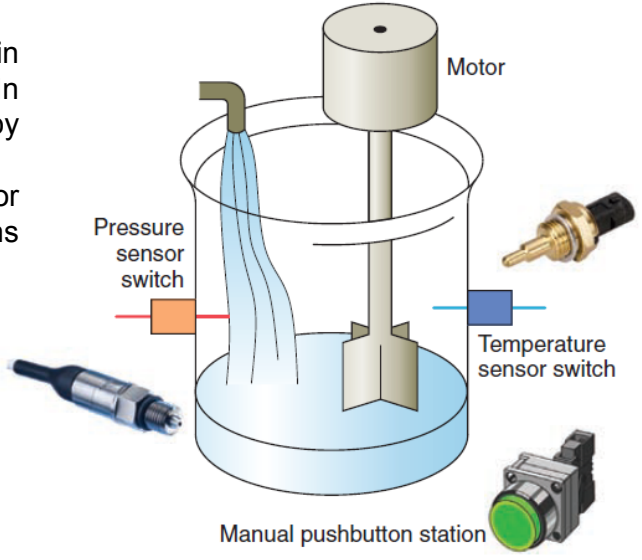

RSLogix 5 - Icon5dm0
File Edit View Search Comms Tools Window Help
OFFLINE
No Forces
No Edits
Forces Disabled
Driver: AB_ETH-1
Node: 3o
User Bit Timer/Counter Input/Output Compare
Icon5dm0
Project
Controller
Controller Properties
Processor Status
IO Configuration
Channel Configuration
Program Files
SYS 0 -
LAD 2 - MAIN_LADDR
LAD 3 - PID_SUB_RT
LAD 4 - SUB_RT_2
LAD 6 - SUB_RT_4
LAD 7 - BITMAP_CHT
LAD 8 - TREND
LAD 9 - WHEELPROG
LAD 10 - WINVIEW
Data Files
Cross Reference
O0 - OUTPUTS
I1 - INPUTS
S2 - STATUS
B3 - BINARY1
T4 - TIMERS
LAD 2 - MAIN_LADDR
Start Up Diagnostic Check
Sample Rung Description
Rung Descriptions can be up to 64K bytes per rung, with 200 lines displayed.
The first rung description might explain the program or give useful information.
Control Enabled
CTRL_ENABLED
I0 Simulator
Toggle Switch 2
SIM_TOGL_1
First Start Up
Check with
I/O Sim Switch
START_UP_CHECK
Diagnostic
Indicator
DIAGN_BIDIC
Toggle bit
for Timer
TOGGLE_TIMER
Run Diagnostic
Check
DIAGN_CHECK
TON
Timer On Delay
Timer T4:5
Time Base 1.0
Preset 450<
Accum 443<
DN
Storage Bit
Manually Evoke
Sequence
Diagnostic only
BREAK_BIT
Circulate
Blower Fan
RECIRC_TANK_C_FAN
MAIN_LADDR
For Help, press F1
2:0000
APP
READ

# Principles of Operation

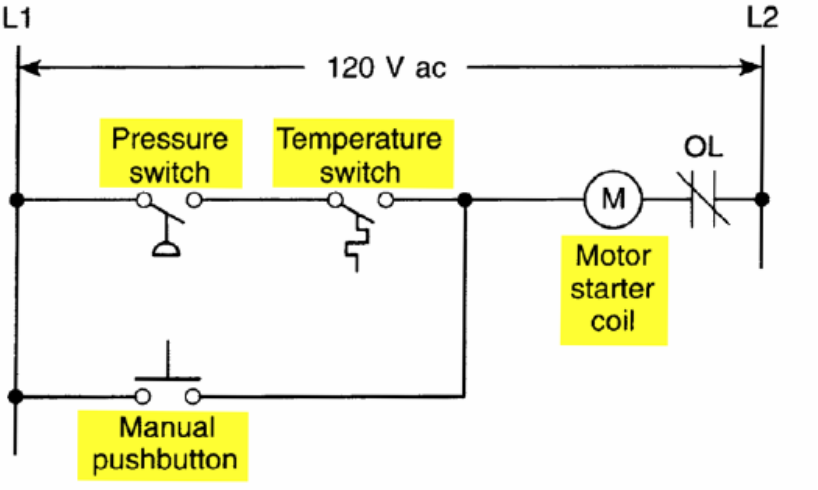
## Mixer process control problem example

Here a mixer motor is to be used to automatically stir the liquid in a vat when the temperature and pressure reach preset values. In addition, direct manual operation of the motor is provided by means of a separate pushbutton station.

The process is monitored with temperature and pressure sensor switches that close their respective contacts when conditions reach their preset values



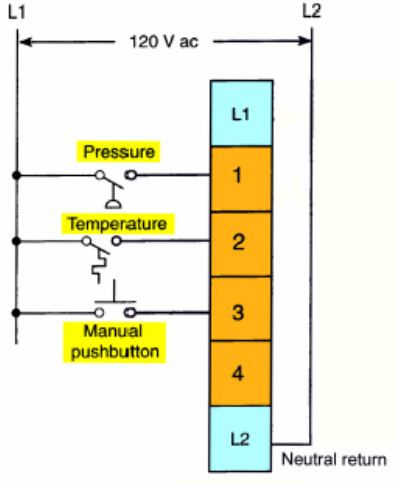
# Process Control Relay Ladder Diagram




Motor starter coil is energized when both the pressure and temperature switches are closed or when the manual pushbutton is pressed

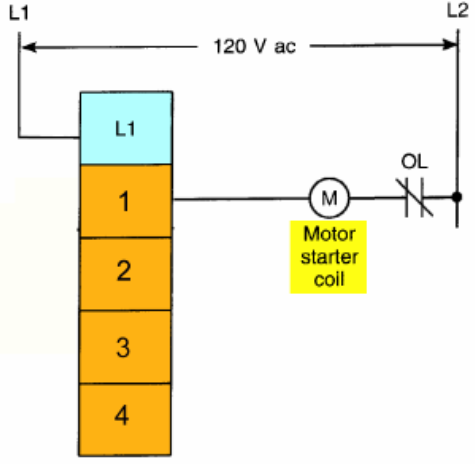
# PLC Input Module Connections

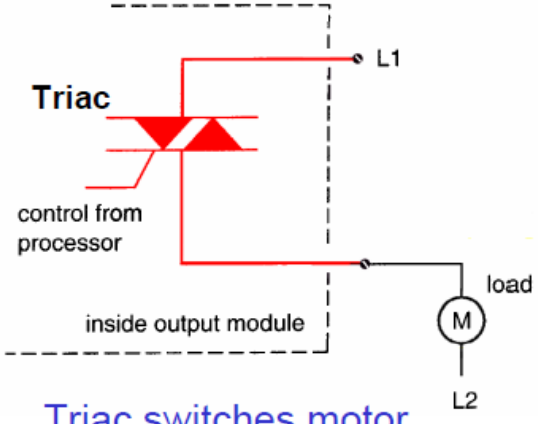
- The same input field devices are used
- These devices are wired to the input module according to the manufacturer's labeling scheme



# PLC Output Module Connections

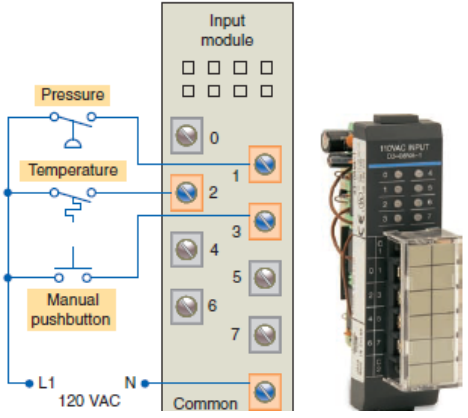
Same output field device is used and wired to the output module

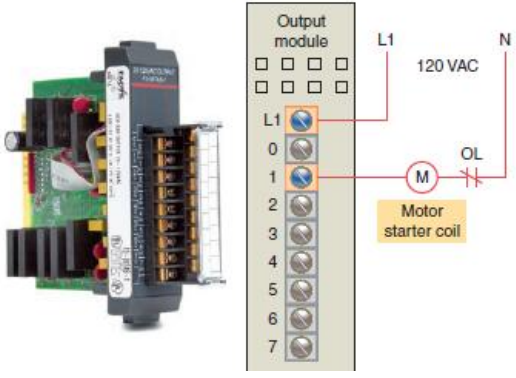


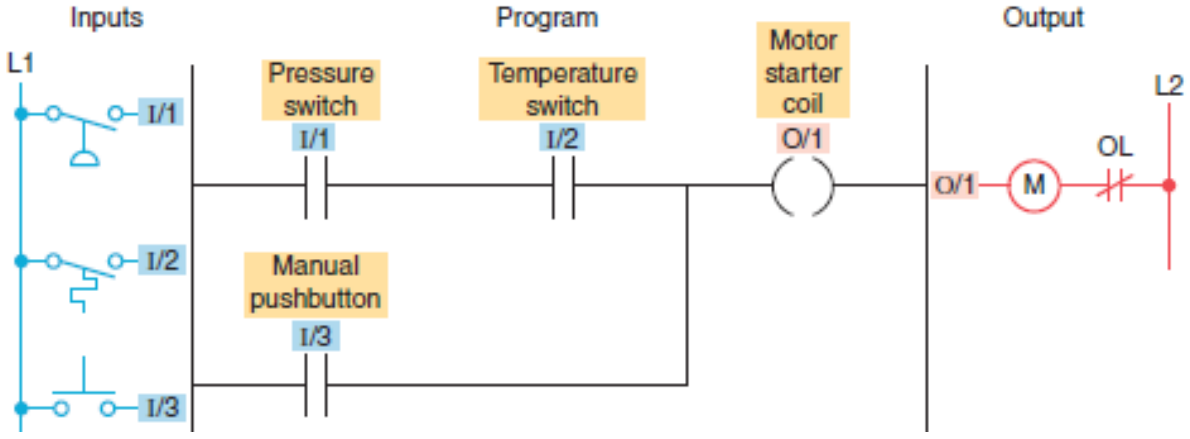

Triac switches motor ON and OFF in accordance with the control signal from the processor

Typical wiring connections

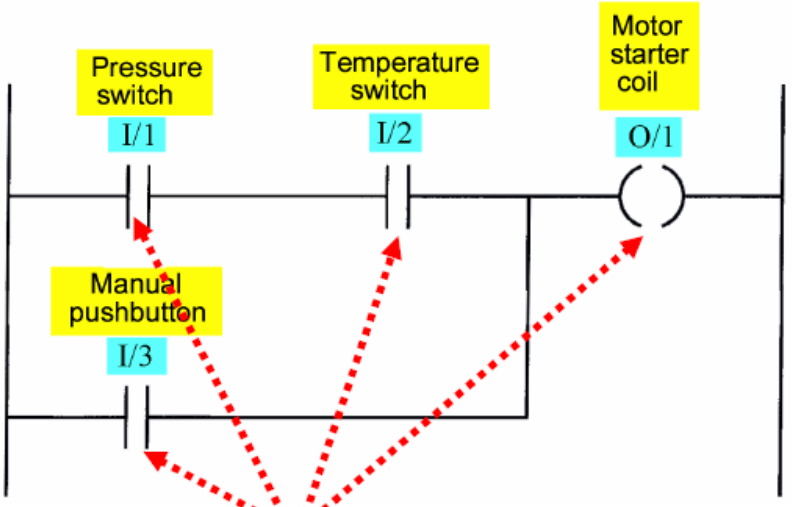


PLC ladder logic program



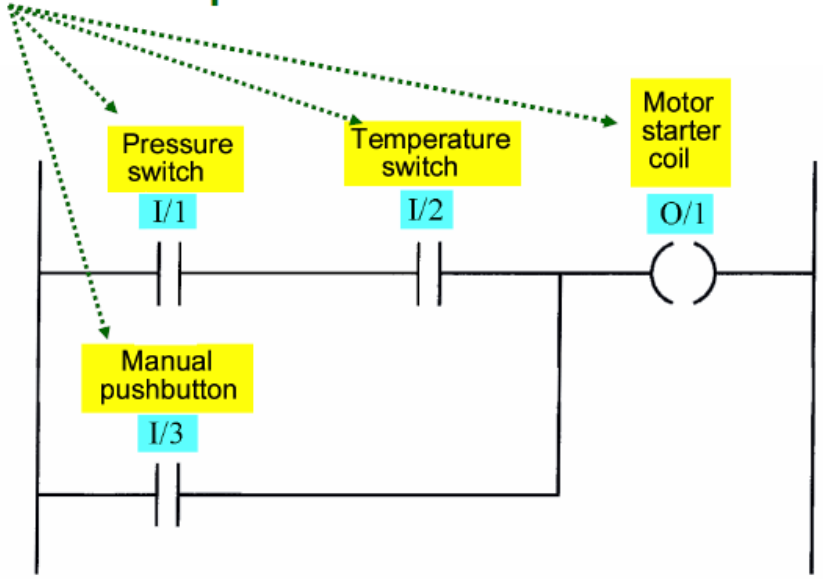
# PLC Ladder Logic Program



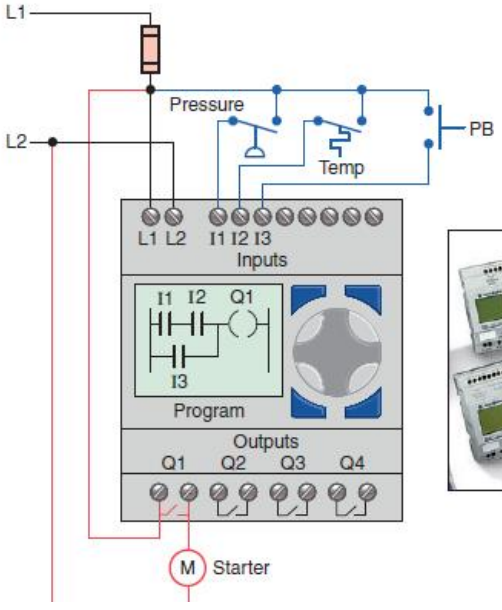

The *symbols* represent *instructions*

# PLC Ladder Logic Program

**The *numbers* represent *addresses***

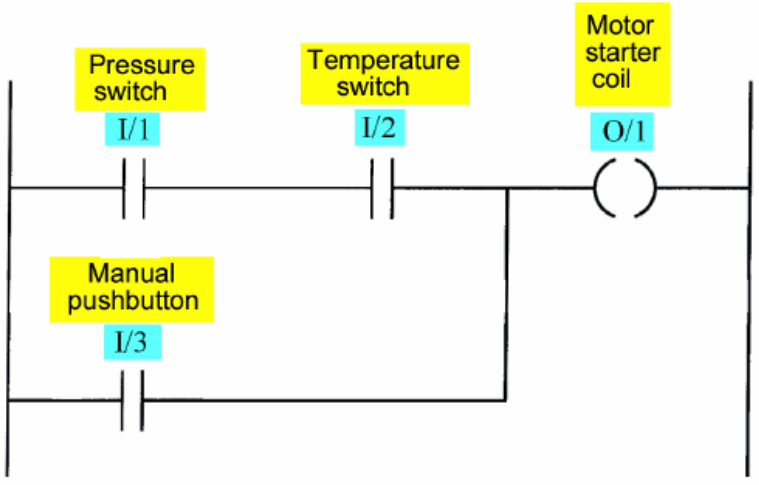


**Typical wiring required to implement the process control scheme using a fixed PLC controller**



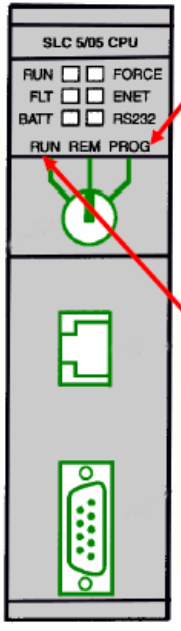



# PLC Ladder Logic Program




- I/O address format will differ, depending on the PLC manufacturer. You give each input and output device an address. This lets the PLC know where they are physically connected

# Entering And Running The PLC Program

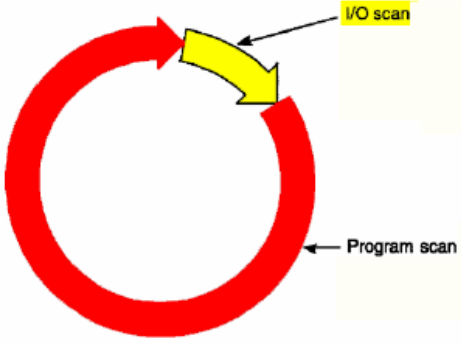



To enter the program into the PLC, place the processor in the PROGRAM mode and enter the instructions one-by-one using the programming device

To operate the program, the controller is placed in the RUN mode, or operating cycle

# PLC Operating Cycle

During each operating cycle, the controller examines the status of input devices, executes the user program, and changes outputs accordingly

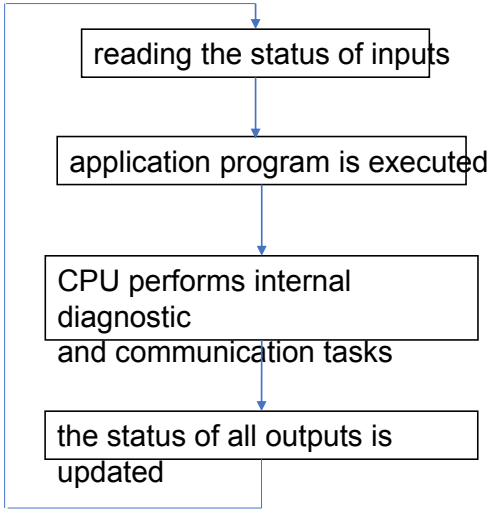



The completion of one cycle of this sequence is called a *scan*. The scan time, the time required for one full cycle, provides a measure of the speed of response of the PLC

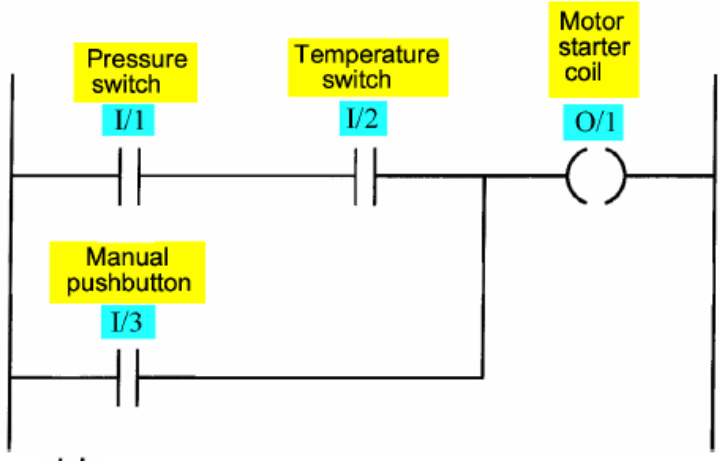
- The PLC program is executed as part of a repetitive process referred to as a scan




Typical PLC scan cycle



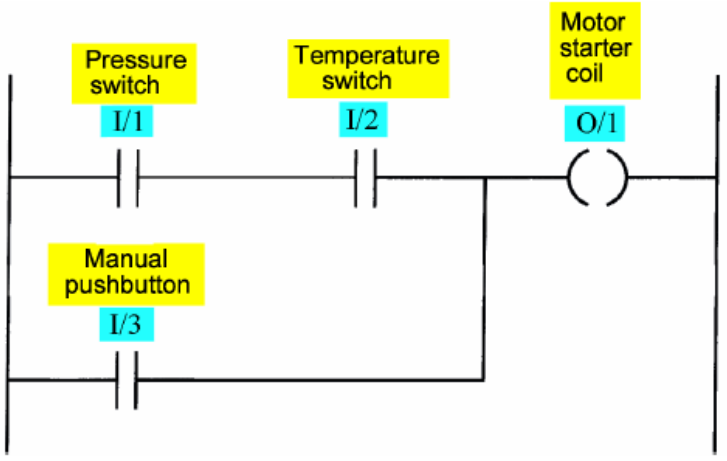
# PLC Operating Cycle




Each  can be though of as a set of normally open contacts

The  can be considered to represent a coil that, when energized, will close a set of contacts.

# PLC Operating Cycle




Coil O/1 is energized when contacts I/1 and I/2 are closed or when contact I/3 is closed. Either of these conditions provides a continuous path from left to right across the rung that includes the coil.

# The RUN operation for the process control scheme for the example

It can be described by the following sequence of events:

1. The pressure switch, temperature switch, and pushbutton inputs are examined and their status is recorded in the controller's memory.
2. Closed contact is recorded in memory as logic 1 and an open contact as logic 0.
3. The ladder diagram is evaluated, with each internal contact given an OPEN or CLOSED status according to its recorded 1 or 0 state.

4. When the states of the input contacts provide logic continuity from left to right across the rung, the output coil memory location is given a logic 1 value and the output module interface contacts will close.

5. When there is no logic continuity of the program rung, the output coil memory location is set to logic 0 and the output module interface contacts will be open.

6. The completion of one cycle of this sequence by the controller is called a *scan.* The scan time, the time required for one full cycle, provides a measure of the speed of response of the PLC.

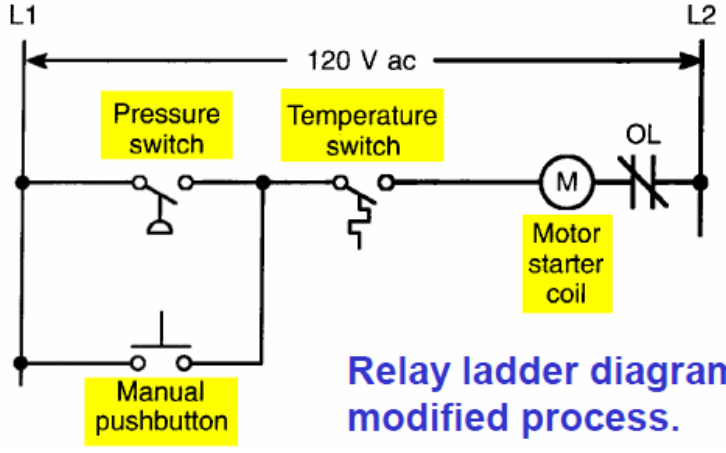
7. Generally, the output memory location is updated during the scan but the actual output is not updated until the end of the program scan during the I/O scan.

# Installation

If Allen-Bradley Pico controller is used, it  equipped with 8 inputs and 4 outputs is used to control and monitor the process. Installation can be summarized as follows:

- Fused power lines, of the specified voltage type and level, are connected to the controller's L1 and L2 terminals.
- The pressure switch, temperature switch, and pushbutton field input devices are hardwired between L1 and controller input terminals I1, I2, and I3, respectively.
- The motor starter coil connects directly to L2 and in series with Q1 relay output contacts to L1.
- The ladder logic program is entered using the front keypad and LCD display.

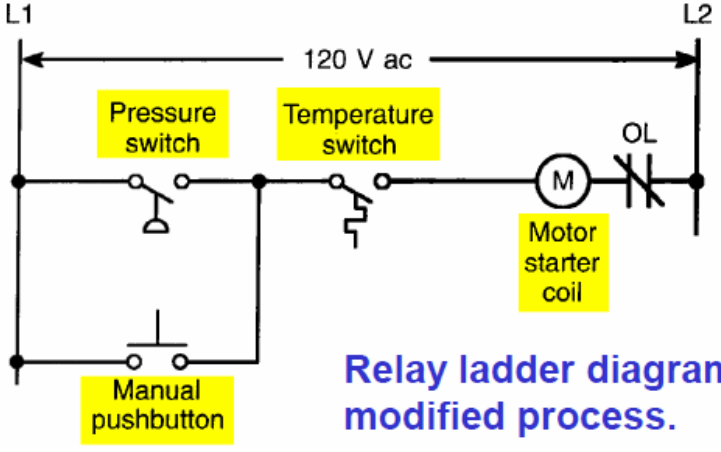
# Modifying A PLC Program




**Relay ladder diagram for modified process.**

The change requires that the manual pushbutton control should be permitted to operate at any pressure but not unless the specified temperature setting has been reached.

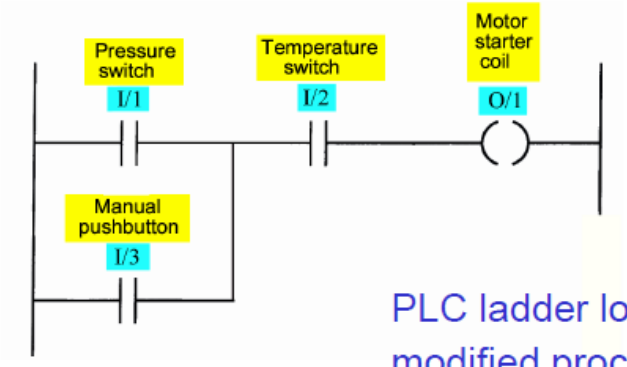
# Modifying A PLC Program




**Relay ladder diagram for modified process.**

If a relay system were used, it would require some rewiring of the system, as shown, to achieve the desired change.

# Modifying A PLC Program




PLC ladder logic diagram for modified process.

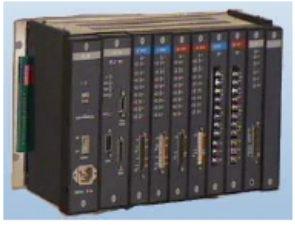
If a PLC is used, no rewiring is necessary!

The inputs and outputs are still the same.

All that is required is to change the PLC program

# PLCs versus Computers

# PLCs Versus Personal Computers



**Same basic architecture**



**PLC**

- Operates in the industrial environment
- Is programmed in relay ladder logic
- Has no keyboard, CD drive, monitor, or disk drive
- Has communications ports, and terminals for input and output devices

**PC**

- Capable of executing several programs simultaneously, in any order
- Some manufacturers have software and interface cards available so that a PC can do the work of a PLC

## PC Based Control Systems

Advantages

- Lower initial cost
- Less proprietary hardware and software required
- Straightforward data exchange with other systems
- Speedy information processing
- Easy customization

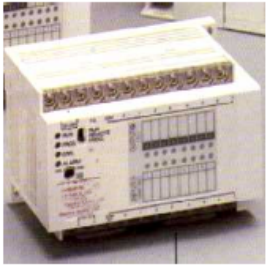


# PLC Size Classification

Criteria
- Number of inputs and outputs (I/O count)
- Cost
- Physical size

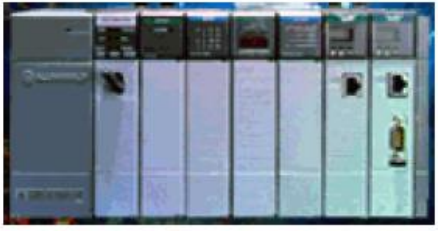


Nano PLC
- Smallest sized PLC
- Handles up to 16 I/O points



Micro PLC
- Handles up to 32 I/O points

## PLC Size Classification



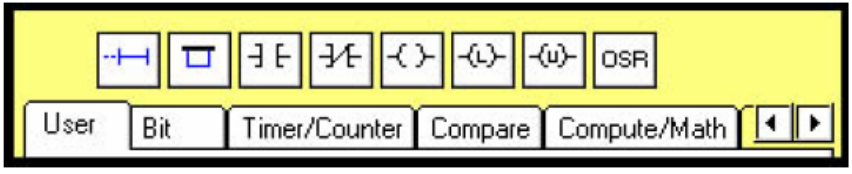
Allen-Bradley SLC-500 Family
- Handles up to 960 I/O points



Allen-Bradley PLC-5 Family
- Handles several thousand I/O points

# PLC Instruction Set

The instruction set for a particular PLC type lists the different types of instructions supported.




An instruction is a command that will cause a PLC to perform a certain predetermined operation.

# Typical PLC Instructions

| | |
|---|---|
| XIC (Examine ON) . . . . . | Examine a bit for an ON condition |
| XIO (Examine OFF) . . . . . | Examine a bit for an OFF condition |
| OTE (Output Energize) . . . . | Turn ON a bit (non retentive) |
| OTL (Output Latch) . . . . . | Latch a bit (retentive) |
| OTU (Output Unlatch) . . . . | Unlatch a bit (retentive) |
| TOF (Timer Off-Delay) . . . . . | Turn an output ON or OFF after its rung has been OFF a preset time interval |
| TON (Timer On-Delay) . . . . | Turn an output ON or OFF after its rung has been ON for a preset time interval |
| CTD (Count Down) . . . . . | Use a software counter to count down from a specified value |
| CTU (Count Up) . . . . . | Use a software counter to count up to a specified value |

- The  PLC is designed to operate in the industrial environment with wide ranges of ambient temperature and humidity and electrical noise
- The PLC is programmed in relay ladder logic or other easily learned languages.
- Most PLCs, on the other hand, execute a single program in an orderly and sequential fashion from first to last instruction.
- PLC control systems have been designed to be
    - easily installed and maintained.
    - Troubleshooted

**Software**

Software associated with a PLC but written and run on a personal computer falls into the following two broad
categories:

- PLC software that allows the user to program and document gives the user the tools to write a PLC program—using ladder logic or another programming language—and document or explain the program in as much detail as is necessary.
- PLC software that allows the user to monitor and control the process is also called a *human machine interface (HMI).*

Programmable automation controllers

- Blending the advantages of PLC-style control with that of PC-based systems



- Programmable automation controllers combine PLC ruggedness with PC functionality.

## PLC Size and Application

- categorizing PLC

  - ❑ functionality
  - ❑ number of inputs and outputs
  - ❑ cost
  - ❑ physical size

| | |
|---|---|
| Nano | less than 15 I/O points |
| Micro | (15 to 128 I/O points) |
| Medium | (128 to 512 I/O points) |
| large | (over 512 I/O points). |

# Types of PLC application

- **A *single ended* or stand-alone PLC application**
  - It involves one PLC controlling one process
  - It would not be used for communicating with other computers or PLCs.

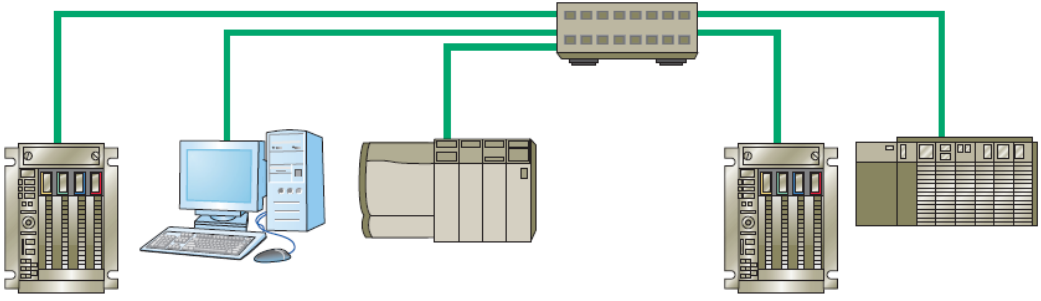


**A *multitask* PLC application**

- It involves one PLC controlling several processes.

## *Control management* PLC application

It involves one PLC controlling several others



**Typical PLC Instruction**

| Instruction | Operation |
|---|---|
| XIC (Examine ON) | Examine a bit for an ON condition |
| XIO (Examine OFF) | Examine a bit for an OFF condition |
| OTE (Output Energize) | Turn ON a bit (nonretentive) |
| OTL (Output Latch) | Latch a bit (retentive) |
| OTU (Output Unlatch) | Unlatch a bit (retentive) |
| TOF (Timer Off-Delay) | Turn an output ON or OFF after its rung has been OFF for a preset time interval |
| TON (Timer On-Delay) | Turn an output ON or OFF after its rung has been ON for a preset time interval |
| CTD (Count Down) | Use a software counter to count down from a specified value |
| CTU (Count Up) | Use a software counter to count up to a specified value |

The *instruction set* for a particular PLC lists the different types of instructions supported. Typically, this ranges
from 15 instructions on smaller units up to 100 instructions on larger, more powerful units

Factors affecting the memory size needed for a particular PLC installation

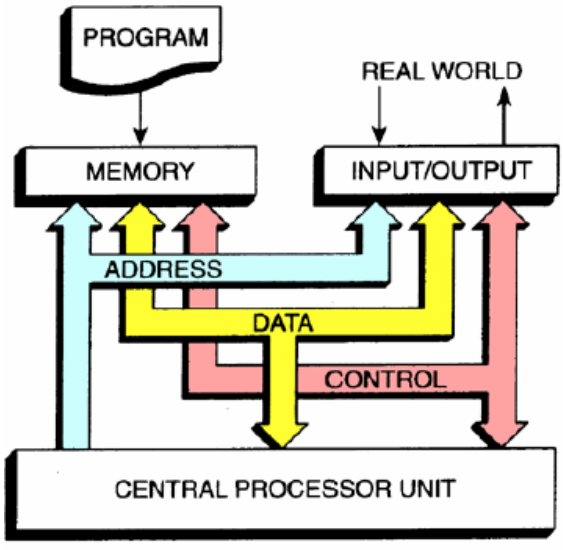
- Number of I/O points used
- Size of control program
- Data-collecting requirements

- Supervisory functions required
- Future expansion

# PLC

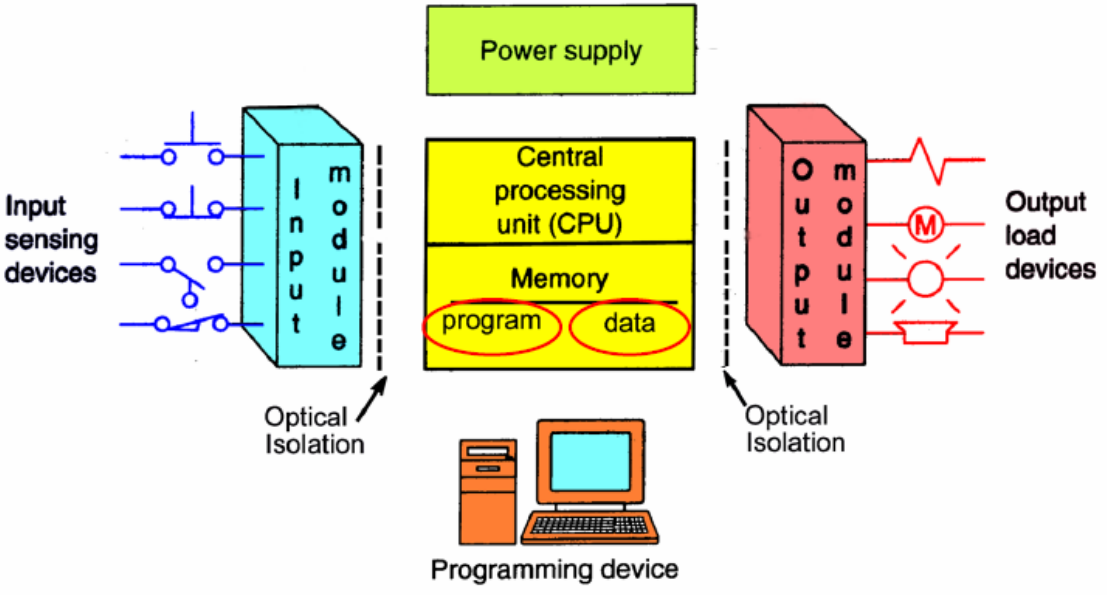
Lecture 2

# PLC Programing




The structure of a PLC is based on the same principles as those employed in computer architecture.

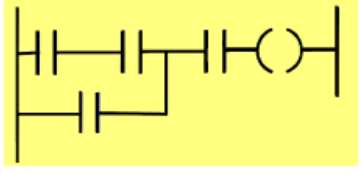
# PLC System


Power supply
Central processing unit (CPU)
Memory
program
data
Input module
Output module
Input sensing devices
Output load devices
Optical Isolation
Optical Isolation
M
Programming device

# PLC Programming Languages

**The term *PLC programming language* refers to the method by which the user communicates information to the PLC.**

**The three most common language structures are: ladder diagram language, Boolean language, and functional chart.**

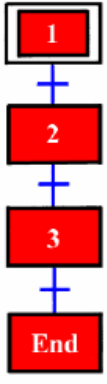


Ladder diagram language

| | |
|---|---|
| START | PB1 |
| AND | CR1 |
| OR | LS1 |
| AND NOT | CR2 |
| OUT | SOL |

Boolean equation: $Y = [(AB) + C]\overline{D}$

Boolean language




Functional chart

# Processor Memory Organization

**The memory of a PLC is organized by types.**

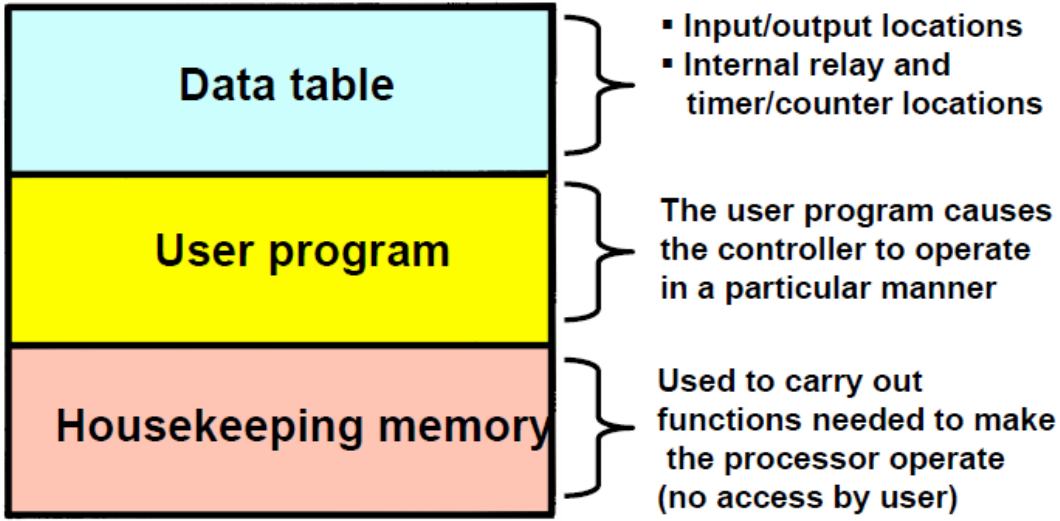
**The memory space can be divided into two broad categories:**

***program* and *data* memory.**

**Advanced ladder logic *functions* allow controllers to perform calculations, make decisions and do other complex tasks. Timers and counters are examples of ladder logic functions. They are more complex than basic inputs contacts and output coils and rely heavily upon data stored in the memory of the PLC.**

# Memory Map

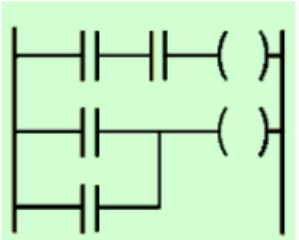
A memory map can be used to show how memory is organized in a PLC.

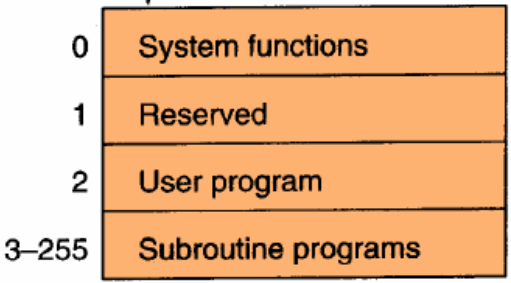


# Program Files




The user program will account for most of the memory of a PLC system.

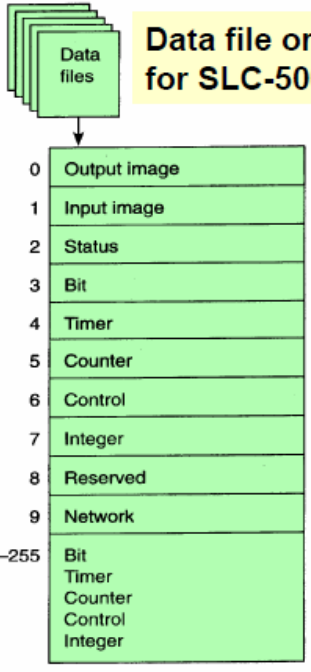





Program file organization for SLC-500 controller.

- Program files contain the logic controlling machine operation.
- This logic consists of instructions that are programmed in a ladder logic format.

# Data Files

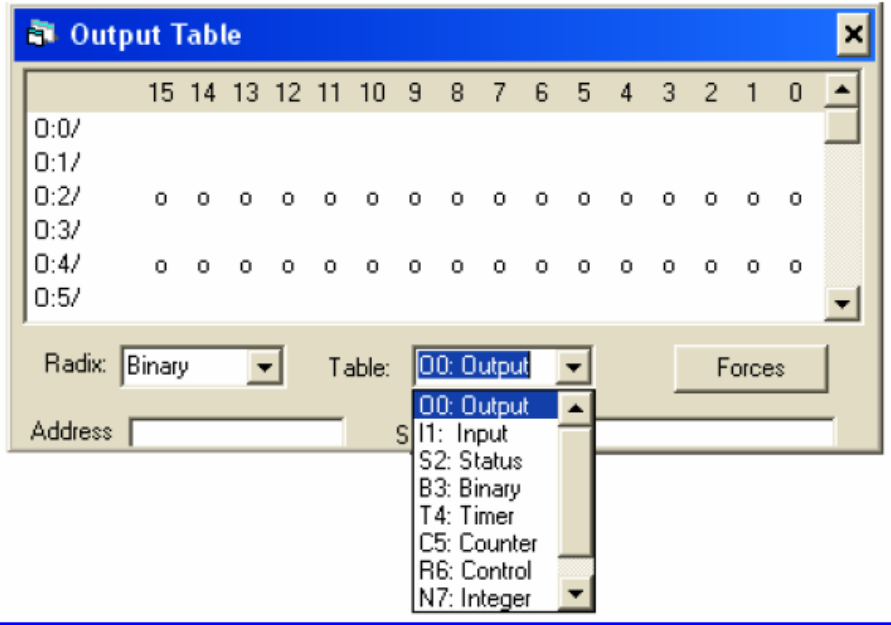



**Data file organization for SLC-500 controller.**

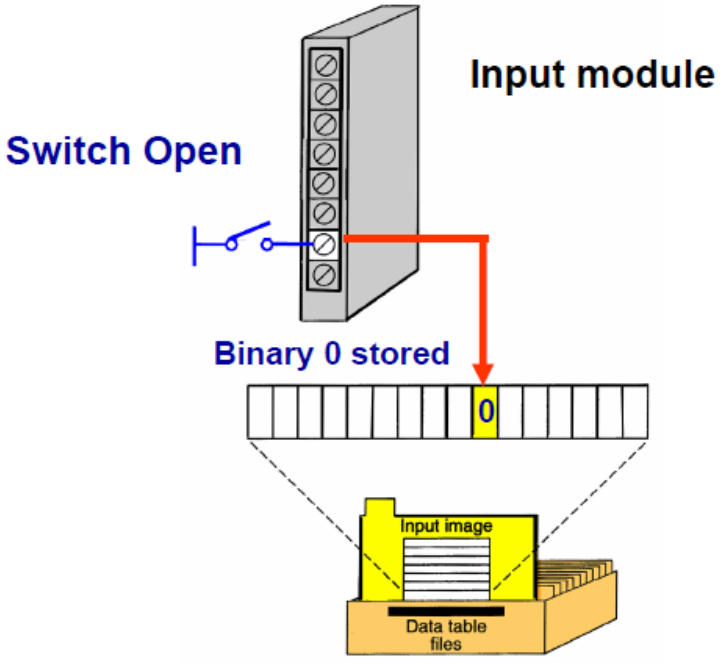
**The data file portion of memory stores input and output status, processor status, the status of various bits and numerical data.**

# Data Files

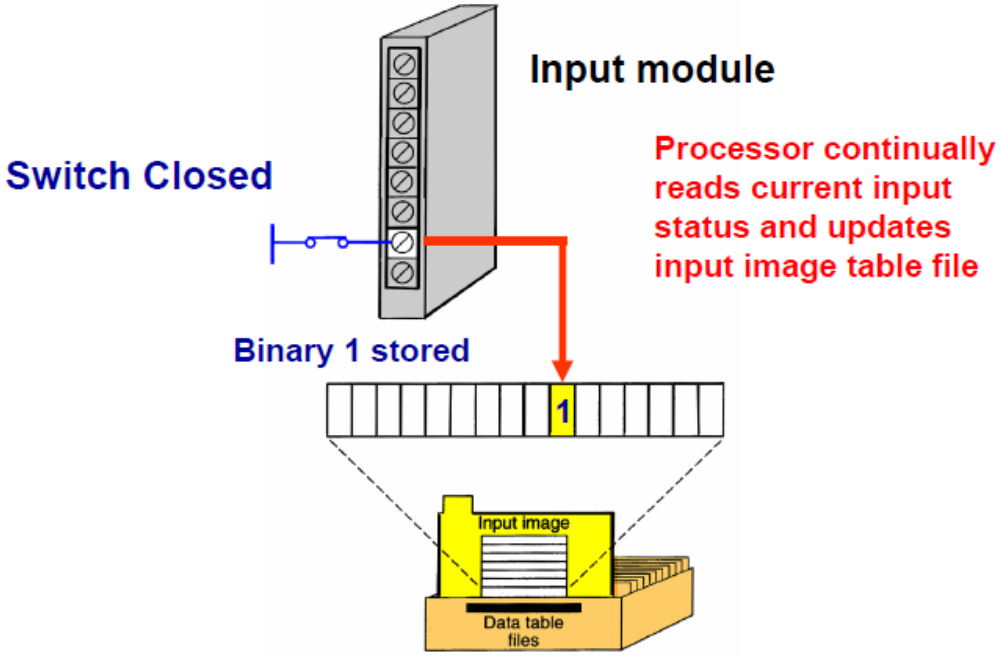
## Data files are organized by the type of data they contain



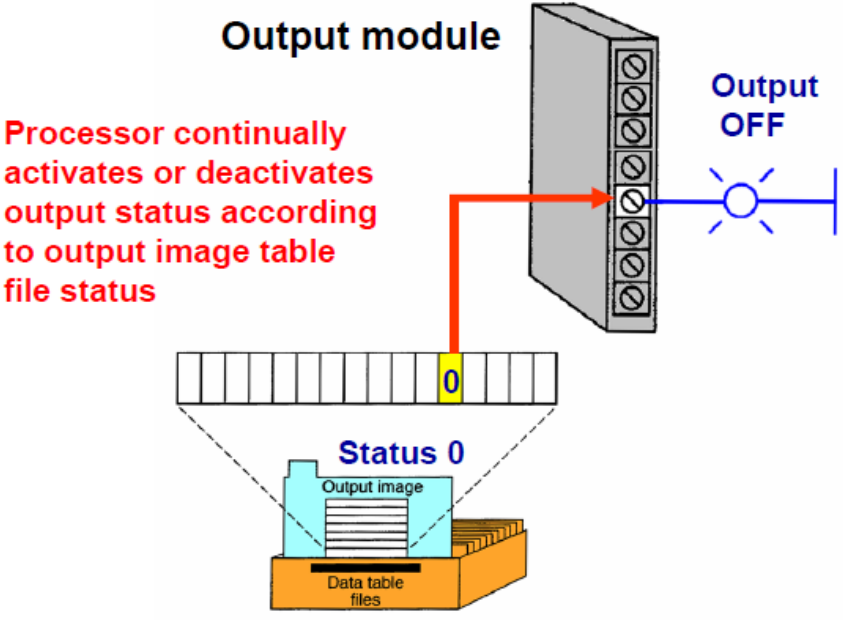
# Input Table File Operation



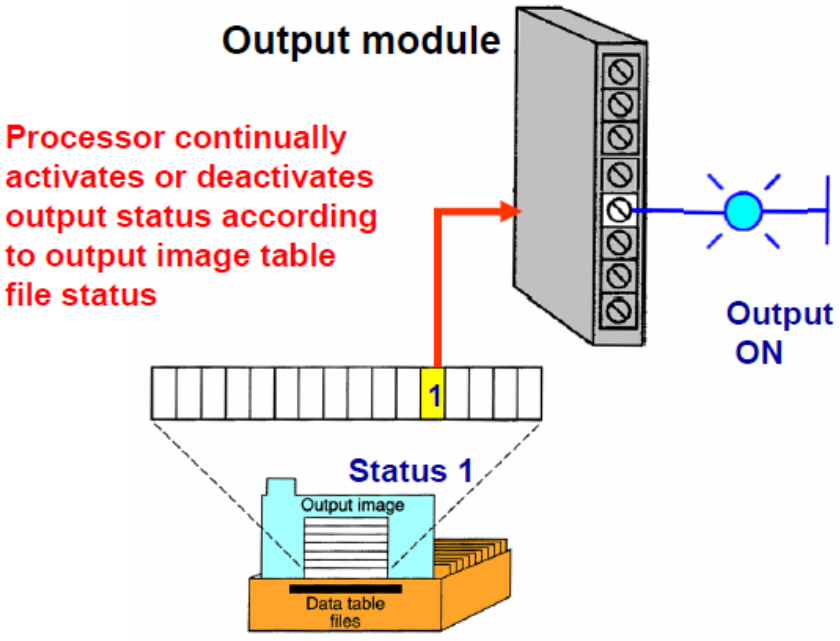
# Input Table File Operation



# Output Table File Operation

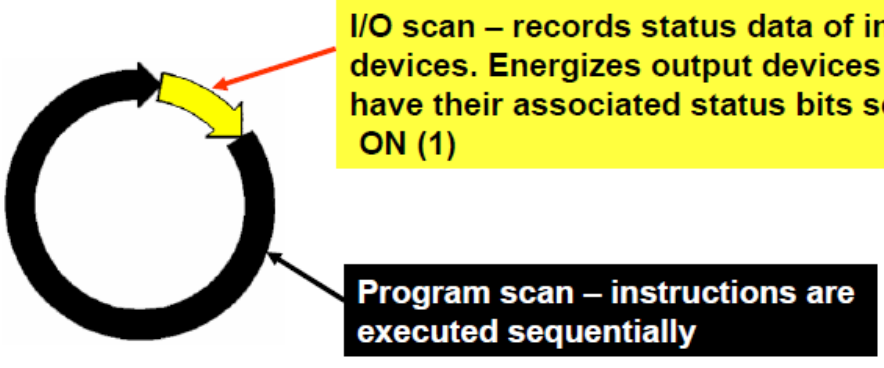


# Output Table File Operation



# Program Scan

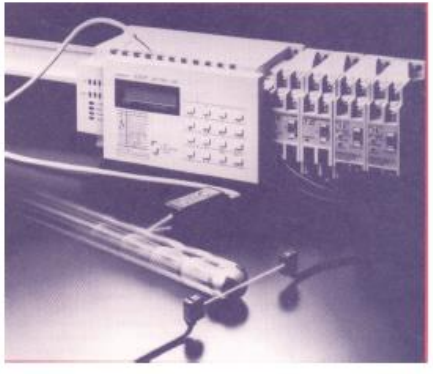
During each operating cycle, the processor reads all inputs, takes these values, and energizes or de-energizes the outputs according to the user program. This process is known as a *scan*.




Because the inputs can change at any time, the PLC must carry on this process continuously.

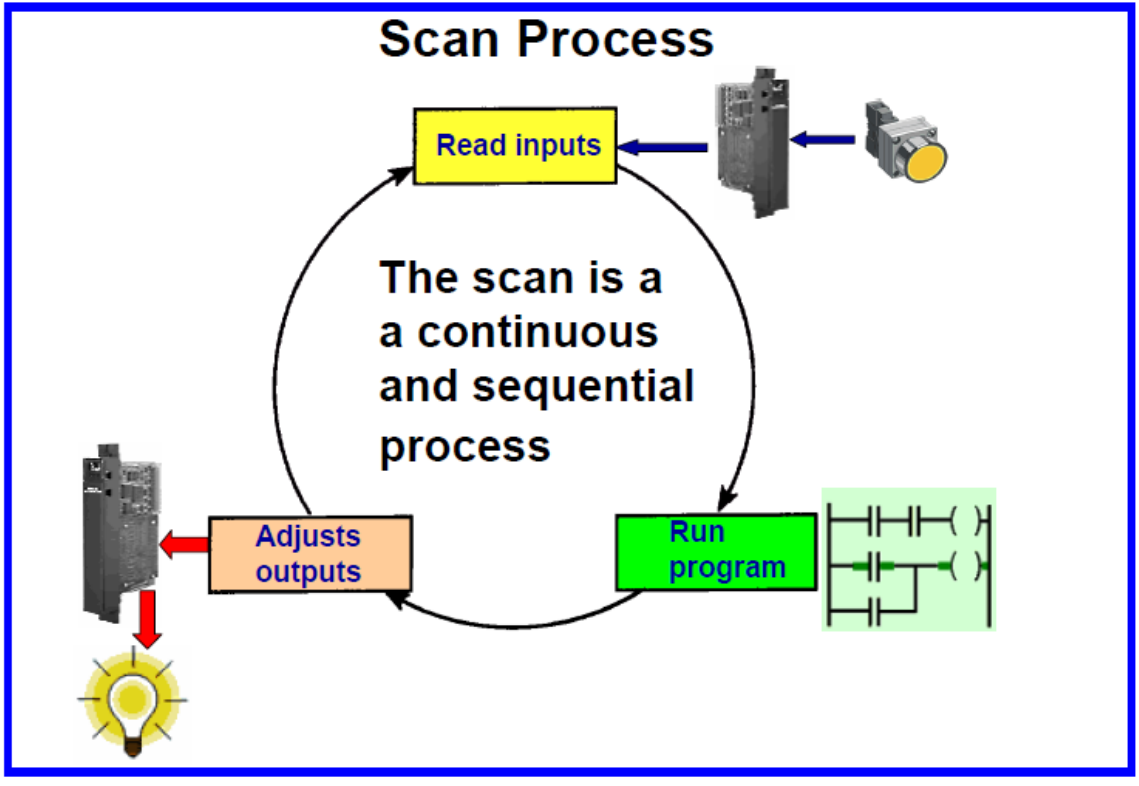
# Scan Process

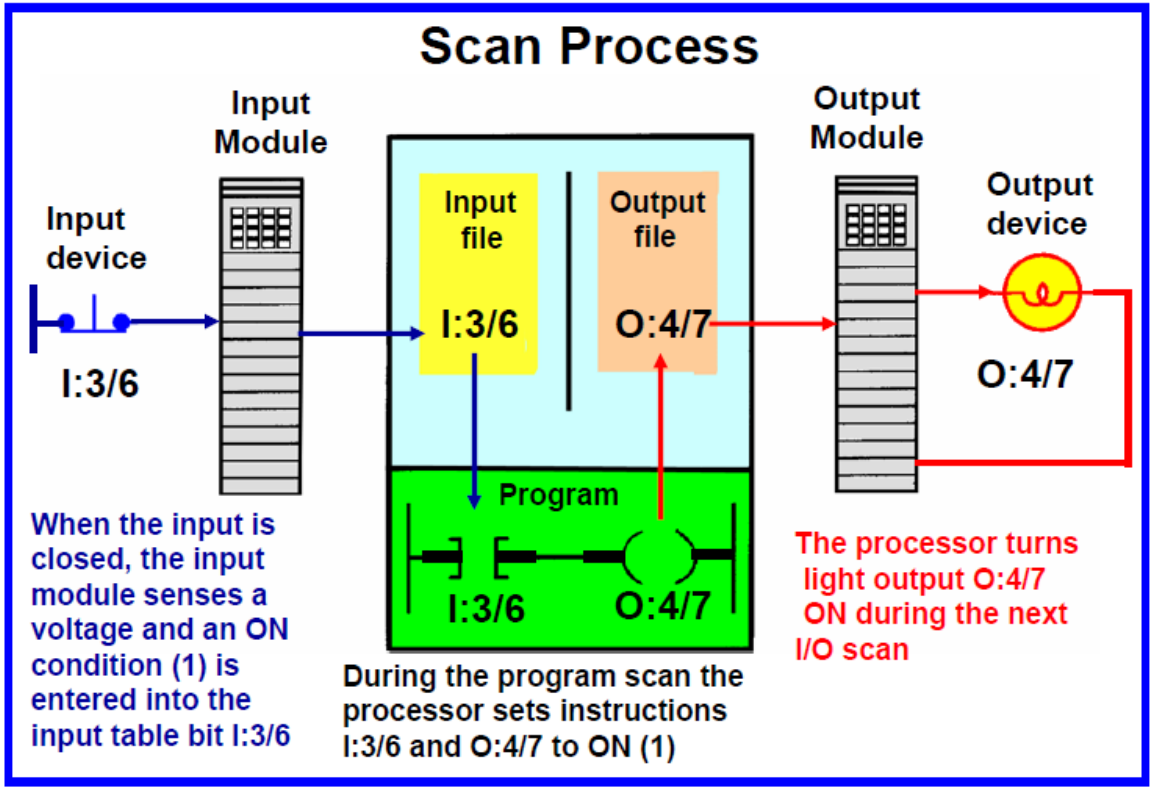
**The scan *time* indicates how fast the controller can react to changes in inputs. Scan times vary with computer model and program content, and length. If a controller has to react to an input signal that changes states twice during the scan time, it is is possible that the PLC will never be able to detect this change.**



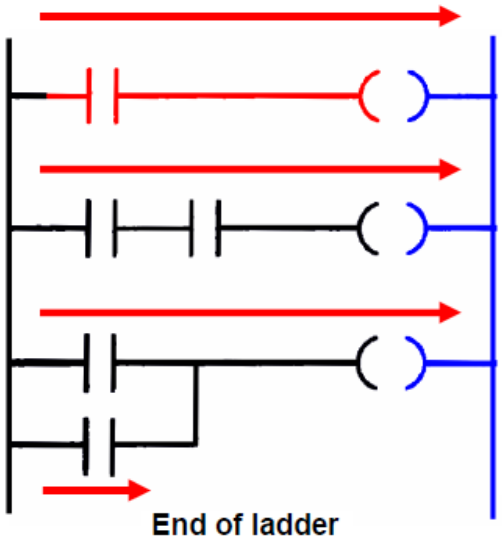
**Scan time may be a concern in high speed operations**

# Scan Process




Scan Process
Input Module
Input device
I:3/6
Input file
I:3/6
Output file
O:4/7
Output Module
Output device
O:4/7
Program
I:3/6
O:4/7
When the input is closed, the input module senses a voltage and an ON condition (1) is entered into the input table bit I:3/6
During the program scan the processor sets instructions I:3/6 and O:4/7 to ON (1)
The processor turns light output O:4/7 ON during the next I/O scan

# Scan Patterns




## Horizontal Scanning Order

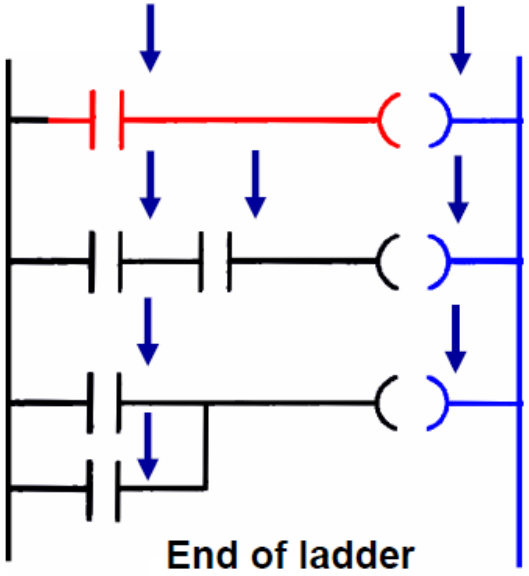
The processor examines input and output instructions from the first command, top left in the program, horizontally, rung by rung.

In addition to the program itself, the scan time is also dependent on the clock frequency of the processor!

# Scan Patterns

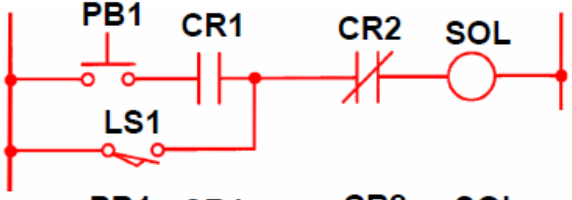
## Vertical Scanning Order

The processor examines input and output instructions from the first command, vertically, column by column and page by page. Pages are executed in sequence.

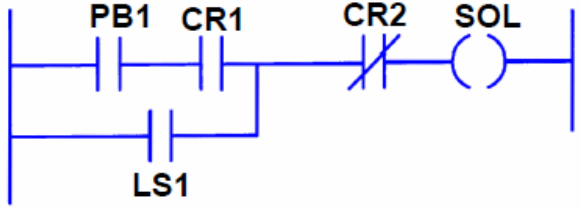



**Misunderstanding the way the PLC scans can cause programming bugs!**

# Comparing Programming Language




Relay Schematic




Equivalent ladder diagram language

| | |
|---|---|
| START | PB1 |
| AND | CR1 |
| OR | LS1 |
| AND NOT | CR2 |
| OUT | SOL |

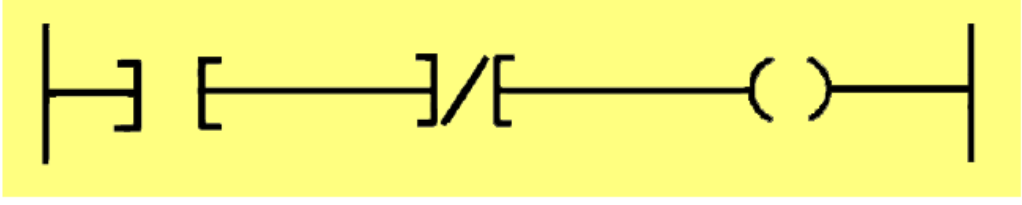
Boolean equation: $Y = [(AB) + C]\,\overline{D}$

Equivalent Boolean language

# Relay-Type Instructions

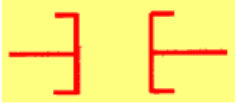
The ladder diagram language is basically a *symbolic* set of instructions used to create the controller program.

These ladder instructions symbols are arranged to obtain the desired control logic.



# Examine If Closed (XIC) Instruction

**Symbol**



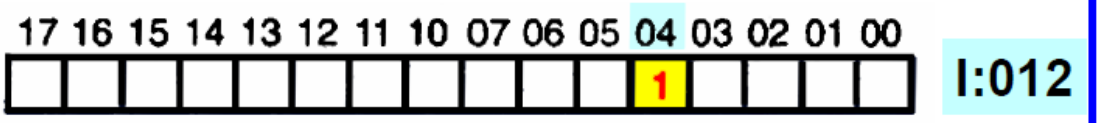
Analogous to the normally open relay contact. For this instruction we ask the processor to EXAMINE IF (the contact is) CLOSED (XIC)

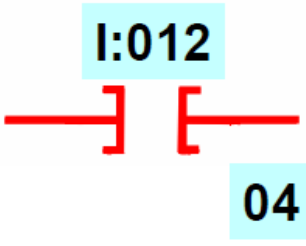
Typically represents any input. Can be a switch or pushbutton, a contact from a connected output, or a contact from an internal output.

Has a bit-level address which is examined for an ON condition.

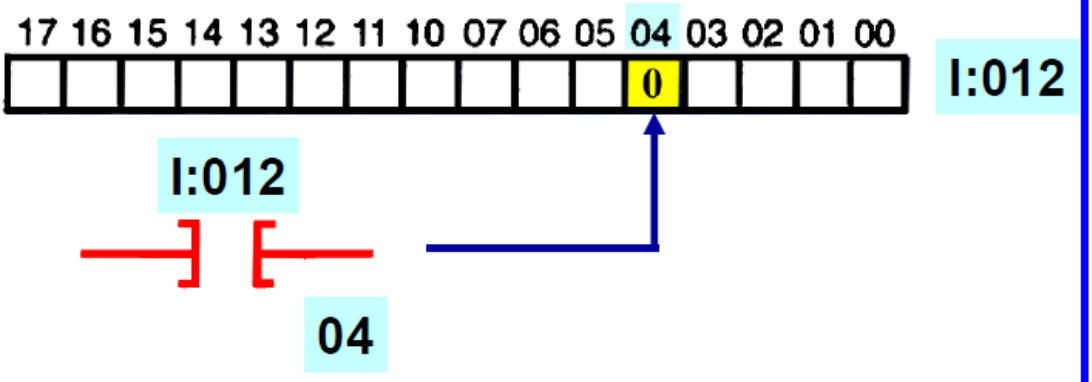
The status bit will be either 1 (ON) or 0 (OFF).

# Examine If Closed (XIC) Instruction



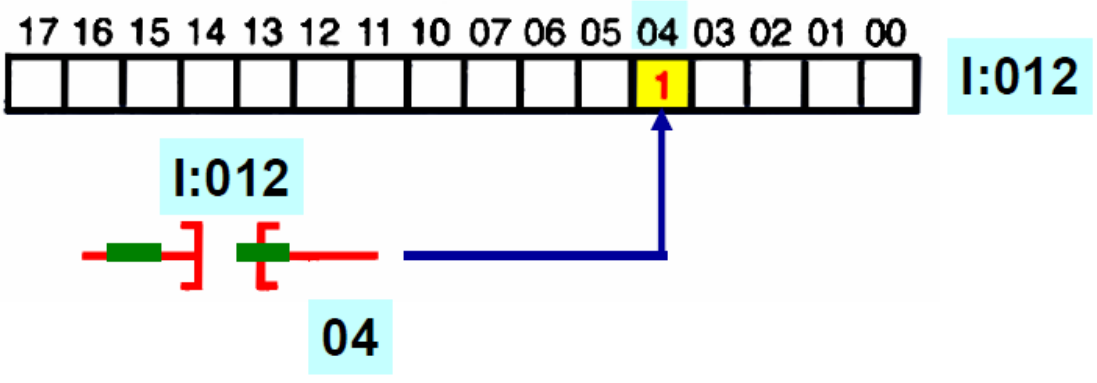



# Examine If Closed (XIC) Instruction




If the status bit is 0 (OFF), then the instruction is FALSE.

# Examine If Closed (XIC) Instruction




If the status bit is 1 (ON), then the instruction is TRUE.

# Examine If Open (XIO) Instruction

**Symbol**



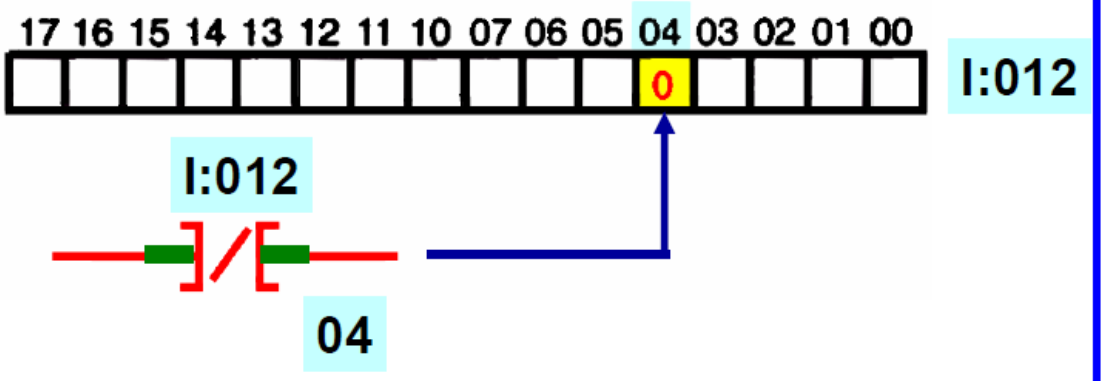
Analogous to the normally closed relay contact. For this instruction we ask the processor to EXAMINE IF (the contact is) OPEN (XIO).

Typically represents any input. Can be a switch or pushbutton, a contact from a connected output, or a contact from an internal output.

Has a bit-level address which is examined for an OFF condition.

The status bit will be either 1 (ON) or 0 (OFF).

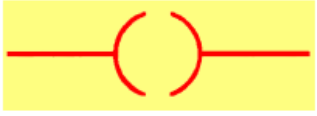
# Examine If Open (XIO) Instruction




If the status bit is 0 (OFF), then the instruction is TRUE.

# Output Energize (OTE) Instruction

**Symbol**

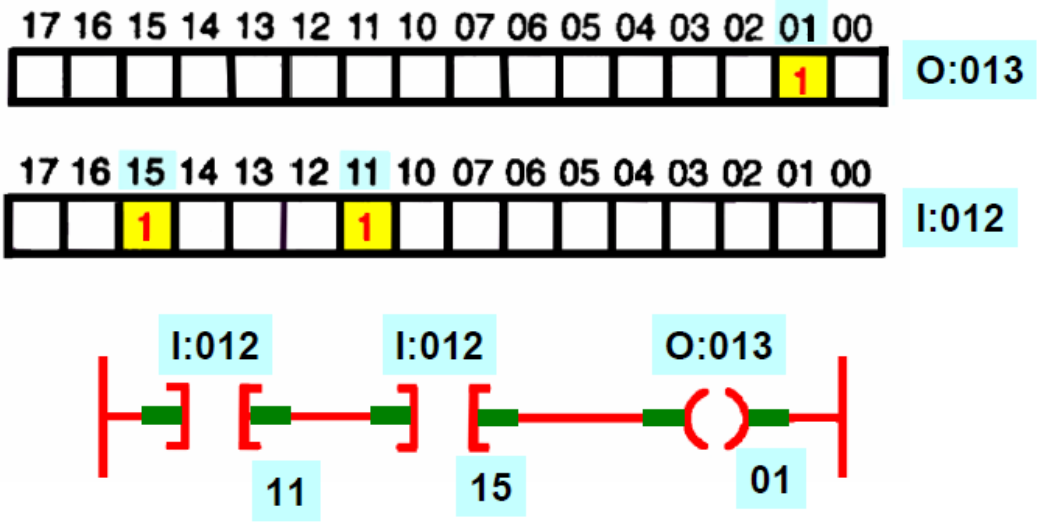


Analogous to the relay coil. The processor makes this instruction true (analogous to energizing a coil) when there is path of true XIC and XIO instructions in the rung.

Typically represents any output that is controlled by some combination of input logic. Can be a connected device or an internal output (internal relay).

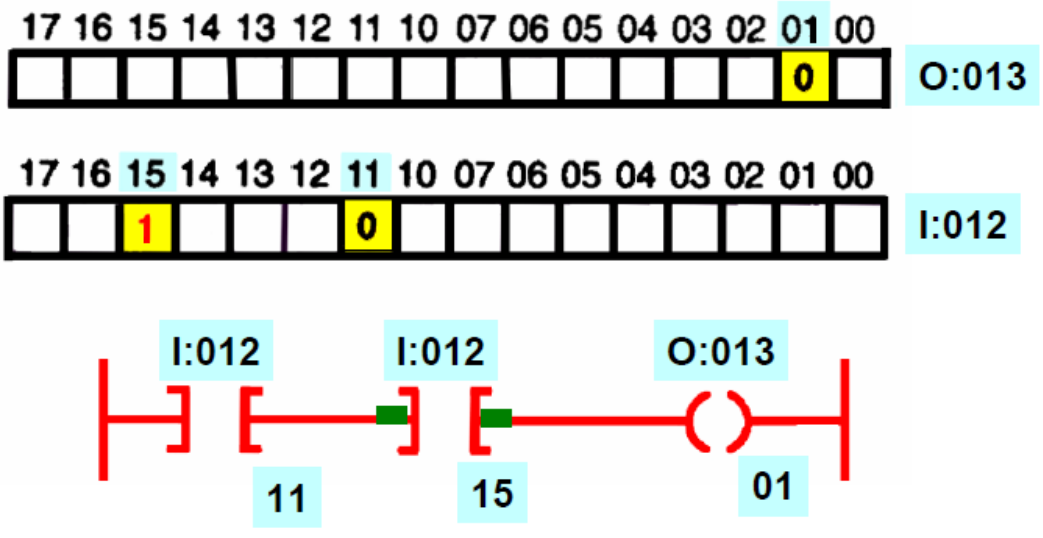
If any left-to-right path of input conditions is TRUE, the output is energized (turned ON).

# Output Energize (OTE) Instruction



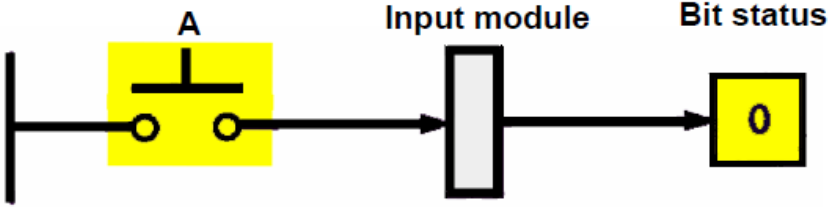

**OUTPUT ENERGIZE instruction - TRUE**

# Output Energize (OTE) Instruction

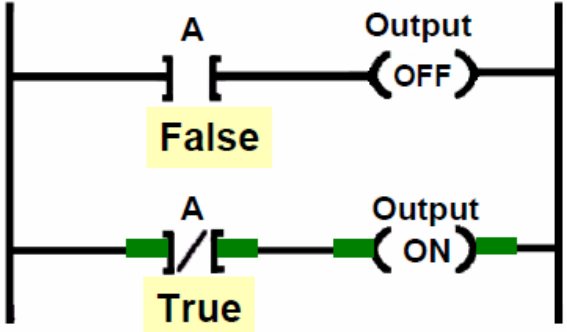



**OUTPUT ENERGIZE instruction - FALSE**

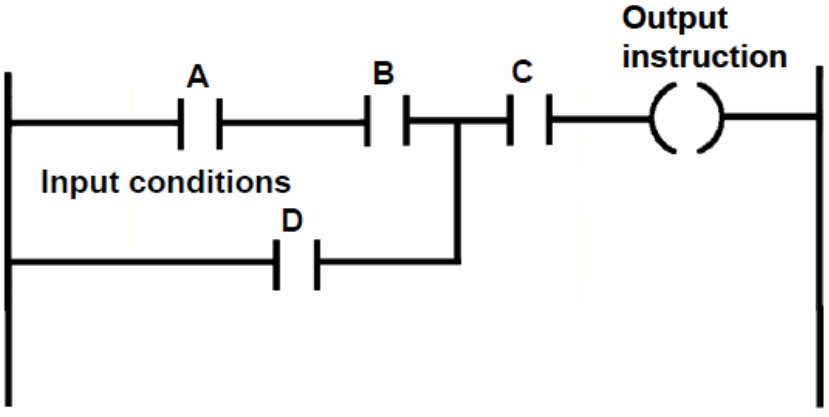
# Status Bit Examples




Button not actuated

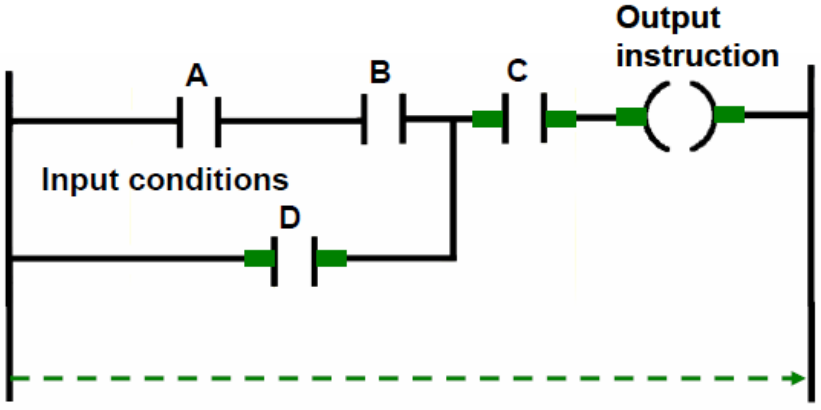


# Ladder Rung



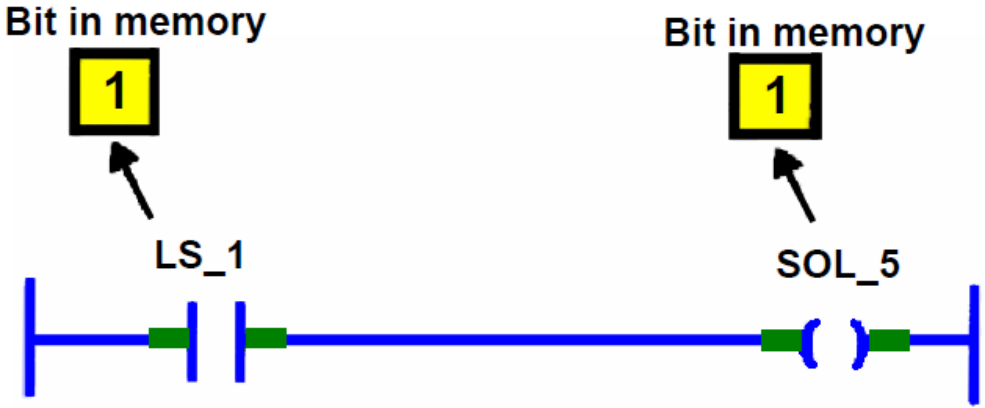

A ladder rung consists of a set of input conditions, represented by contact instructions, and an output instruction at the end of the rung, represented by the coil symbol.

# Ladder Rung



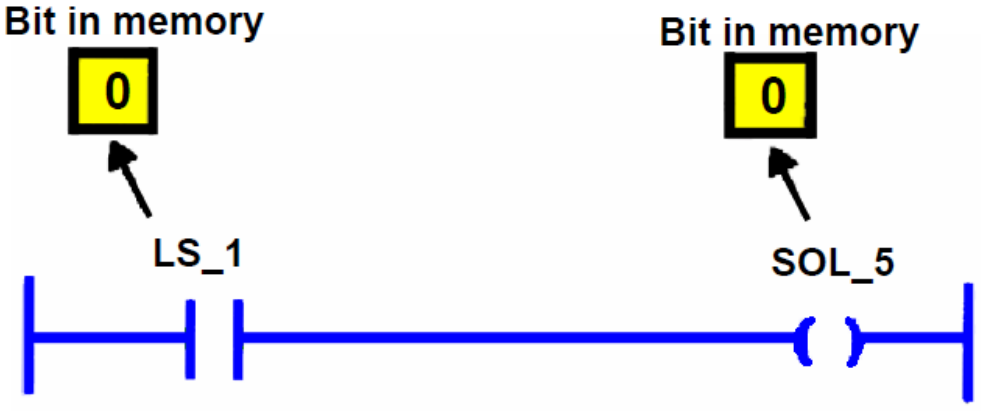

For an output to be activated or energized, at least *one* left-to-right path of contacts most be closed. A complete path is referred to as having *logic continuity*. When logic exists the rung condition is said to be TRUE.

# Rung Continuity



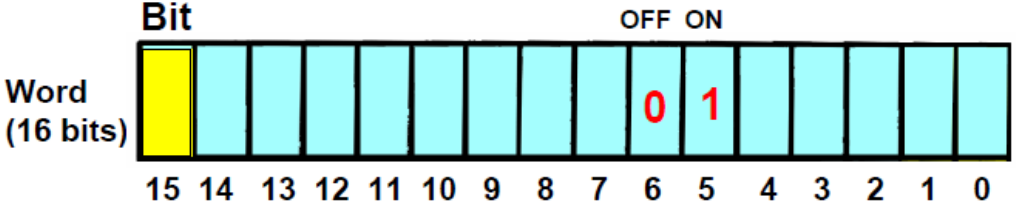

The Examine If Closed instruction is TRUE making the rung TRUE

# Rung Continuity

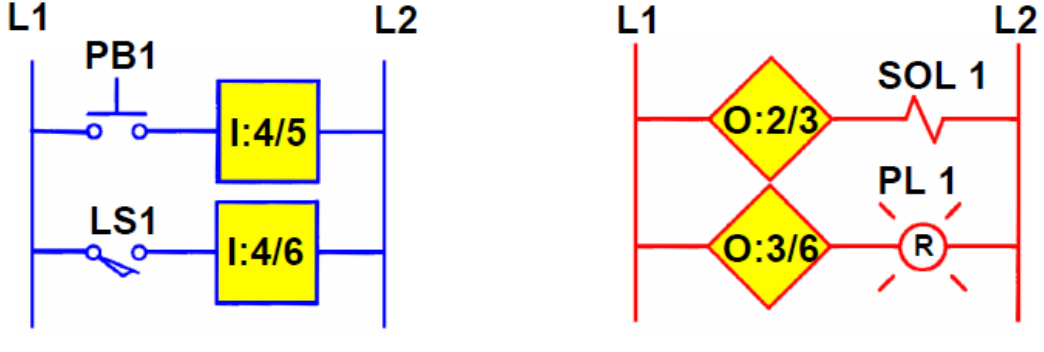



The Examine If Closed instruction is FALSE making the rung False

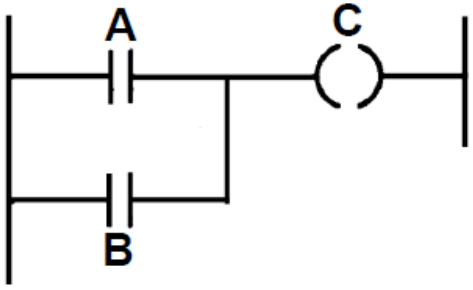
# Structure of A 16-Bit Word




## I/O Connection Diagram

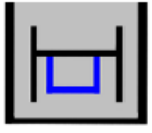


# Parallel Input Branch Instructions



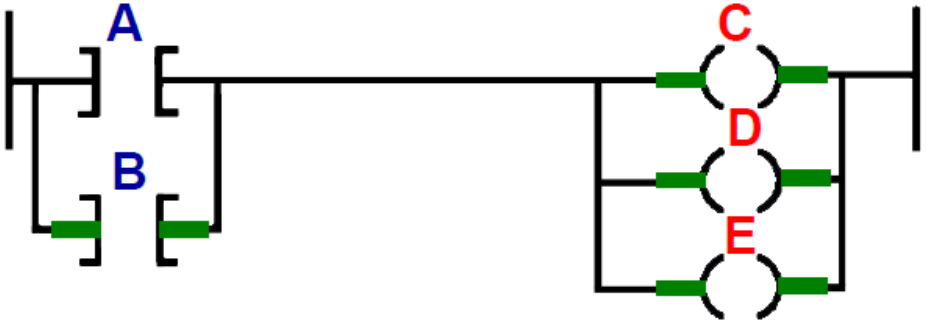

**Branch instructions are used to create parallel paths of input condition instructions. If at least one of these parallel branches forms a true logic path, the logic is enabled.**

## *Add A Branch*



Click this icon on the instruction toolbar to place a branch in your ladder logic.
If your cursor is on an instruction, the branch is placed immediately to the right of the instruction.
If your cursor is on the rung number, the branch is placed on the rung.

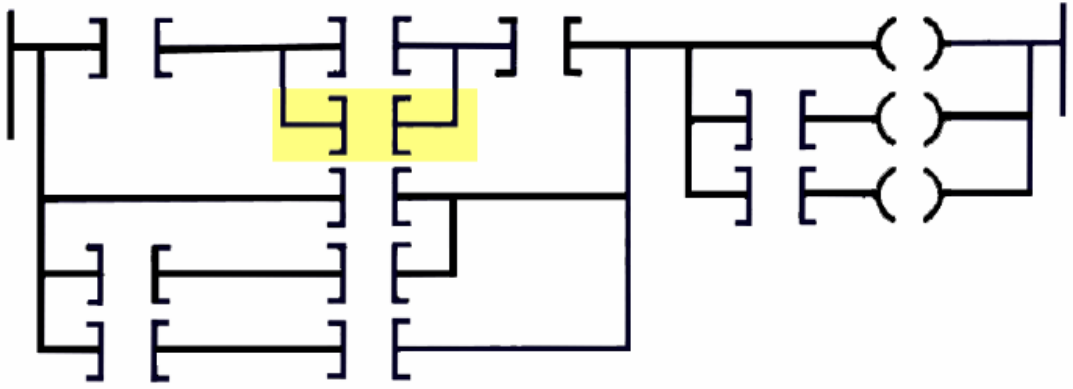
# Parallel Output Branching




On most PLC models, branches can be established at both the input and output portion of the rung.

With output branching, you can program parallel outputs on a rung to allow a true logic path to control multiple outputs.

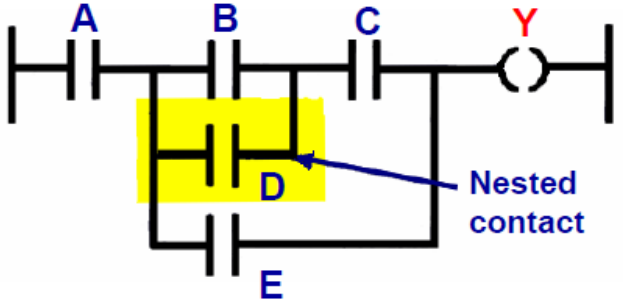
# Nested Input and Output Branches



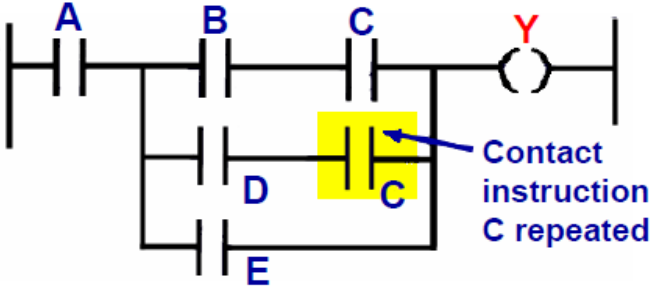
Input and output branches can be *nested* to avoid redundant instructions and to speed up the processor scan time.

A nested branch starts or ends within another branch.

# Nested Contact Program

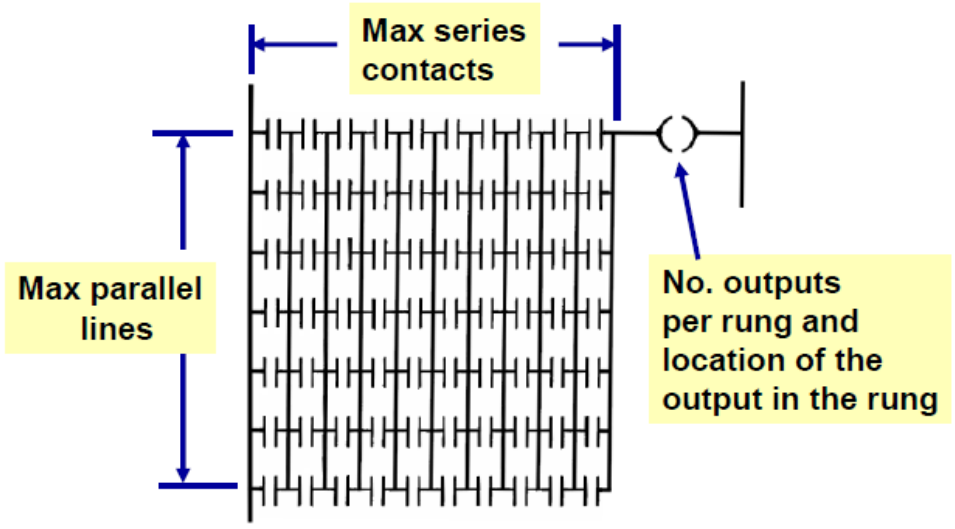



On some PLC models, the programming of a nested branch circuit cannot be done directly.



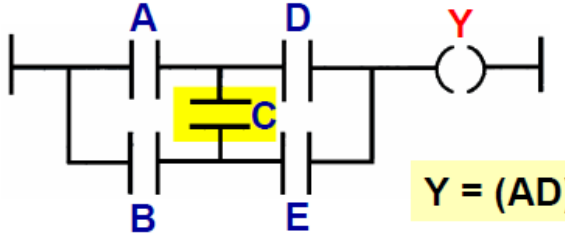

Reprogrammed to obtain the required logic.

# PLC Matrix Limitation Diagram



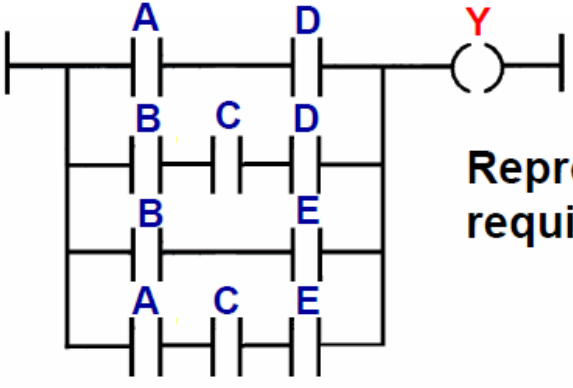

There may be limitations to the number of series contacts instructions, number of parallel lines, and the number of outputs and their location on the rung.

# Programming of Vertical Contacts



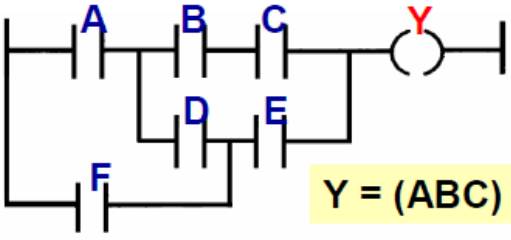

Original program

$Y = (AD) + (BCD) + (BE) + (ACE)$



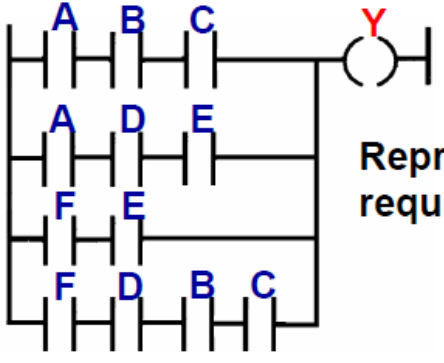

Reprogrammed to obtain the required logic

# Programming for Different Scan Patterns




Original program

Y = (ABC) + (ADE) + (FE) + (FDBC)




Reprogrammed to obtain the required logic

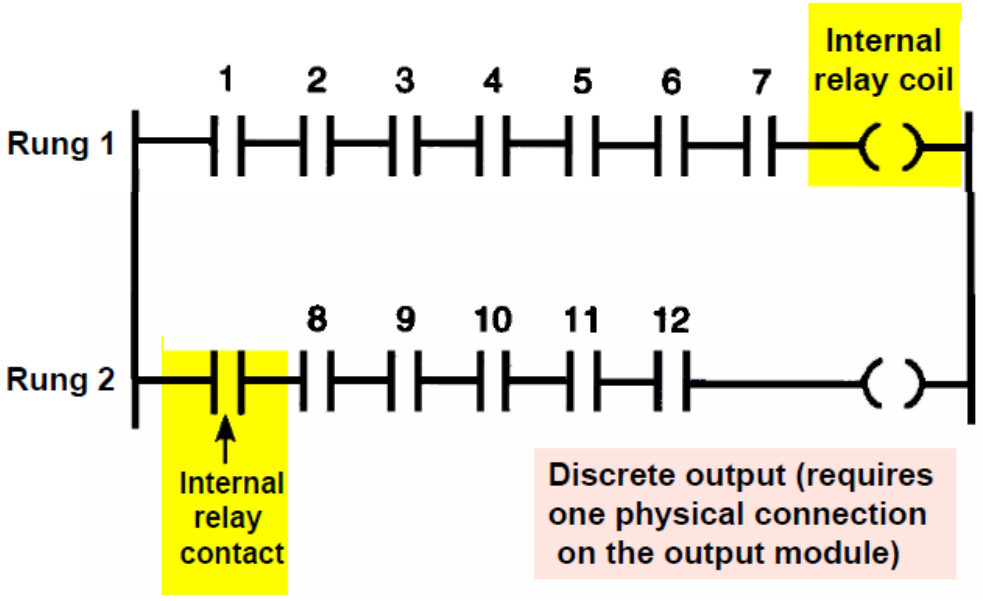
# Internal Control Relay

The internal output operates just as any other output that is controlled by programmed logic; however, the output is used strictly for internal purposes.

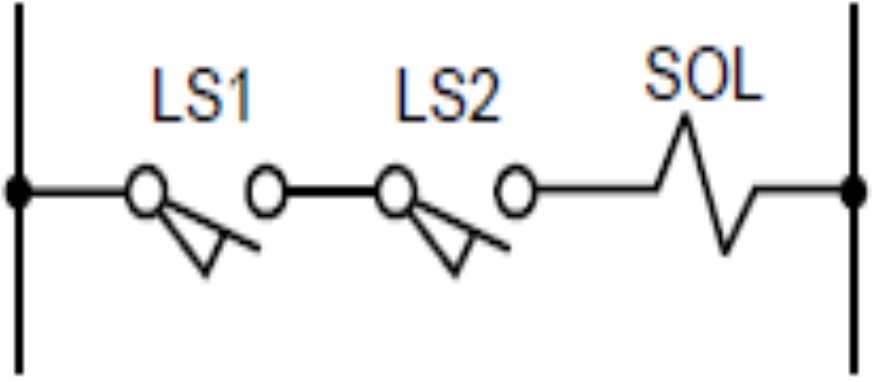
The internal output does not directly control an output device.

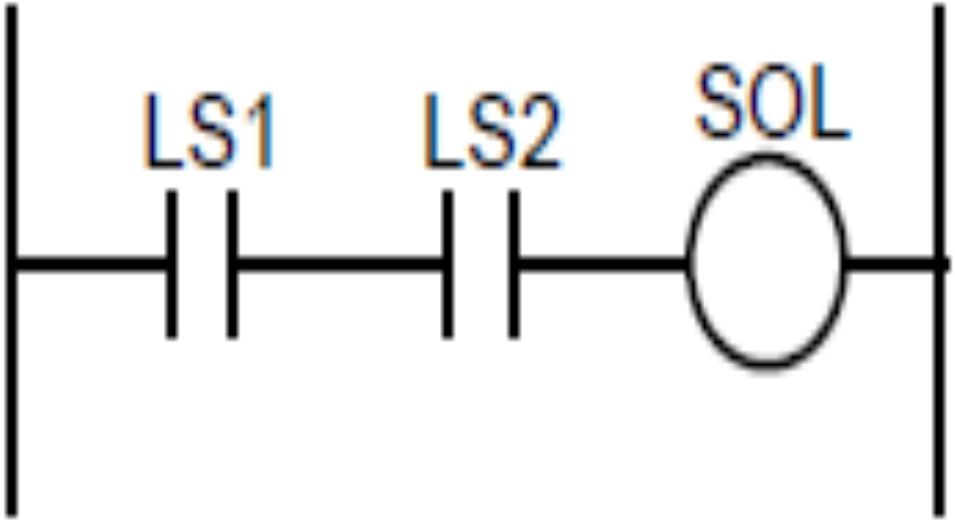
The advantage of using internal outputs is that there are many situations where an output instruction is required in a program, but no physical connection to a field device is needed. Their use in this type of instance can minimize output card requirements.

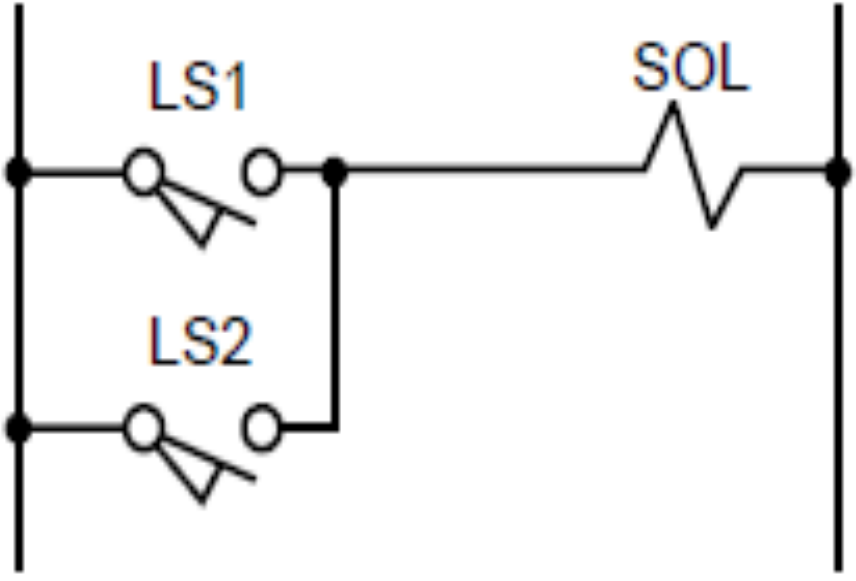
## Extending the Number of Series Contacts Using an Internal Control Relay

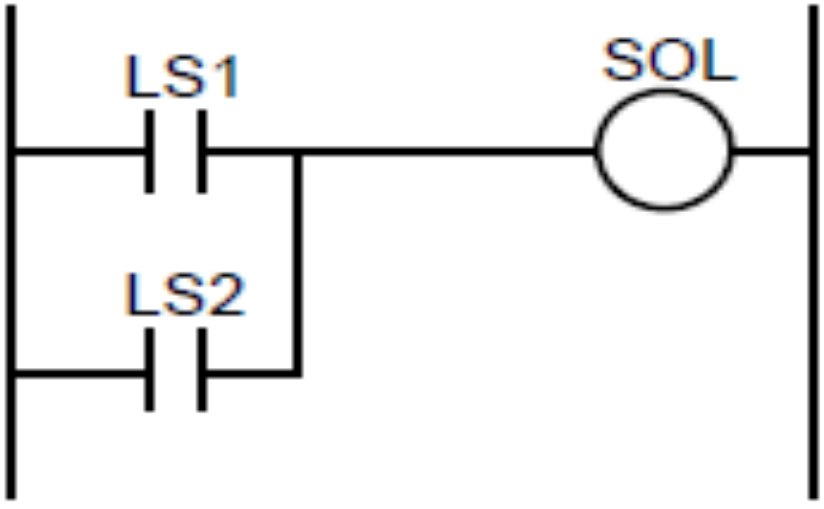


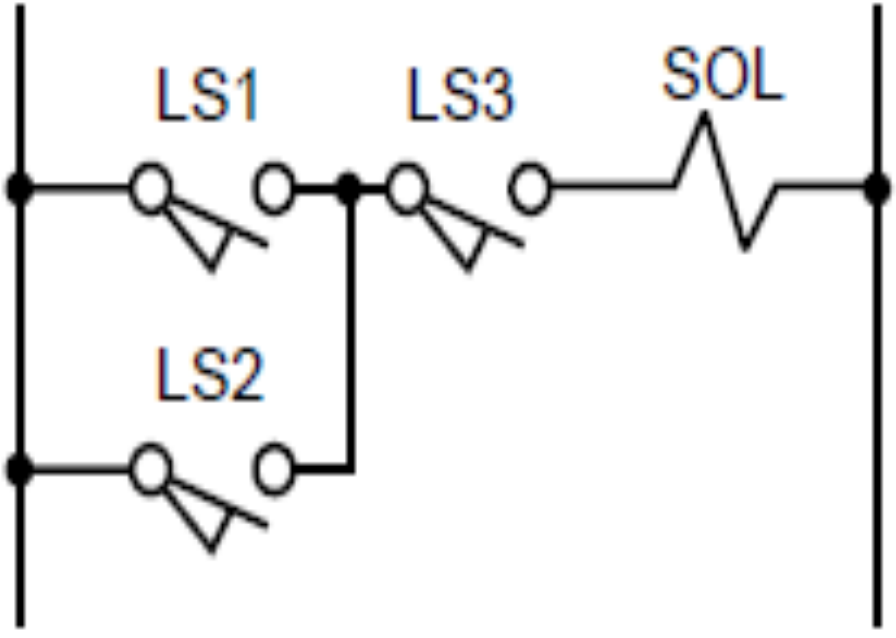
# Ladder diagram

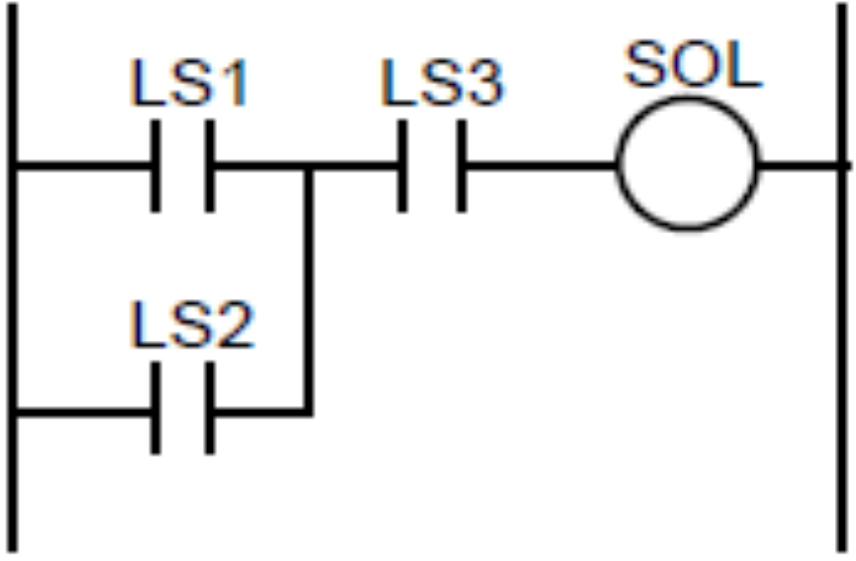


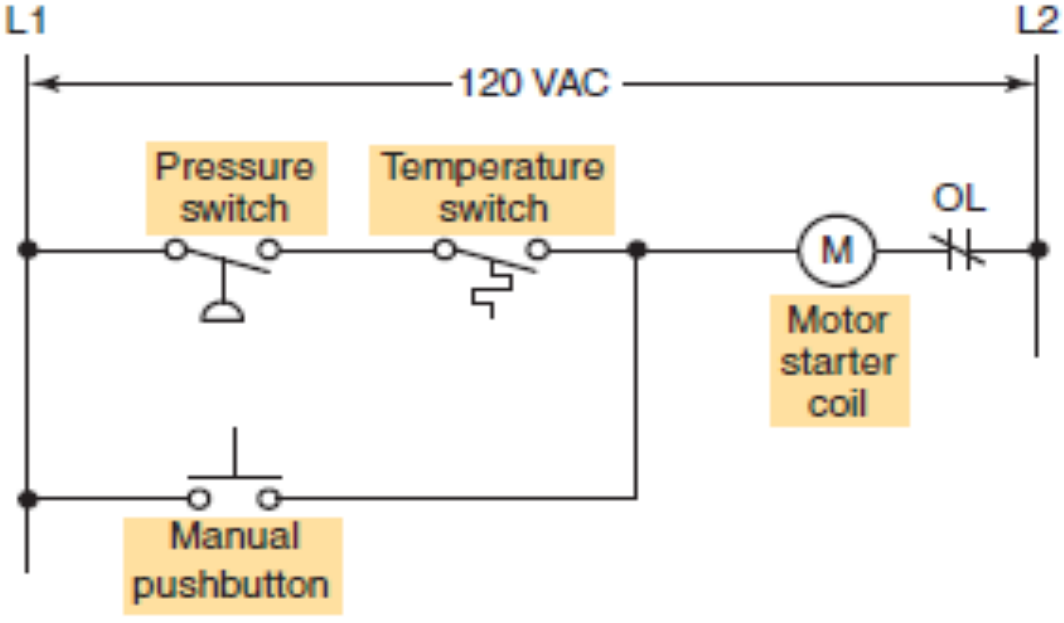

LS1
LS2
SOL

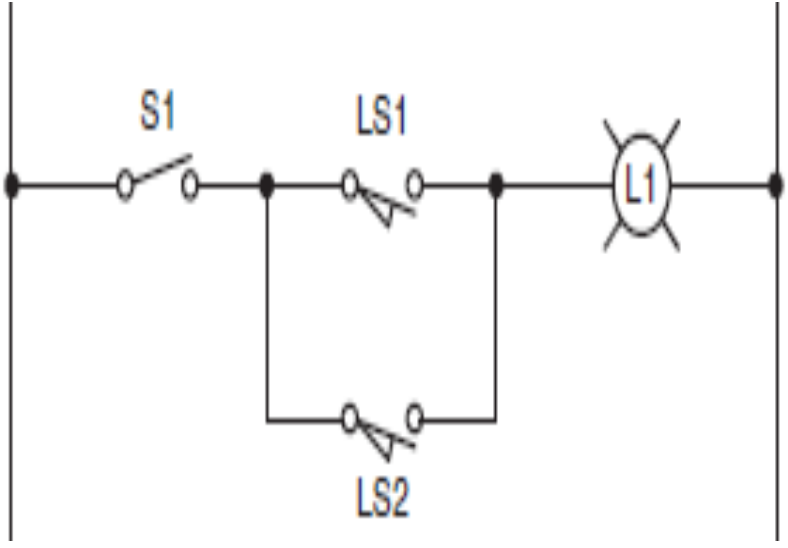

LS1
SOL
LS2

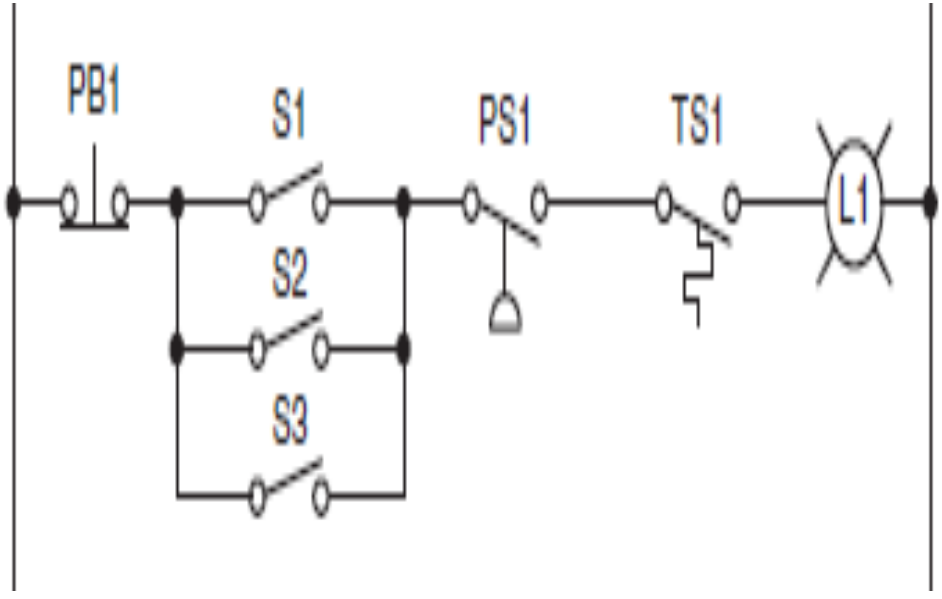

LS1
SOL
LS2

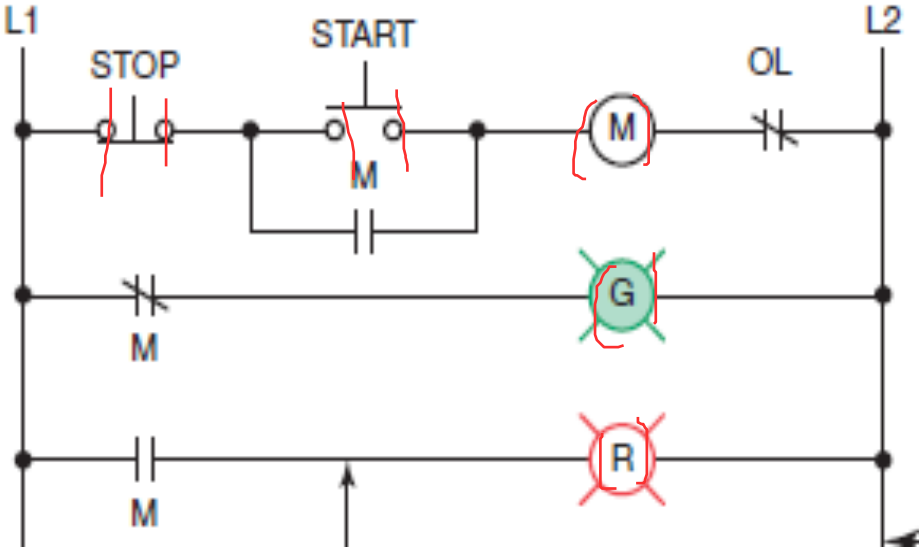

LS1
LS3
SOL
LS2

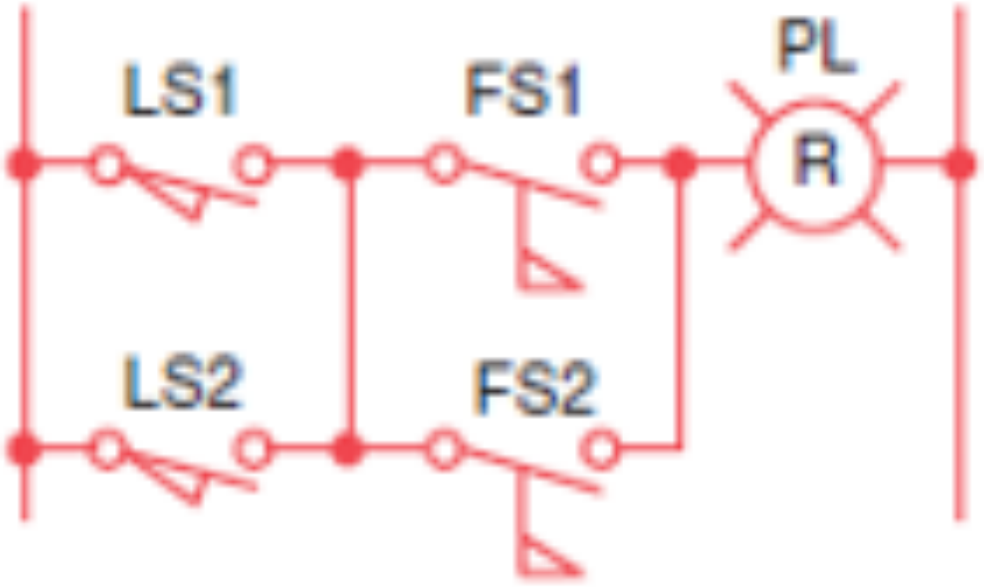

LS1
LS3
SOL
LS2

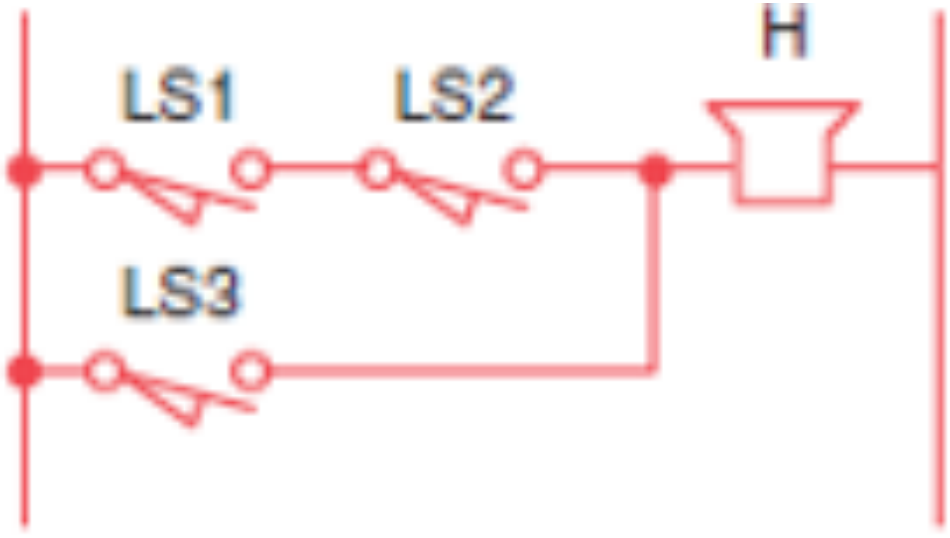

L1
L2
120 VAC
Pressure switch
Temperature switch
OL
M
Motor starter coil
Manual pushbutton

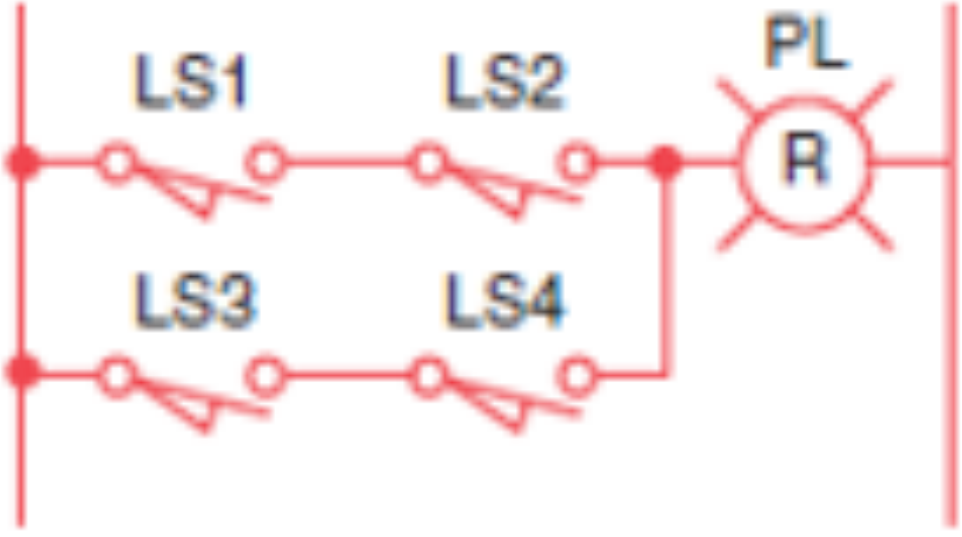

S1
LS1
L1
LS2

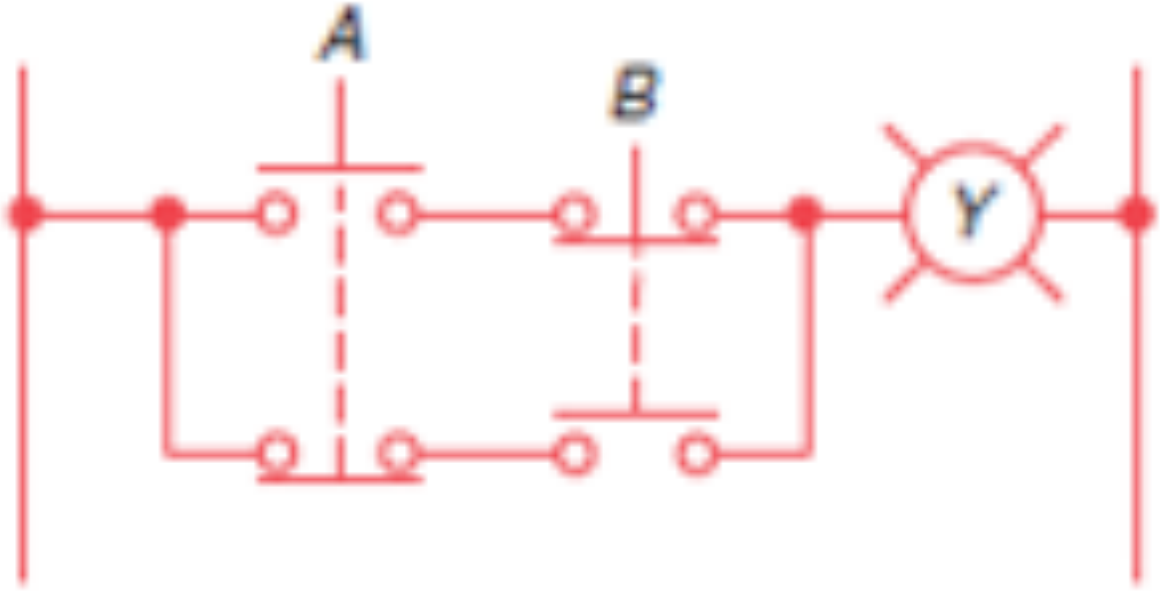

PB1
S1
PS1
TS1
L1
S2
S3

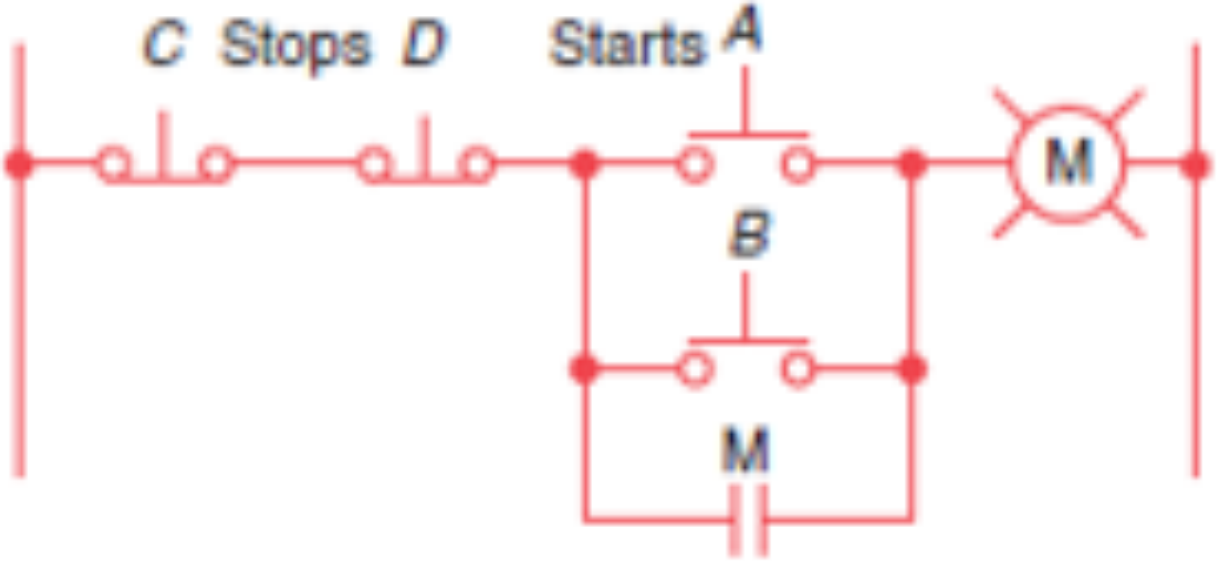

L1
STOP
START
M
OL
L2
M
G
M
R
M

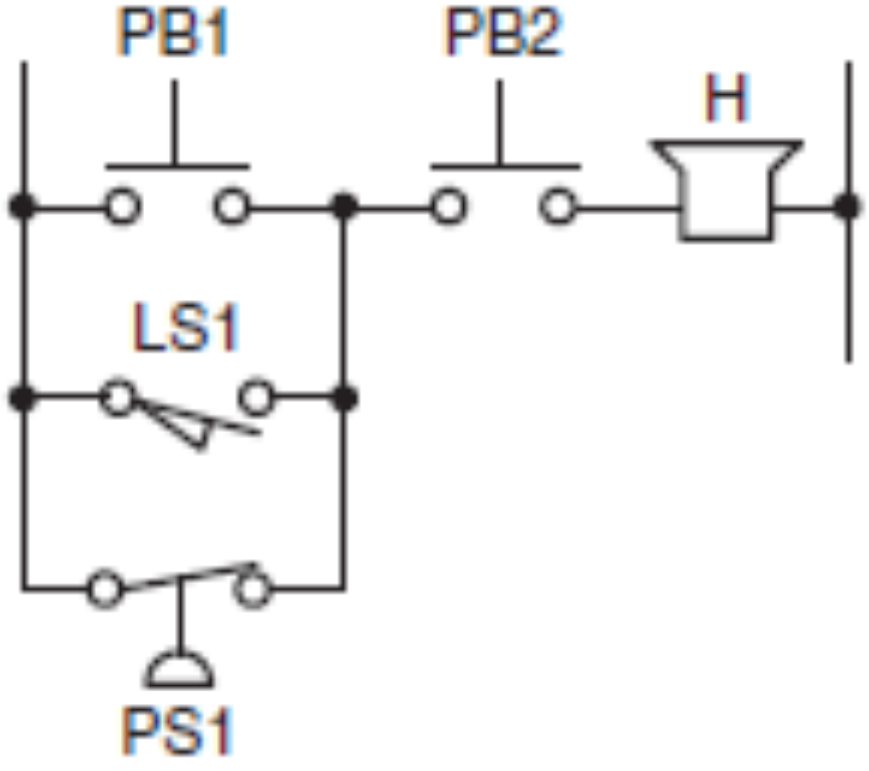

LS1
FS1
PL
R
LS2
FS2

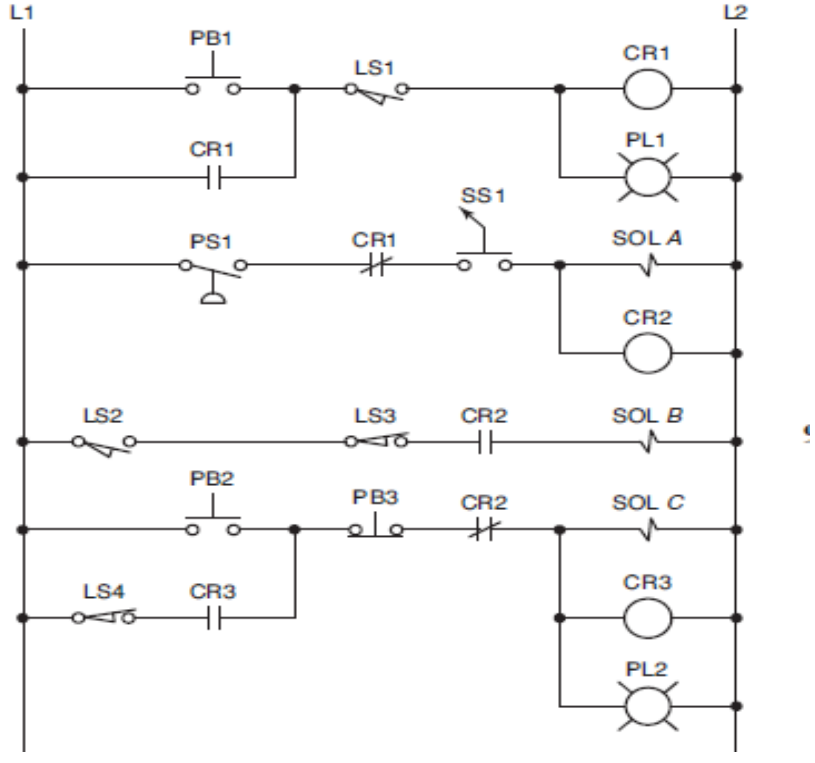

H
LS1
LS2
LS3

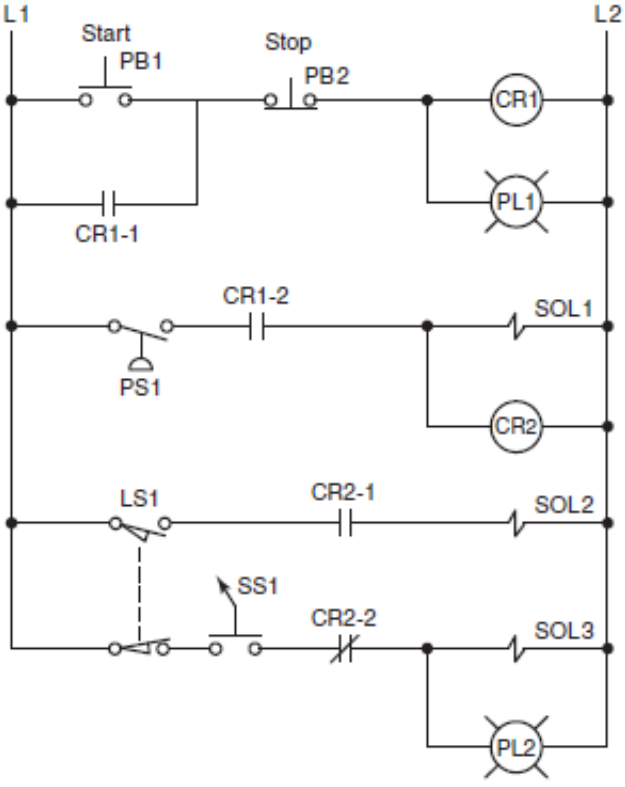

LS1
LS2
PL
R
LS3
LS4


A
B
Y

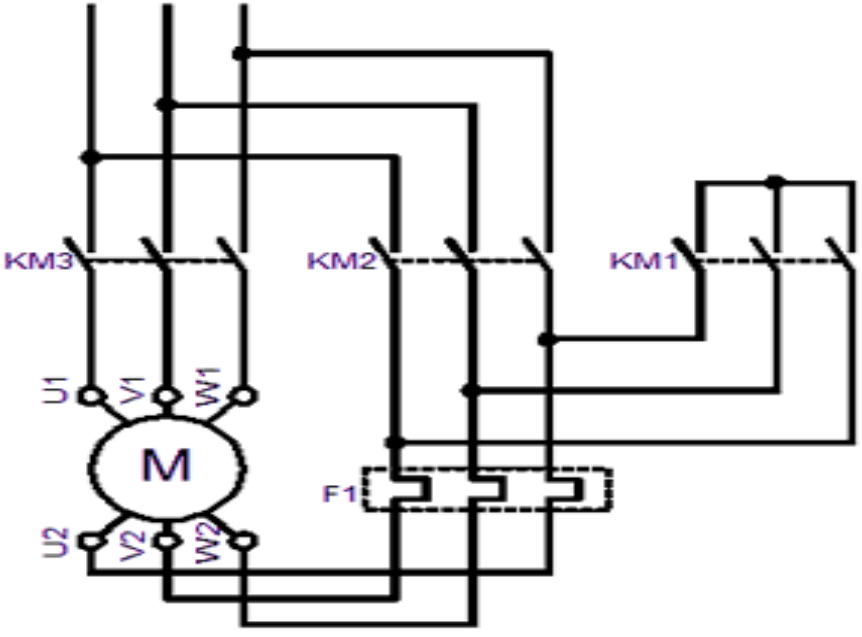

C Stops D
Starts A
M
B
M


PB1
PB2
H
LS1
PS1


L1
L2
PB1
LS1
CR1
CR1
PL1
SS1
PS1
CR1
SOL A
CR2
LS2
LS3
CR2
SOL B
PB2
PB3
CR2
SOL C
LS4
CR3
CR3
PL2


L1
L2
Start
PB1
Stop
PB2
CR1
PL1
CR1-1
CR1-2
SOL1
PS1
CR2
LS1
CR2-1
SOL2
SS1
CR2-2
SOL3
PL2

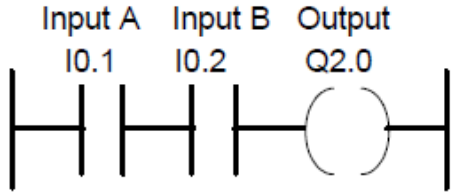
# Star-Delta For three phase motor



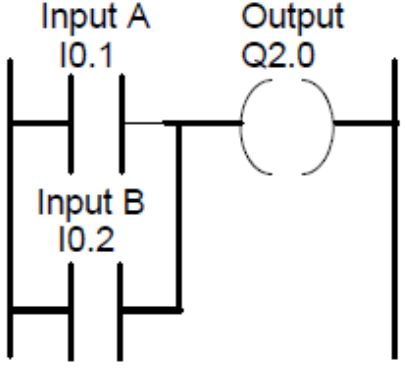
# PLC

INSTRUCTION

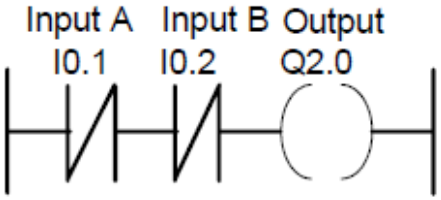
| IEC 1131-3 Operators | Mitsubishi | OMRON | Siemens/ Telemecanique | Operation | Ladder diagram |
|---|---|---|---|---|---|
| LD | LD | LD | LD | Load operand into result register | Start a rung with open contacts |
| LDN | LDI | LD NOT | LDN | Load negative operand into result register | Start a rung with closed contacts |
| AND | AND | AND | A | Boolean AND | A series element with open contacts |
| ANDN | ANI | AND NOT | AN | Boolean AND with negative operand | A series element with closed contacts |
| OR | OR | OR | O | Boolean OR | A parallel element with open contacts |
| ORN | ORI | OR NOT | ON | Boolean OR with negative operand | A parallel element with closed contacts |
| ST | OUT | OUT | = | Store result register into operand | An output |



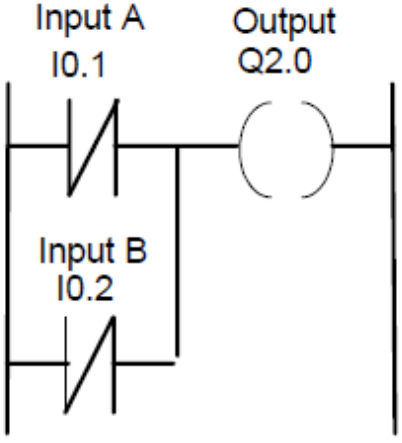

```
LD      I0.1
A       I0.2
=       Q2.0
```



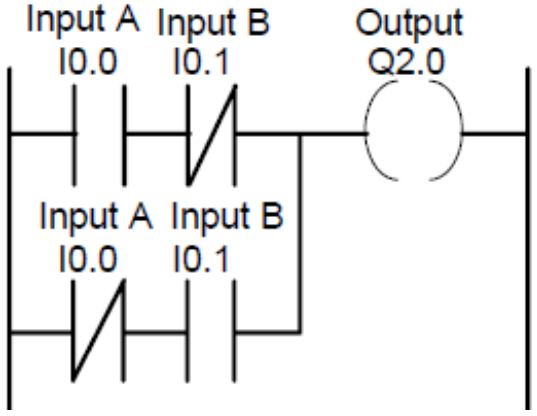

```
LD      I0.1
O       I0.2
=       Q2.0
```



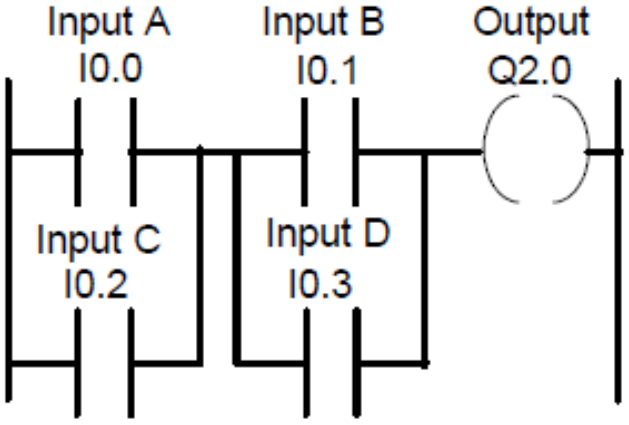

```
LDN     I0.1
AN      I0.2
=       Q2.0
```



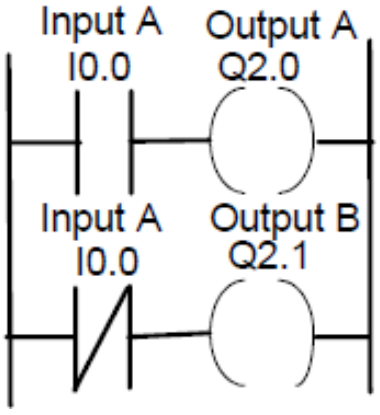

```
LDN     I0.1
ON      I0.2
=       Q2.0
```



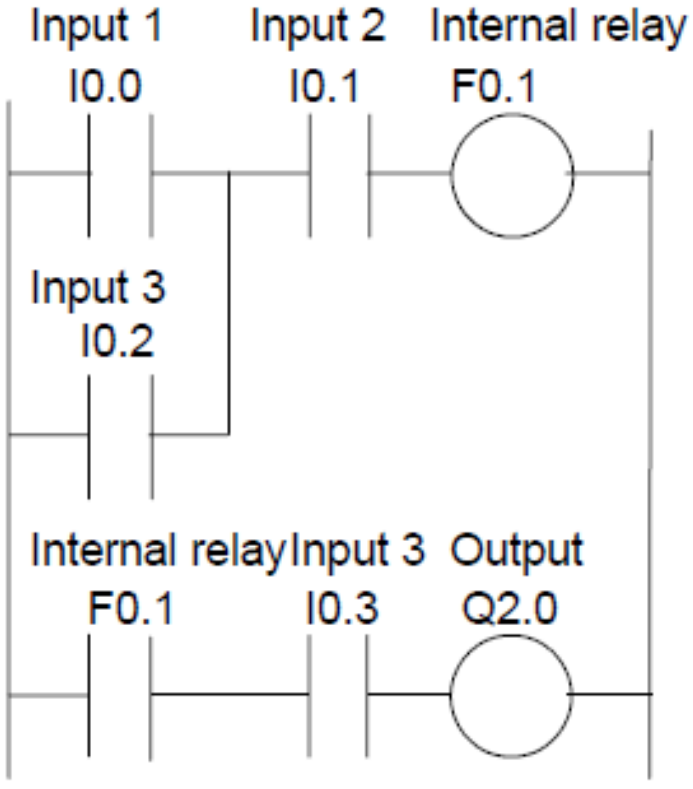

```
LD      I0.0
AN      I0.1
LDN     I0.0
A       I0.1
OLD
=       Q2.0
```




```
LD      I0.0
O       I0.2
LD      I0.1
O       I0.3
ALD
=       Q2.0
```

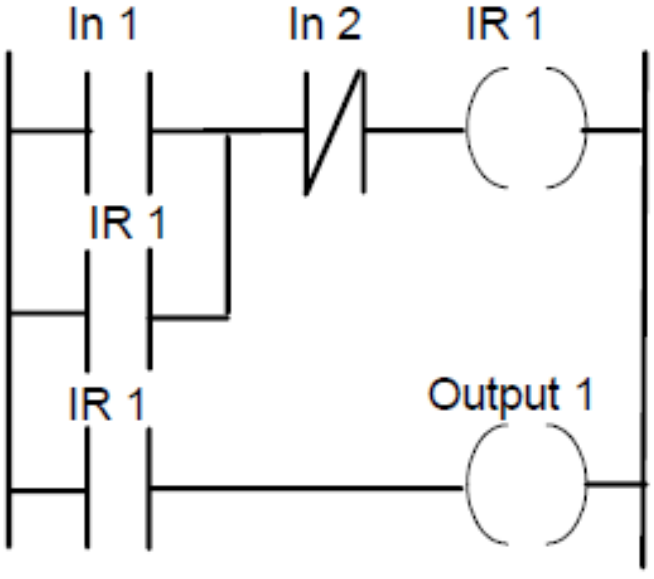


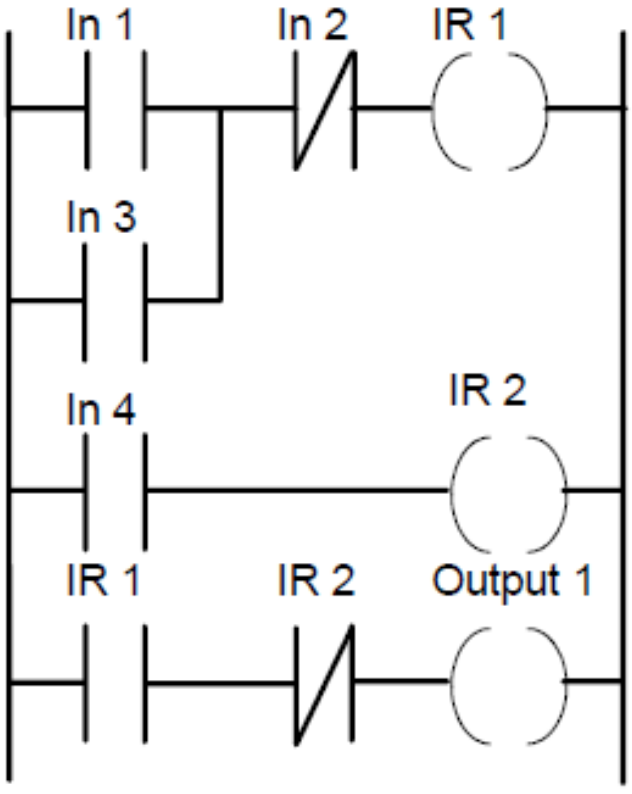

```
LD      I0.0
=       Q2.0
LDN     I0.0
=       Q2.1
```

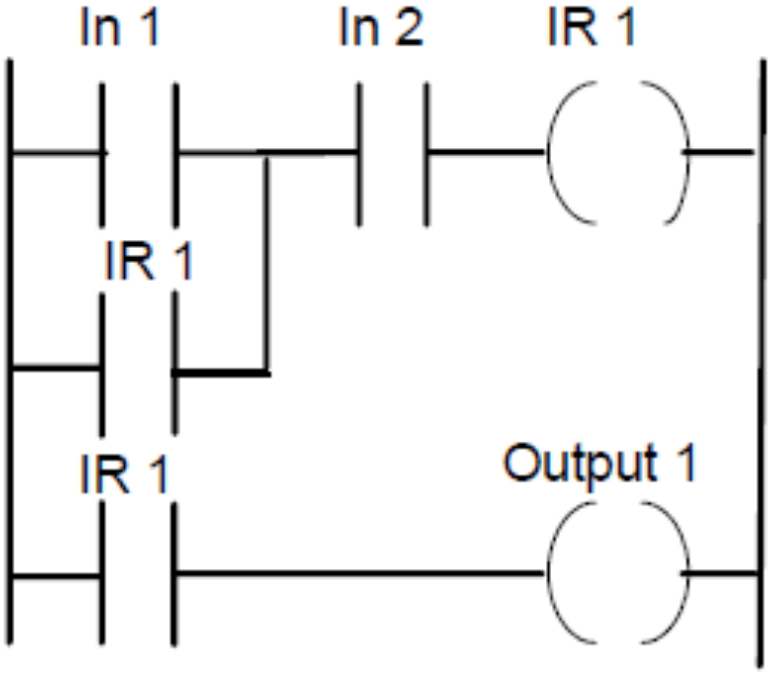


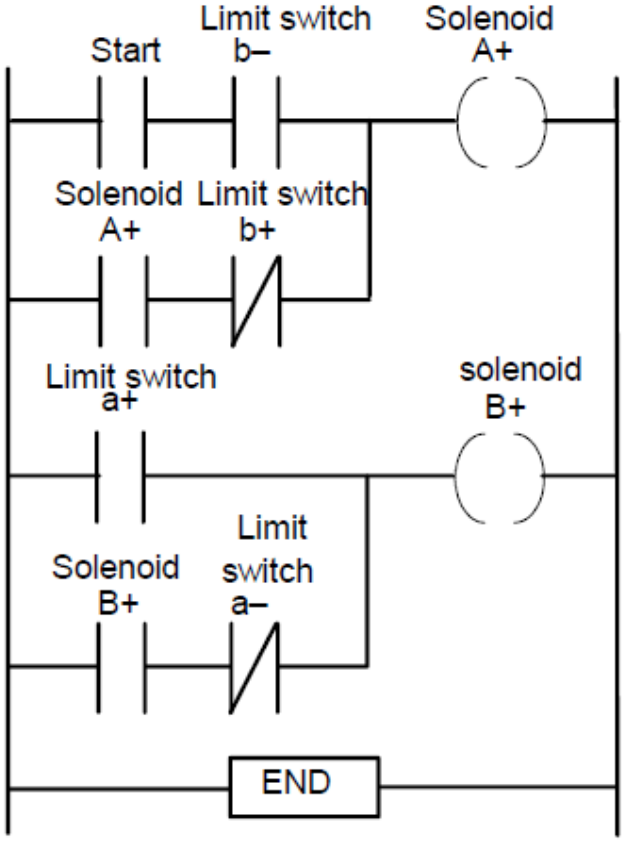

```
LD      I0.0
O       I0.2
A       I0.1
=       F0.1
LD      F0.1
A       I0.3
=       Q2.0
```

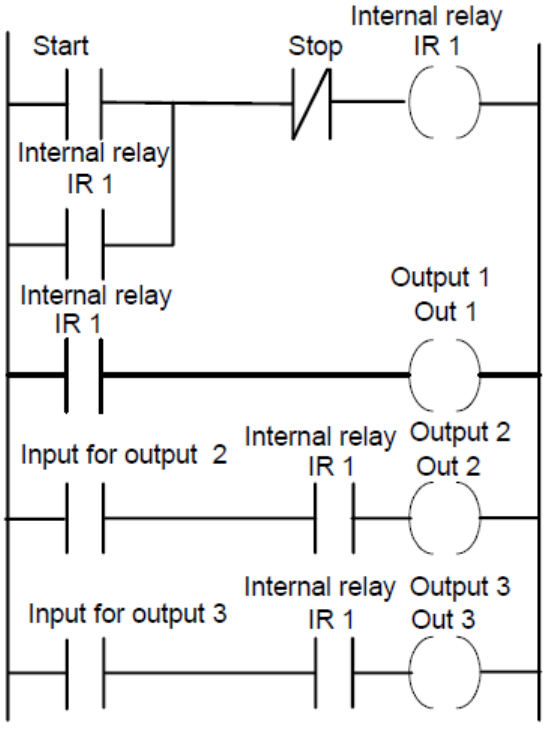
# Examples

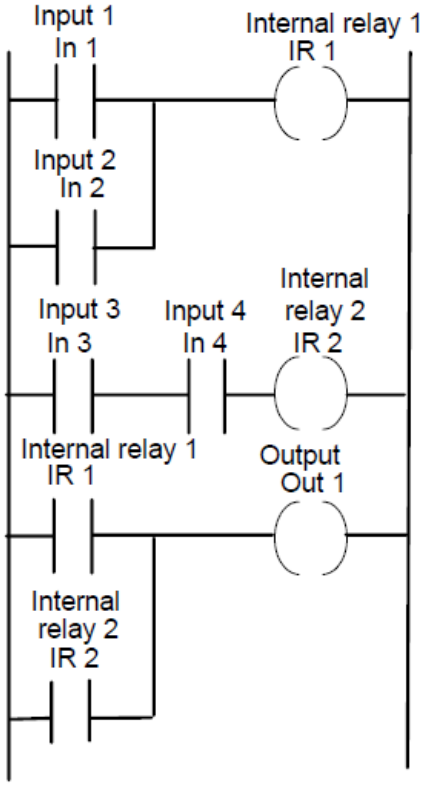



In 1
In 2
IR 1
In 3
In 4
IR 2
IR 1
IR 2
Output 1

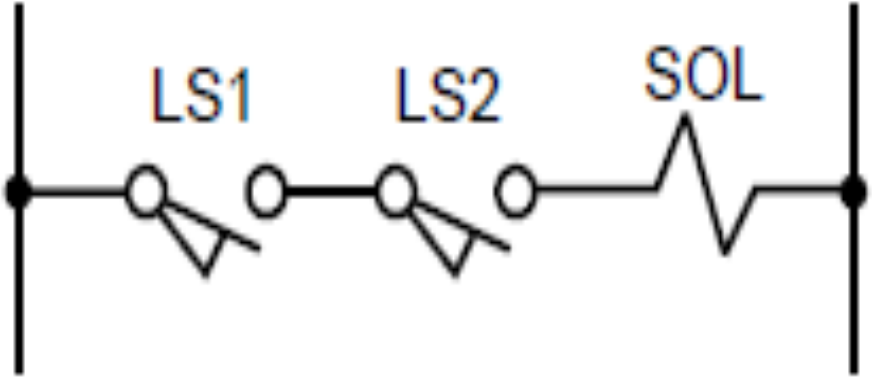

In 1
In 2
IR 1
IR 1
IR 1
Output 1

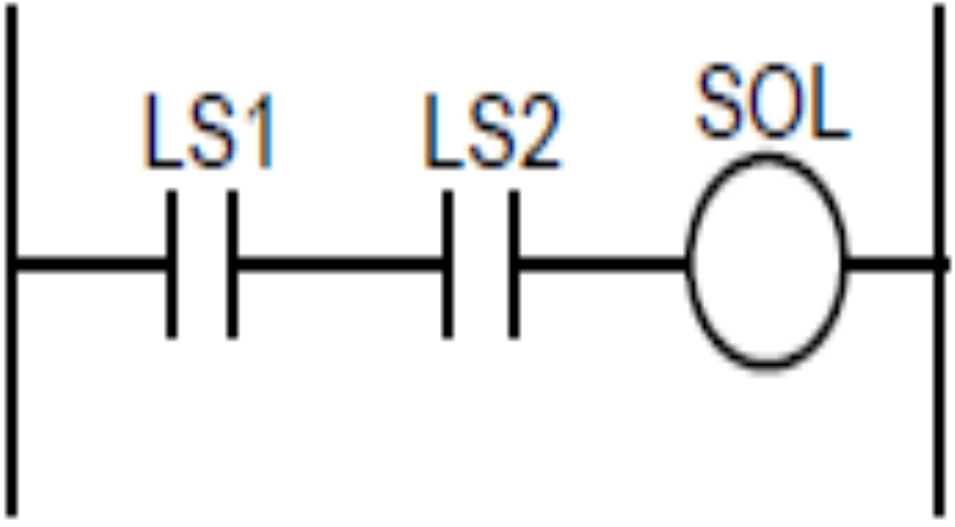

Start
Limit switch
b–
Solenoid
A+
Solenoid
A+
Limit switch
b+
Limit switch
a+
solenoid
B+
Solenoid
B+
Limit
switch
a–
END

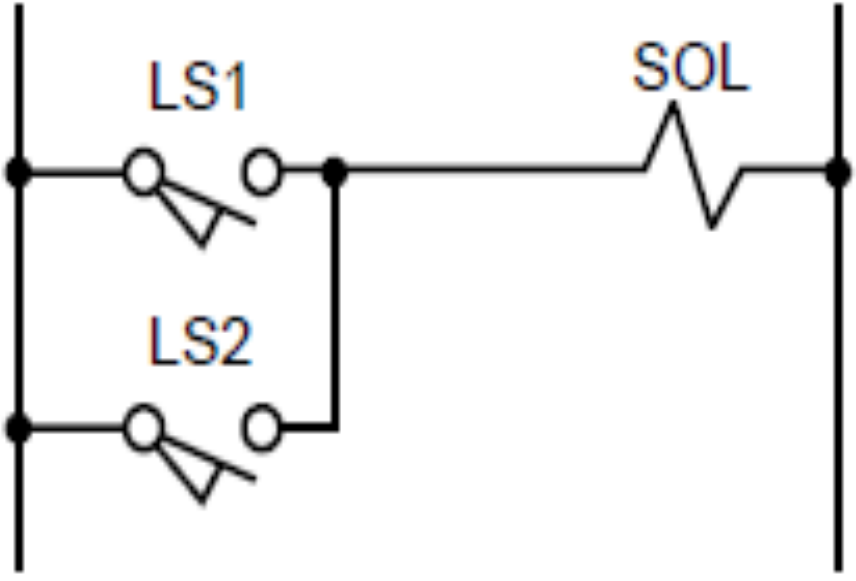

Start
Stop
Internal relay
IR 1
Internal relay
IR 1
Internal relay
IR 1
Output 1
Out 1
Input for output 2
Internal relay
IR 1
Output 2
Out 2
Input for output 3
Internal relay
IR 1
Output 3
Out 3

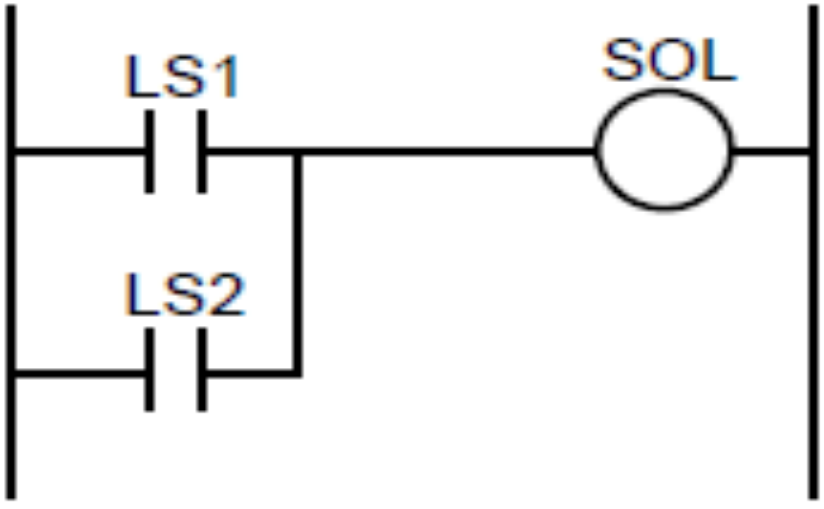

Input 1
In 1
Internal relay 1
IR 1
Input 2
In 2
Input 3
In 3
Input 4
In 4
Internal
relay 2
IR 2
Internal relay 1
IR 1
Output
Out 1
Internal
relay 2
IR 2

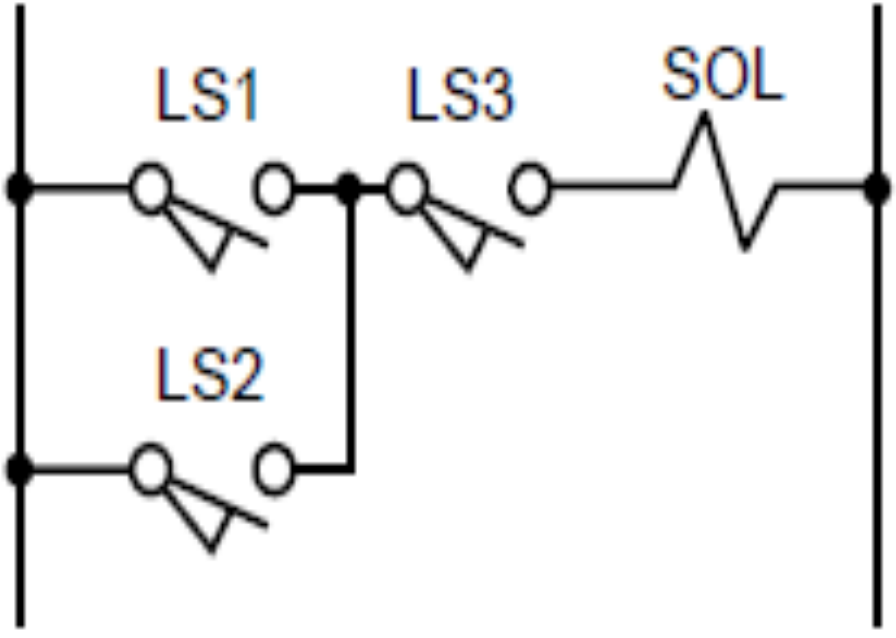
# Ladder diagram

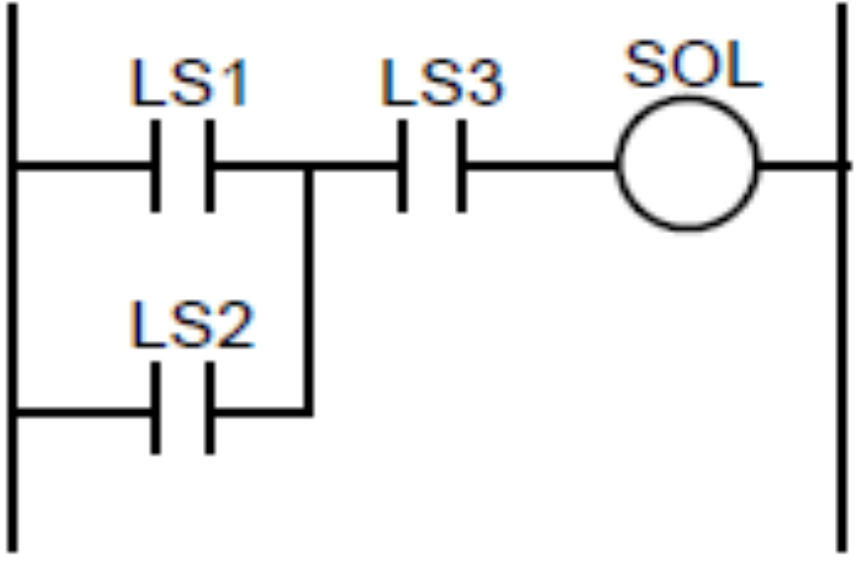


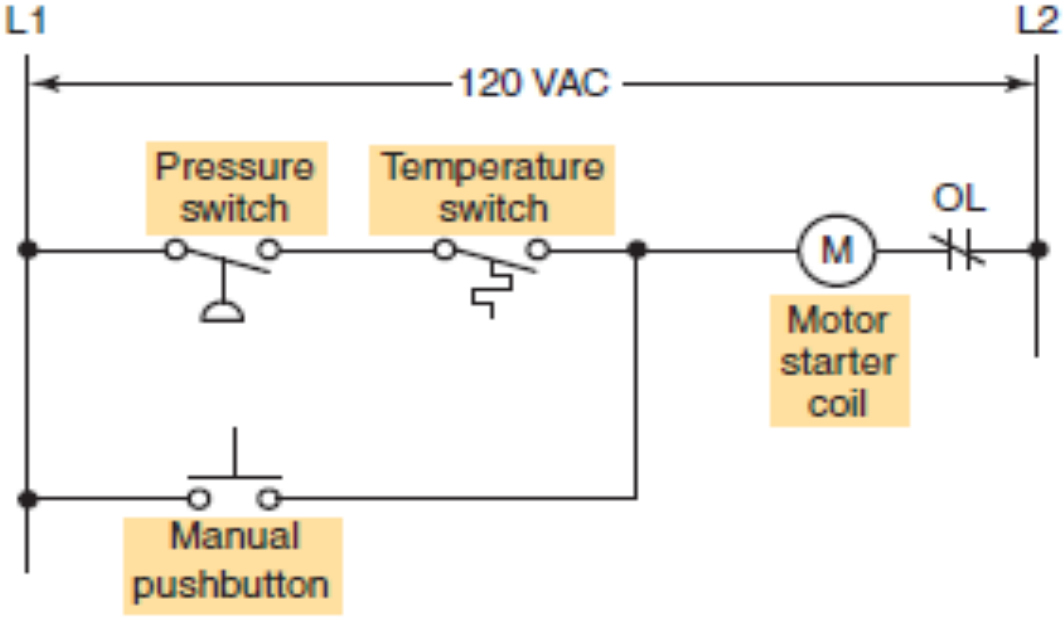

LS1
LS2
SOL

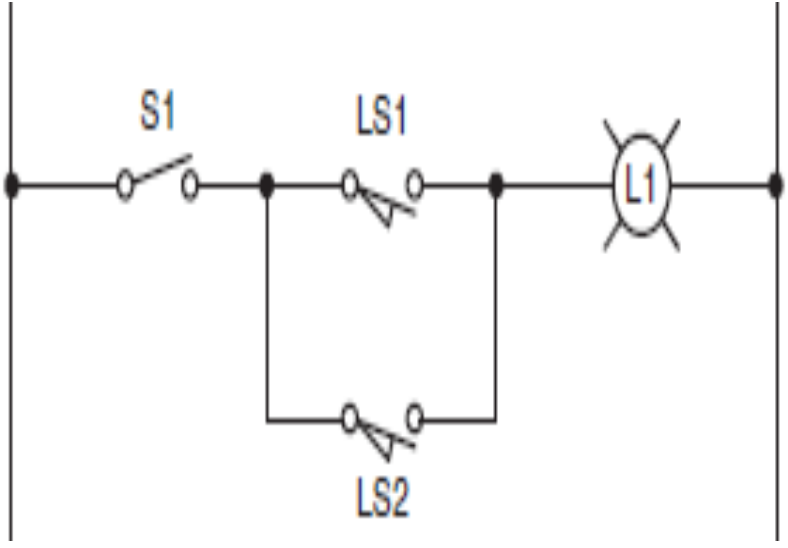

LS1
SOL
LS2

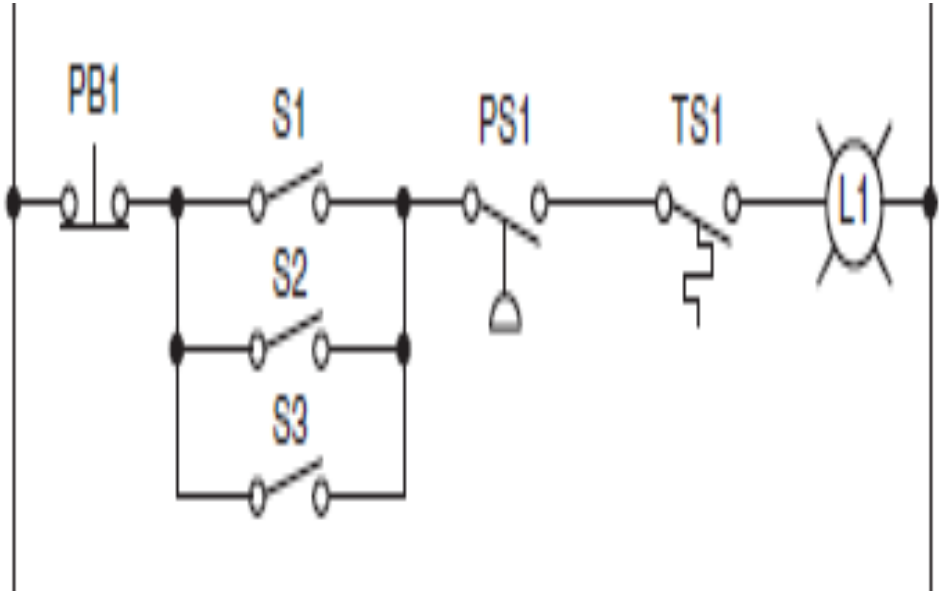

LS1
SOL
LS2

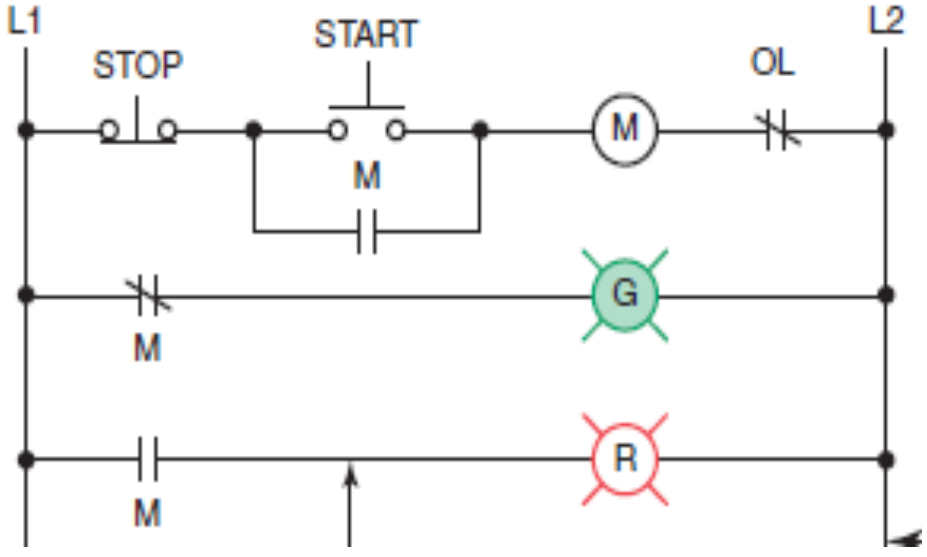

LS1
LS3
SOL
LS2

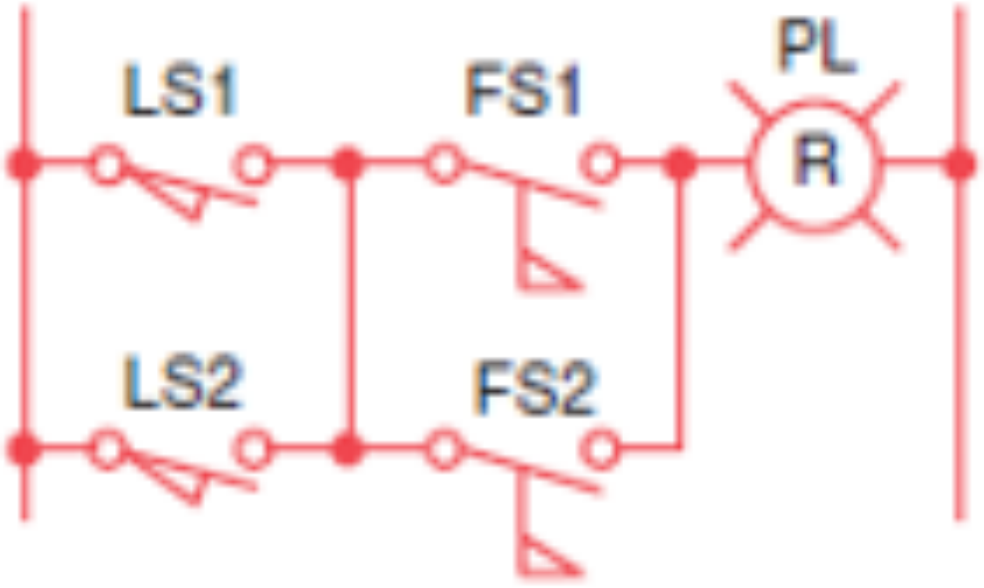

LS1
LS3
SOL
LS2

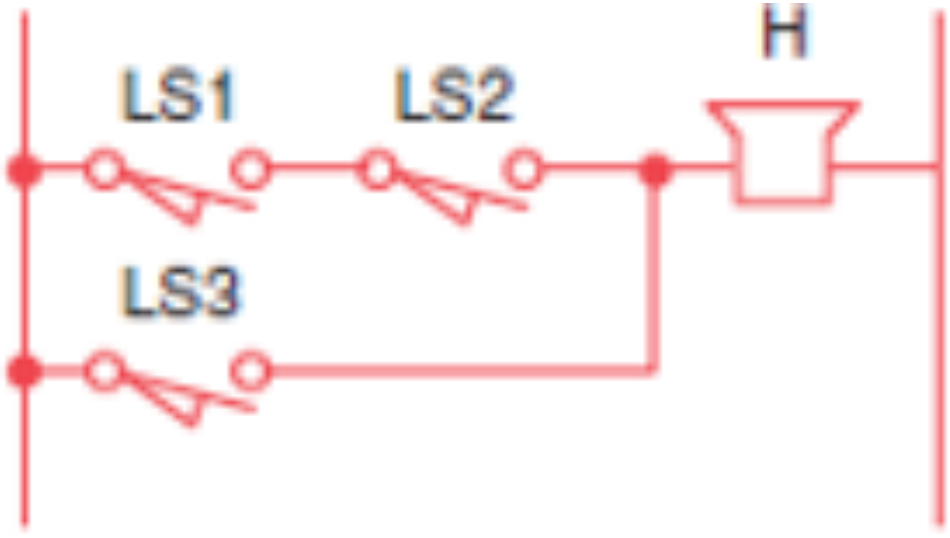

L1
L2
120 VAC
Pressure switch
Temperature switch
OL
M
Motor starter coil
Manual pushbutton

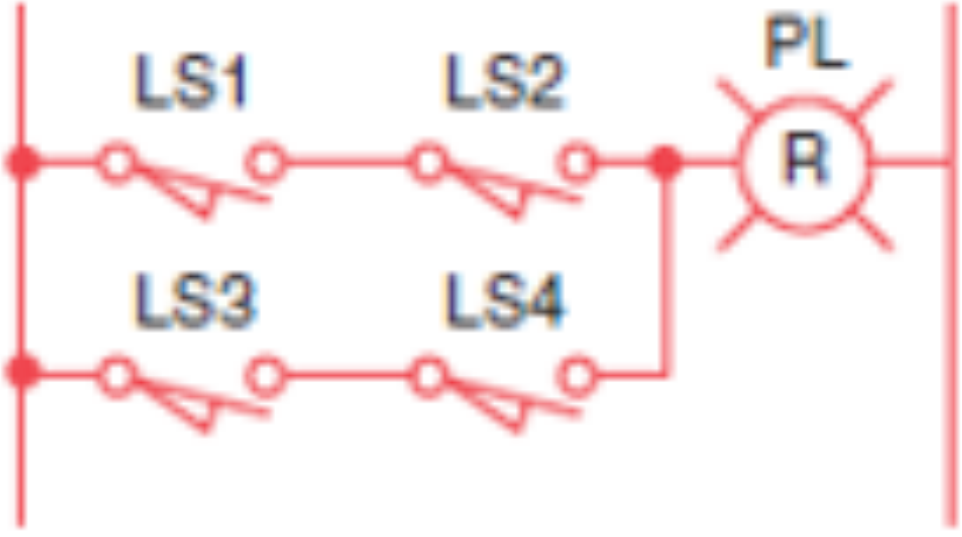

S1
LS1
L1
LS2

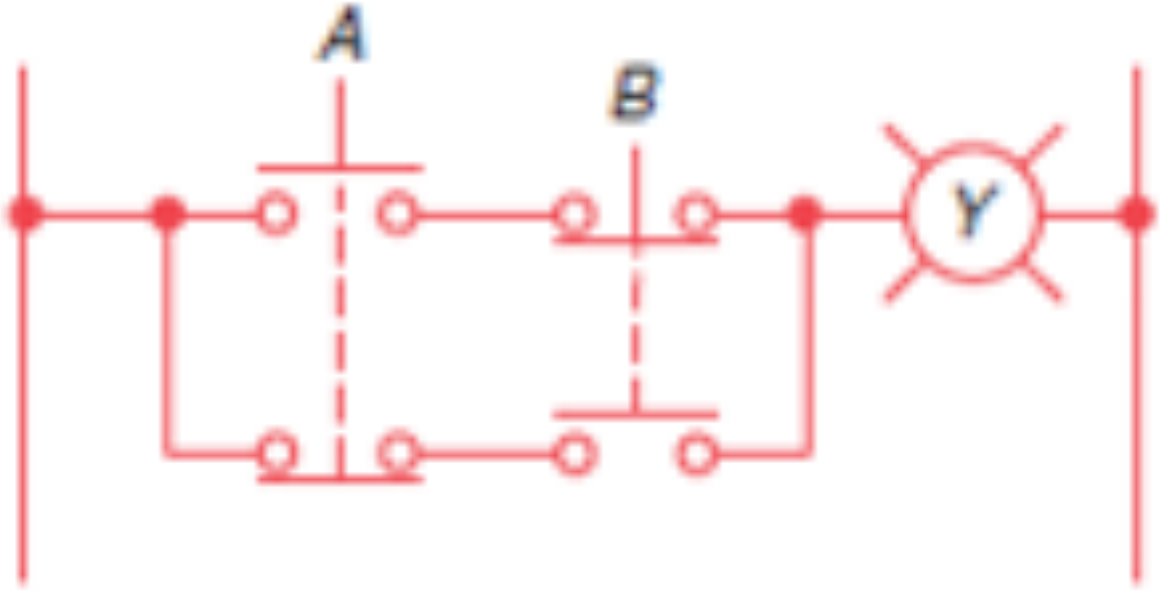

PB1
S1
PS1
TS1
L1
S2
S3

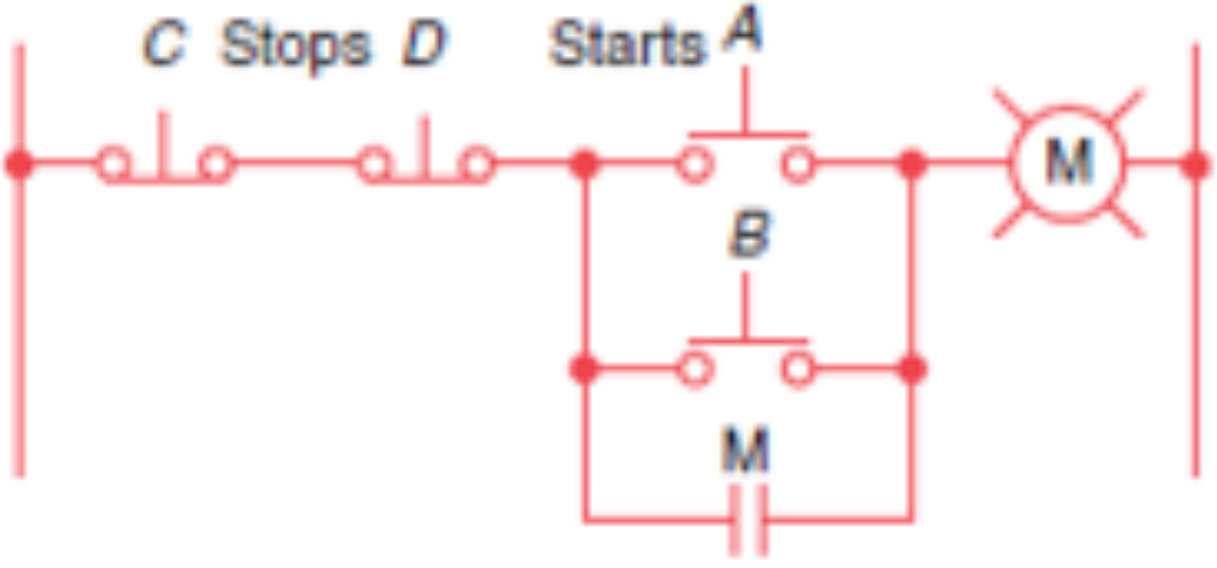

L1
STOP
START
M
OL
L2
M
M
G
M
R

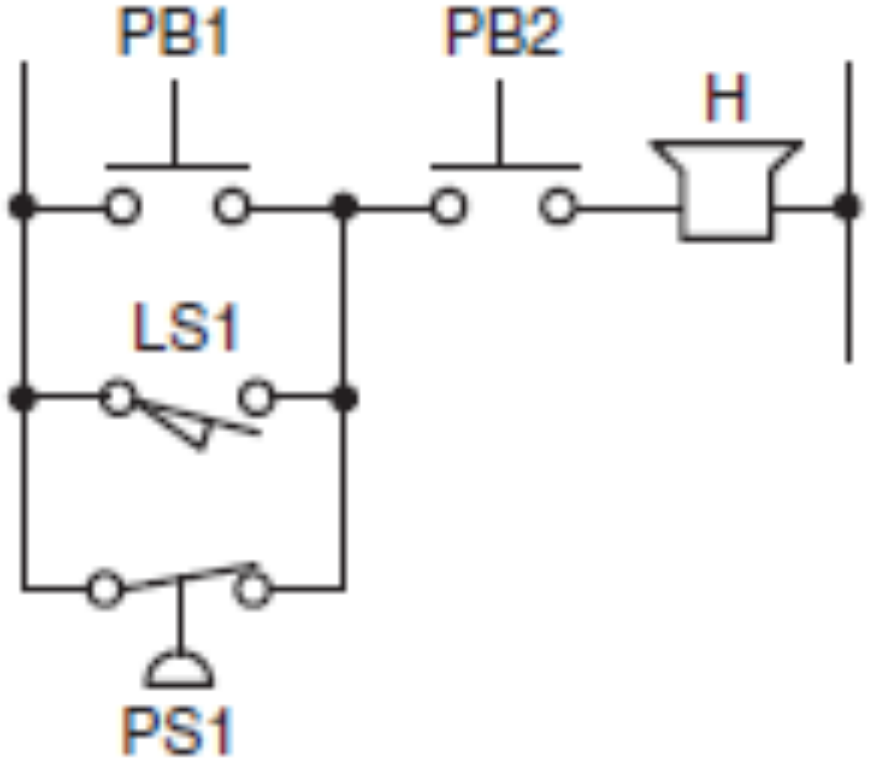

LS1
FS1
PL
R
LS2
FS2

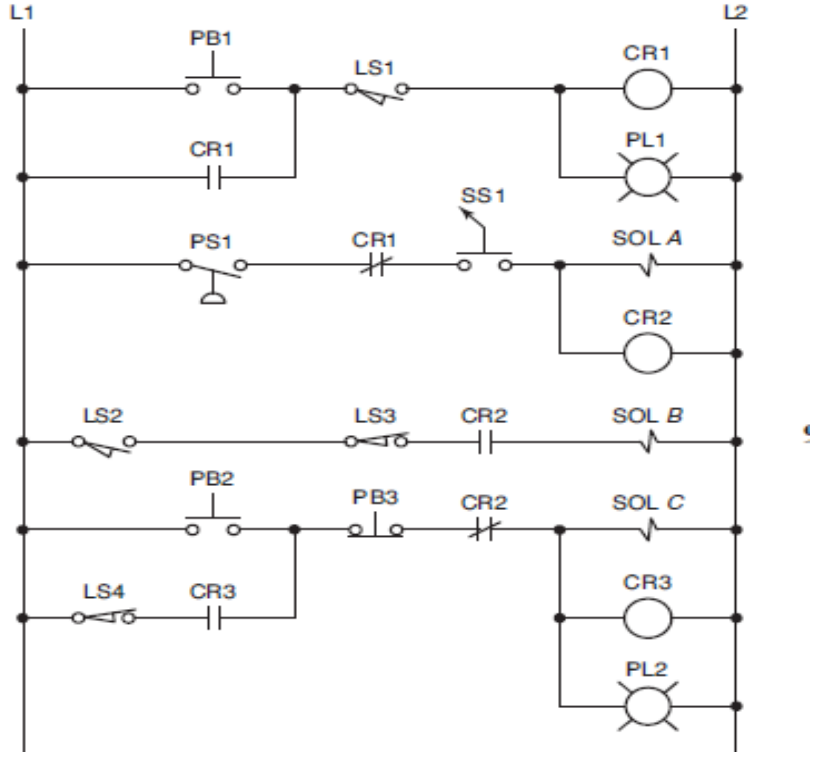

H
LS1
LS2
LS3

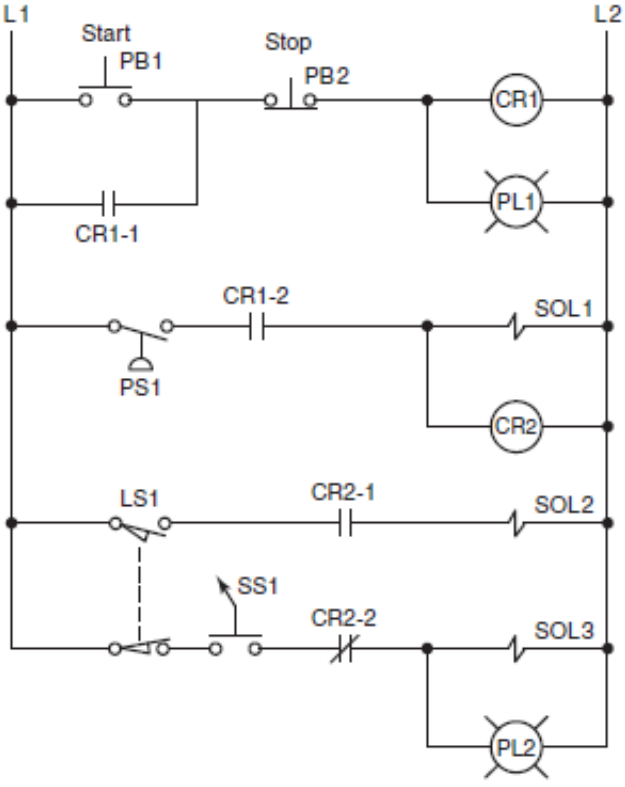

LS1
LS2
PL
R
LS3
LS4


A
B
Y

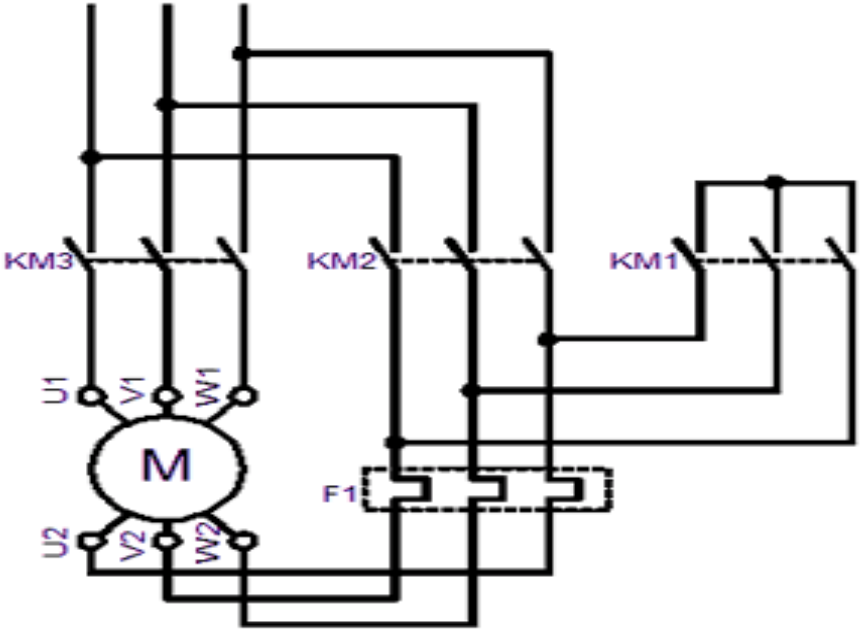

C Stops D
Starts A
M
B
M


PB1
PB2
H
LS1
PS1


L1
L2
PB1
LS1
CR1
CR1
PL1
SS1
PS1
CR1
SOL A
CR2
LS2
LS3
CR2
SOL B
PB2
PB3
CR2
SOL C
LS4
CR3
CR3
PL2


L1
L2
Start
PB1
Stop
PB2
CR1
PL1
CR1-1
CR1-2
SOL1
PS1
CR2
LS1
CR2-1
SOL2
SS1
CR2-2
SOL3
PL2

# Star-Delta For three phase motor

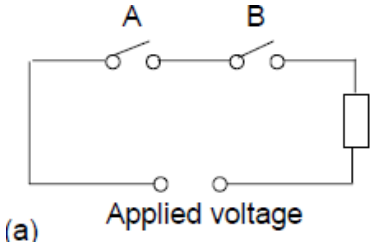


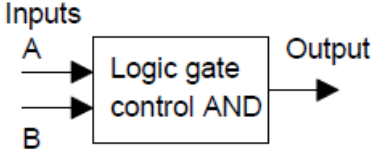
# PLC

Logic Function

# Logic functions

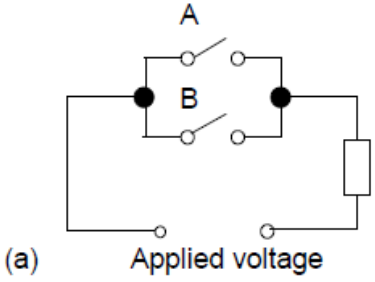
AND **Logic functions:**

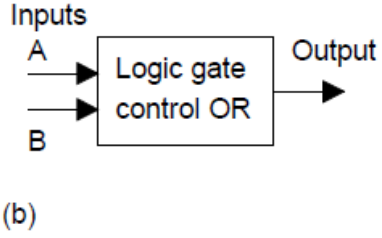



| Inputs | | Output |
|---|---|---|
| A | B | |
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

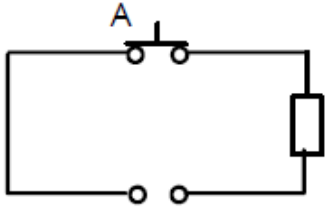
# OR




| Inputs | | Output |
|---|---|---|
| A | B | |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

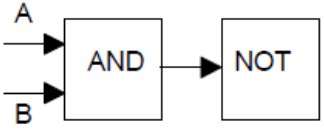
# NOT



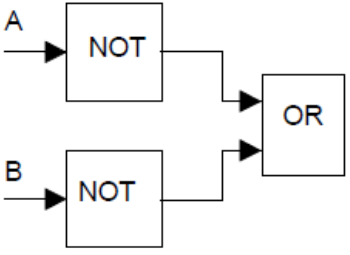

(a) Applied voltage

| Input A | Output |
|---|---|
| 0 | 1 |
| 1 | 0 |

# NAND



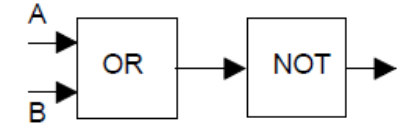

(a)



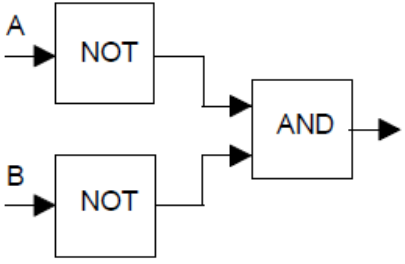

(b)

| Inputs | | Output |
|---|---|---|
| A | B | |
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

# NOR



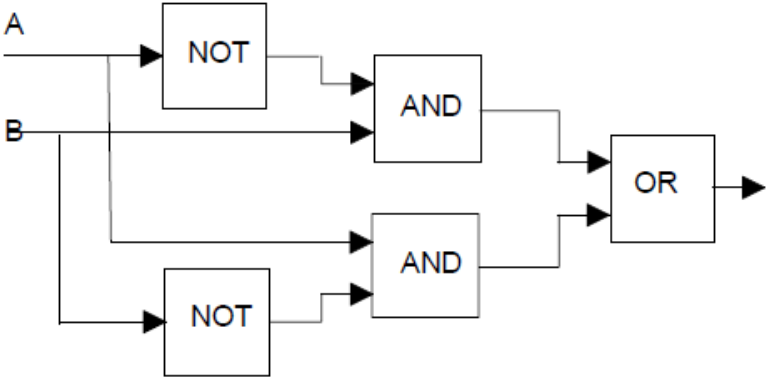

(a)




(b)

| Inputs | | Output |
|---|---|---|
| A | B | |
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

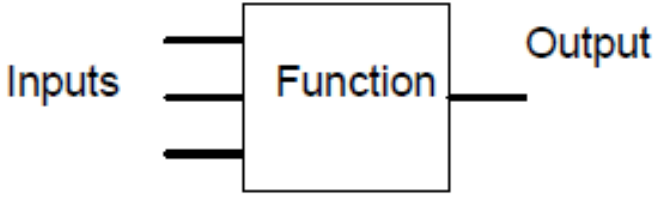
# Exclusive OR (XOR)



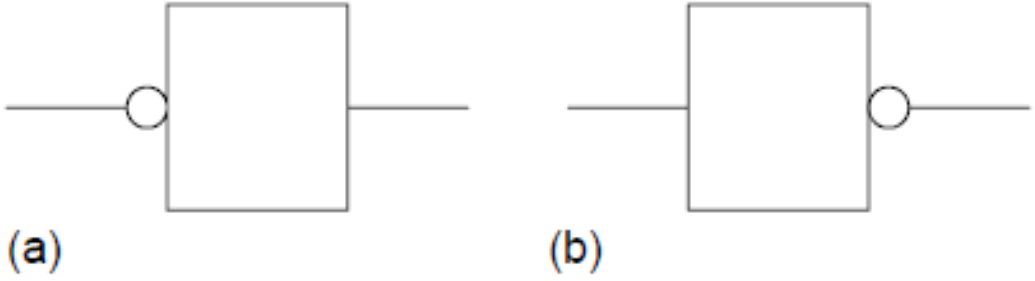

| Inputs | | Output |
|---|---|---|
| A | B | |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

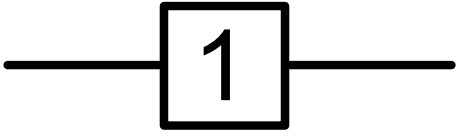
# Logic gates Function blocks

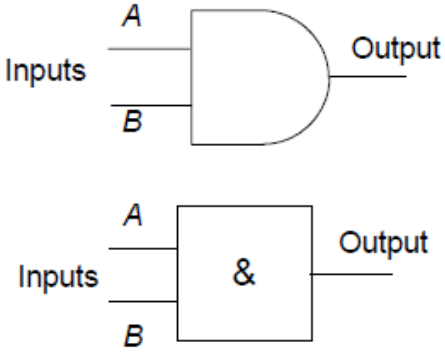
*Function block*



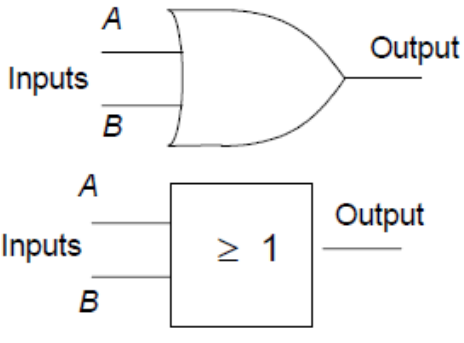
*(a) Negated input, (b) negated output*



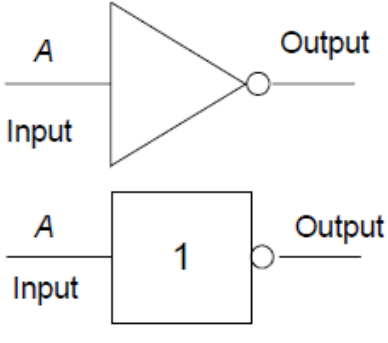






*AND gate*

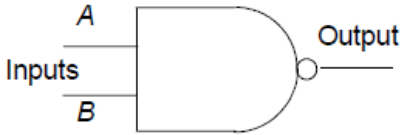


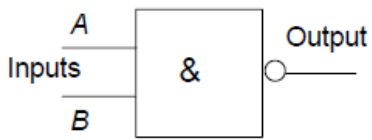

*OR gate*



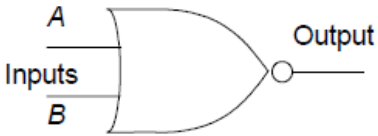

*Not gate*

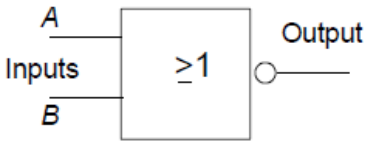
> = 1



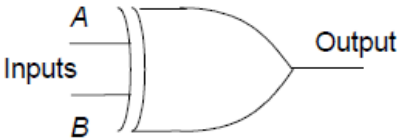



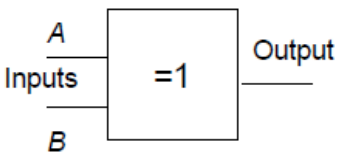

*NAND gate*



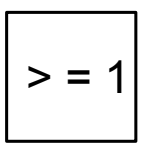




*NOR Gate*



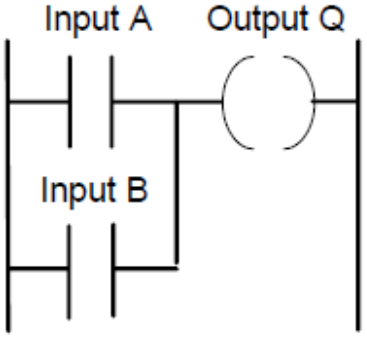



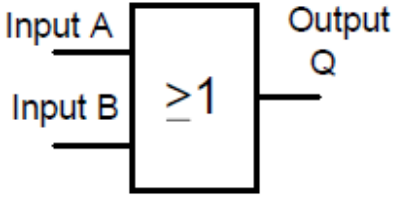

*XOR Gate*

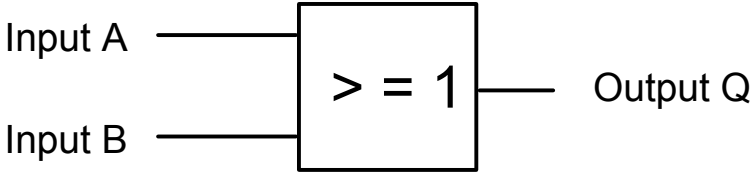


# *Ladder diagram and equivalent functional block diagram*

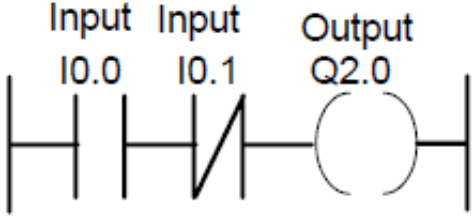
OR gate

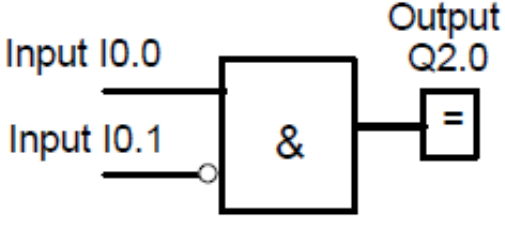


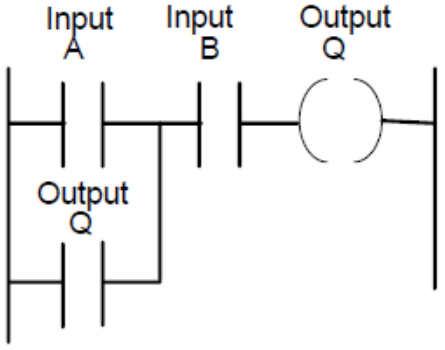



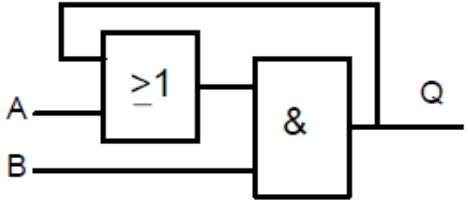



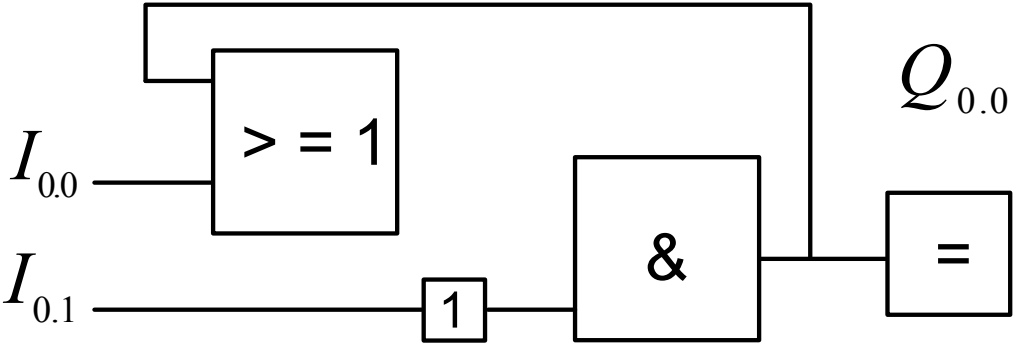
# AND gate (Siemens notation)

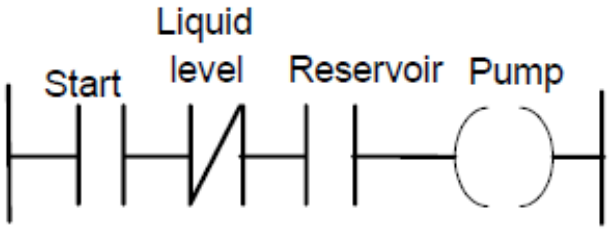


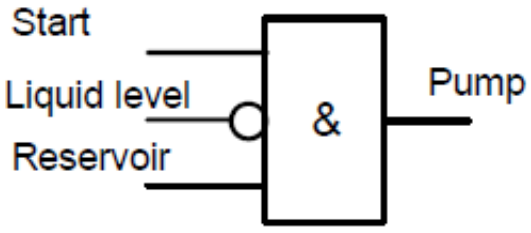



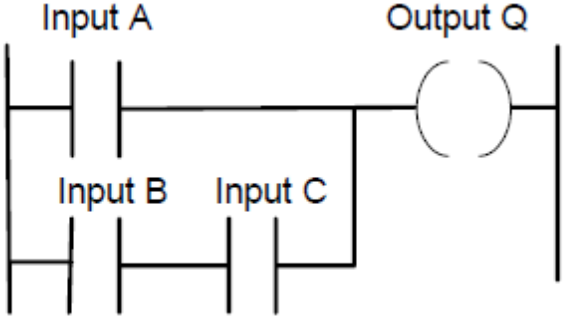

Input
A
Input
B
Output
Q
Output
Q

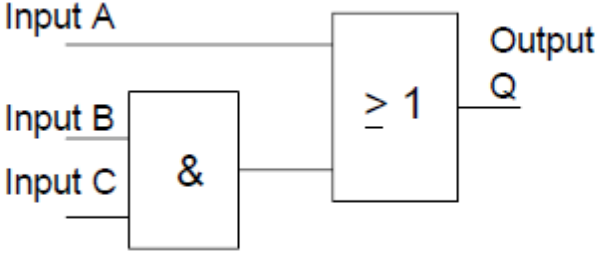

≥1
A
B
&
Q

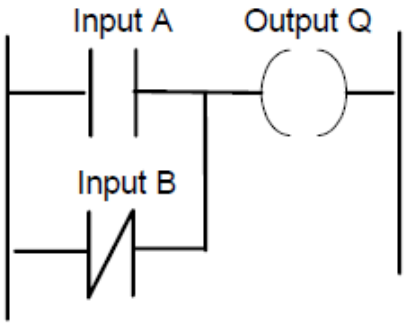

$I_{0.0}$
$I_{0.1}$
> = 1
1
&
=
$Q_{0.0}$

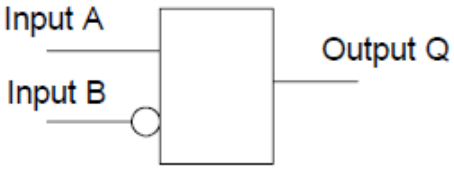

Start
Liquid level
Reservoir
Pump

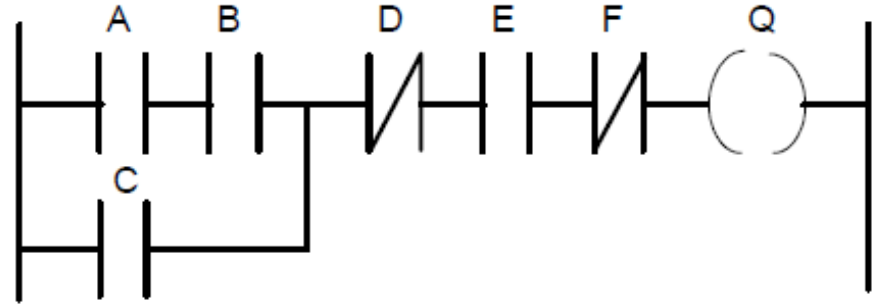

Start
Liquid level
Reservoir
&
Pump

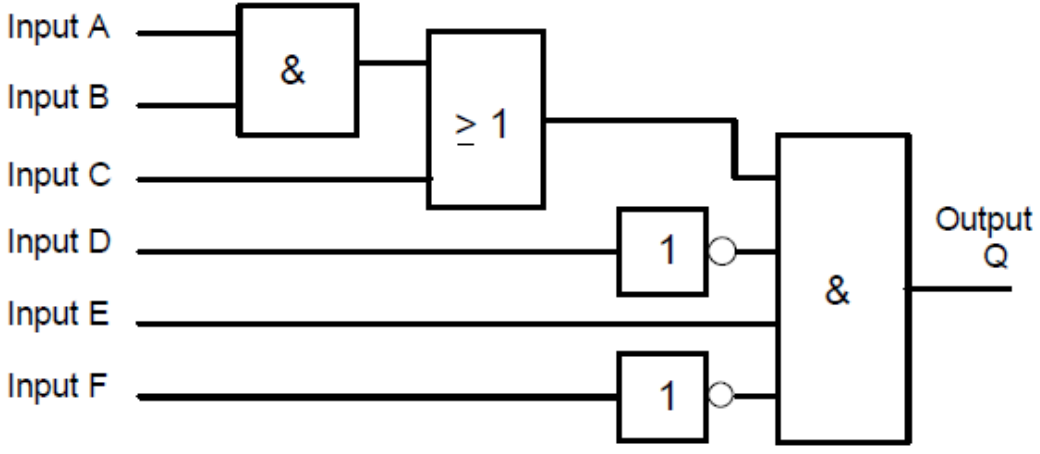

Input A
Output Q
Input B
Input C

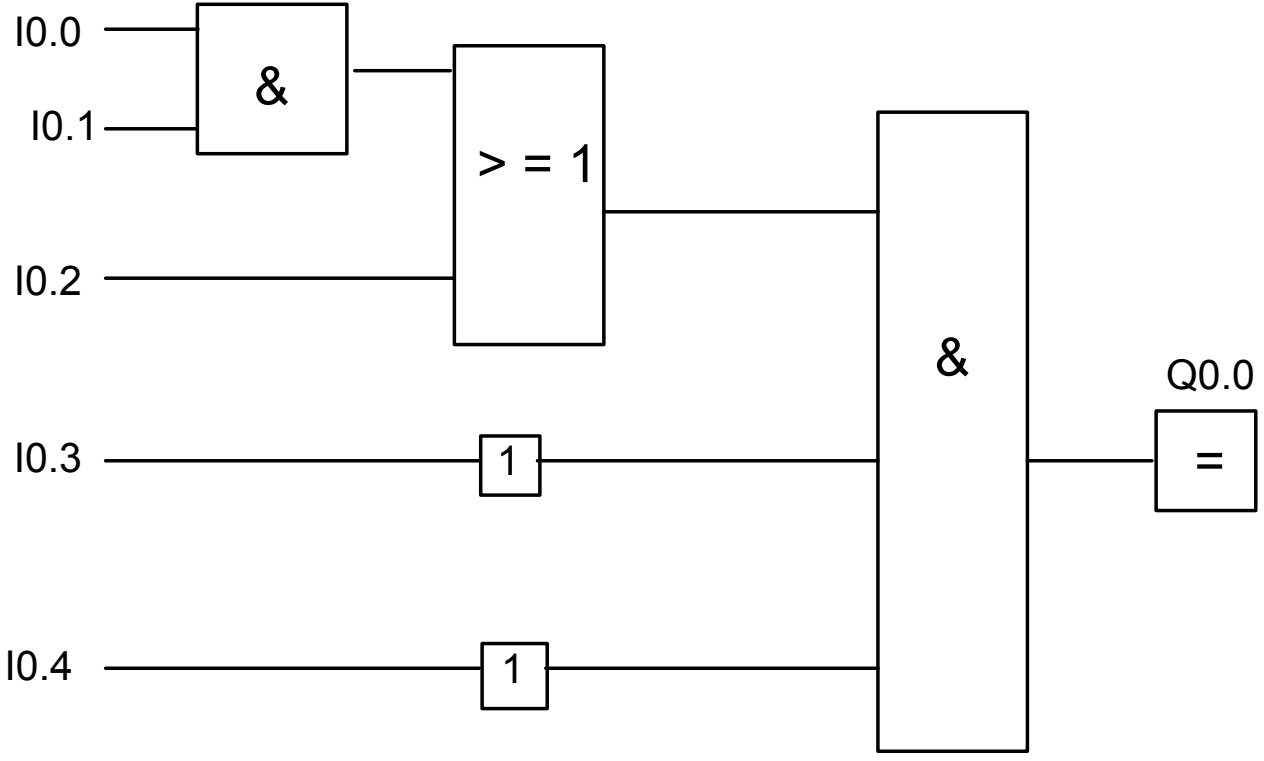

Input A
Input B
Input C
&
≥ 1
Output Q


Input A
Output Q
Input B


Input A
Input B
Output Q


A
B
D
E
F
Q
C


Input A
Input B
Input C
Input D
Input E
Input F
&
≥ 1
1
1
&
Output
Q


I0.0
I0.1
&
> = 1
I0.2
I0.3
1
&
Q0.0
=
I0.4
1

# PLC

Mat-Comp-Logic Instructions

# Compare Instructions

| | | | |
|---|---|---|---|
| Comparison | = | equal to | if A = B, then ... |
| | <> | not equal to | if A <> B, then ... |
| | == | exactly equal | if A == B, then ... |
| | < | less than | if A < B, then ... |
| | <= | less than or equal to | if A <= B, then ... |
| | > | greater than | if A > B, then ... |
| | >= | greater than or equal to | if A >= B, then ... |

# Arithmetic

| | | |
|---|---|---|
| + | add | A+B |
| – | subtract | A-B |
| * | multiply | A*B |
| / | divide | A/B |
| SQR | square root | SQR I0:0 |

# Example

Compare

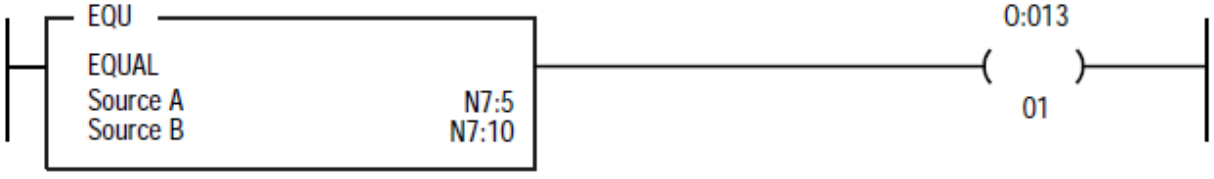



- The CMP instruction tells the processor: if the sum of the values in N7:0 and N7:1 is greater than the sum of the values in N7:2 and N7:3, set output bit O:013/01.

**Equal to (EQU)**



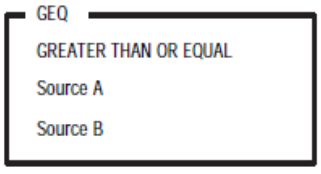

**Example:**



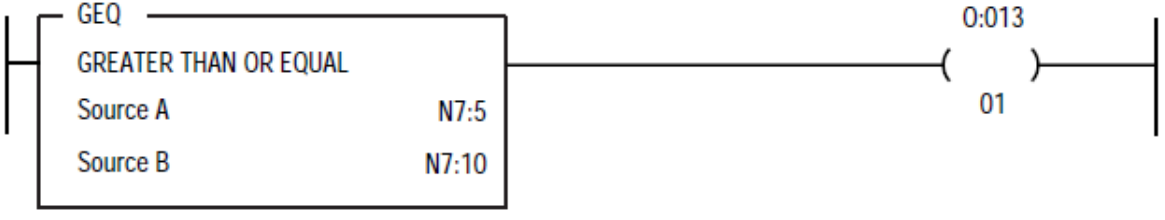

If the value in N7:5 is equal to the value in N7:10, set output bit O:013/01.

# Greater than or Equal to (GEQ)

**Description:** Use the GEQ instruction to test whether one value (Source A) is greater than or equal to another value (Source B). Source A and Source B can be values or addresses that contain values.



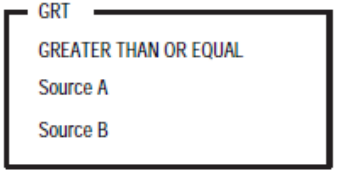

## Example:




If the value in N7:5 is greater than or equal to the value in N7:10, set output bit O:013/01.

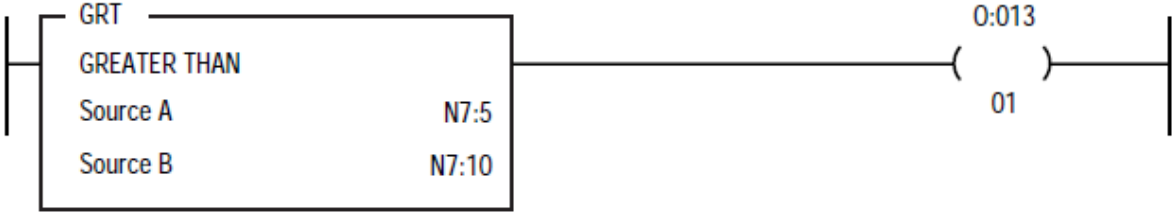
# Greater than (GRT)

**Description:**




Use the GRT instruction to test whether one value (Source A) is greater than another value (Source B). Source A and Source B can either be values or addresses that contain values.

## Example:

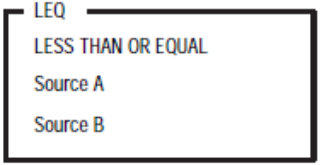



If the value in N7:5 is greater than the value in N7:10, set output bit O:013/01.

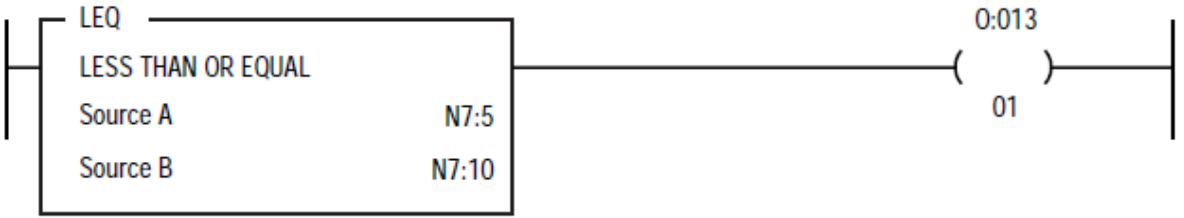
# Less than or Equal to (LEQ)

**Description:**

Use the LEQ instruction to test whether one value (Source A) is less than or equal to another value (Source B). Source A and Source B can either be values or addresses that contain values.



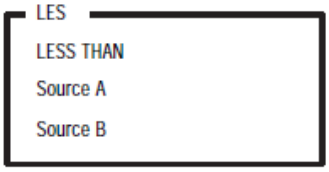

## Example:




If the value in N7:5 is less than or equal to the value in N7:10, set output bit O:013/01.

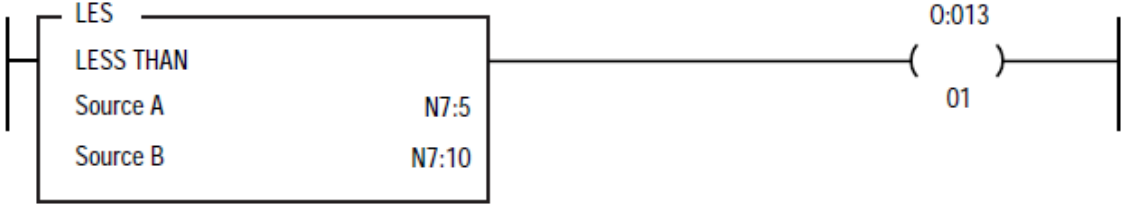
# Less than (LES)

**Description:**




Use the LES instruction to test whether one value (Source A) is less than another value (Source B). Source A and Source B can be values or addresses that contain values.

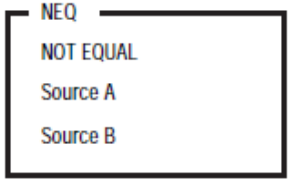
## Example:



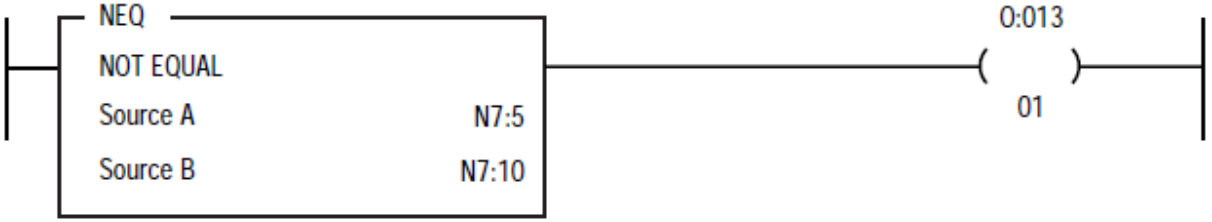

If the value in N7:5 is less than the value in N7:10, set output bit O:013/01.

# Not Equal to (NEQ)

**Description:** Use the NEQ instruction to test whether two values are not equal. Source A and Source B can be values or addresses.



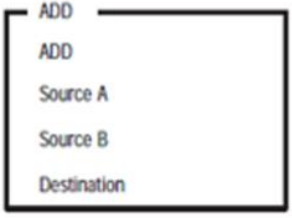

## Example:



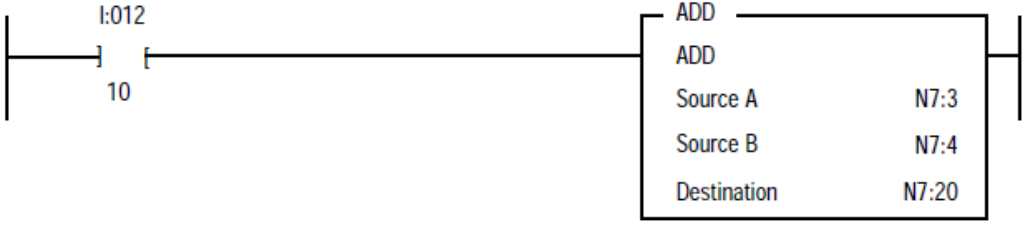

If the value in N7:5 is not equal to the value in N7:10, set output bit O:013/01.

# Addition (ADD)

**Description:** Use the ADD instruction to add one value (Source A) to another value (Source B) and place the result in the destination. Source A and Source B can either be values or addresses that contain values.



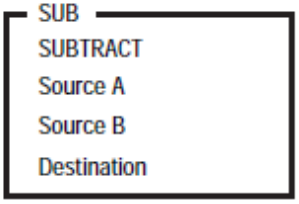

## Example:




If input word 12, bit 10 is set, add the value in N7:3 to the value in N7:4 and store the result in N7:20.

# Subtract (SUB)

**Description:**




Use the SUB instruction to subtract one value (Source B) from another value (Source A) and place the result in the destination. Source A and Source B can be values or addresses that contain values.

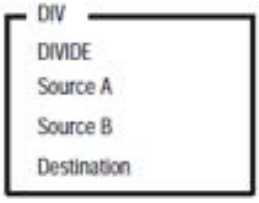
**Example:**




If input word 12, bit 10 is set, subtract the value in N7:4 from the value in N7:3 and store the result in N7:20.

# Divide (DIV)

**Description:** Use the DIV instruction to divide one value (Source A) by another value (Source B) and place the result in the Destination. Source A and Source B can either be values or addresses that contain values.



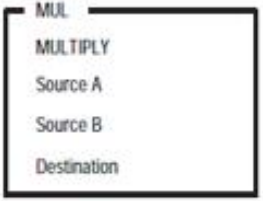

## Example:



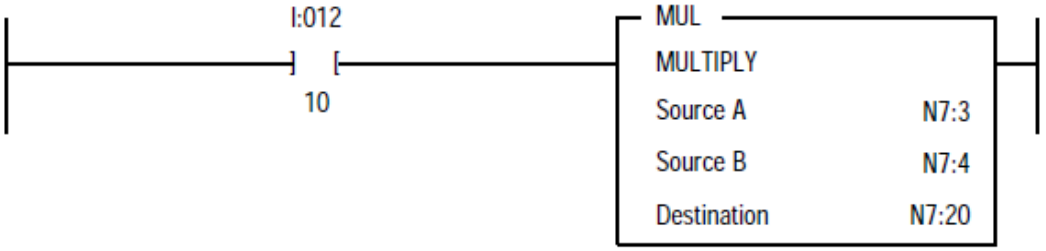

If input word 12, bit 10 is set, divide the value in N7:3 by the value in N7:4 and store the result in N7:20.

# Multiply (MUL)

**Description:** Use the MUL instruction to multiply one value (Source A) by another value (Source B) and place the result in the destination. Source A and Source B can be values or addresses.



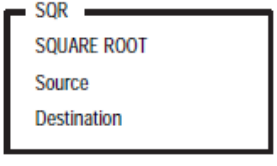

## Example:




If input word 12, bit 10 is set, multiply the value in N7:3 by the value in N7:4 and store the result in N7:20.

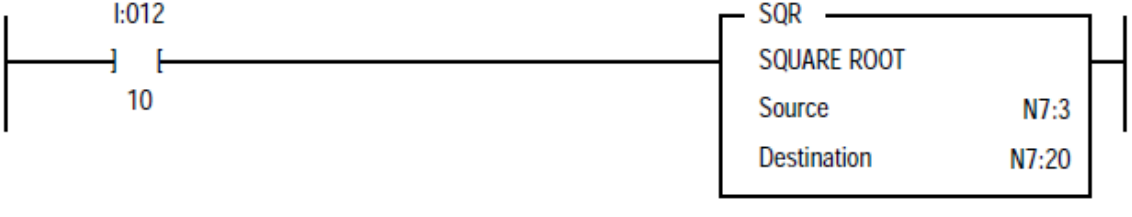
# Square Root (SQR)

**Description:**




Use the SQR instruction to take the square root of a value and store the result in the destination. The source can be a value or an address. If the source value is negative, the processor takes its absolute value and performs the square root function.

**Example:**

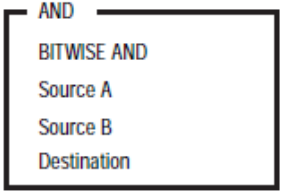



If input word 12, bit 10 is set, take the square root of the value in N7:3 and store the result in N7:20.

# Logical Instructions AND, NOT, OR, XOR

## AND Operation (AND)

**Description:** Use the AND instruction to perform an AND operation using the bits in the two source addresses.

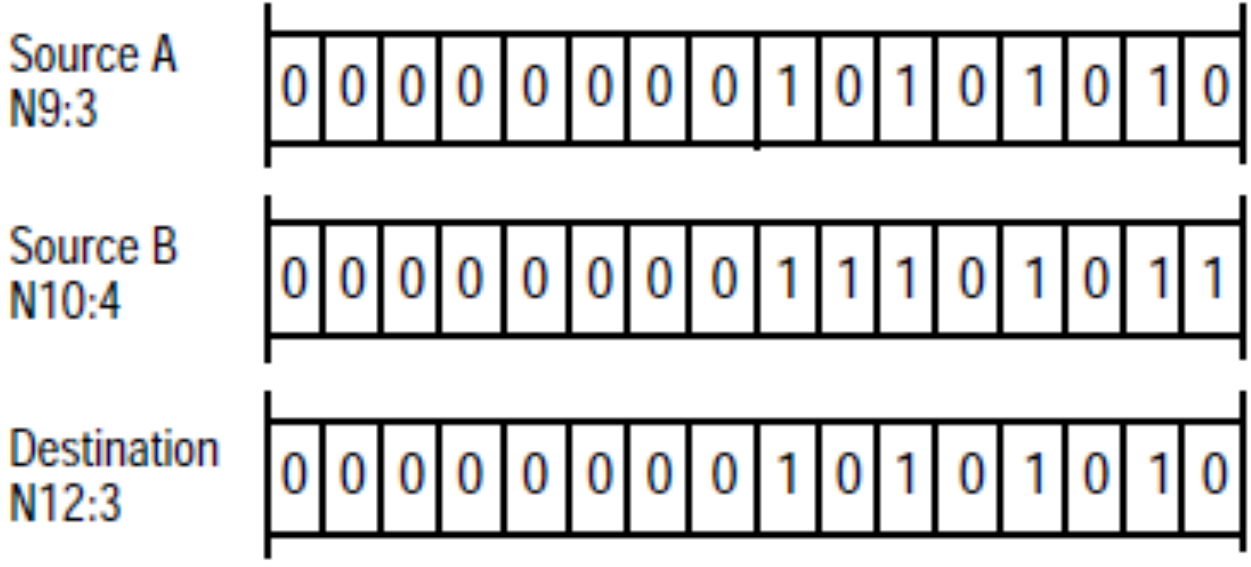



| Source A | Source B | Result |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 1 |

**Example:**

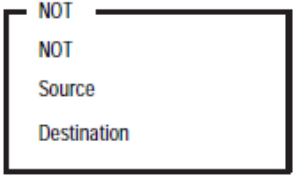



If input word 12, bit 10 is set, the processor performs an AND operation on N9:3 and N10:4 and stores the result in N12:3.


Source A
N9:3
0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0
Source B
N10:4
0 0 0 0 0 0 0 0 1 1 1 0 1 0 1 1
Destination
N12:3
0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0

# NOT Operation (NOT)

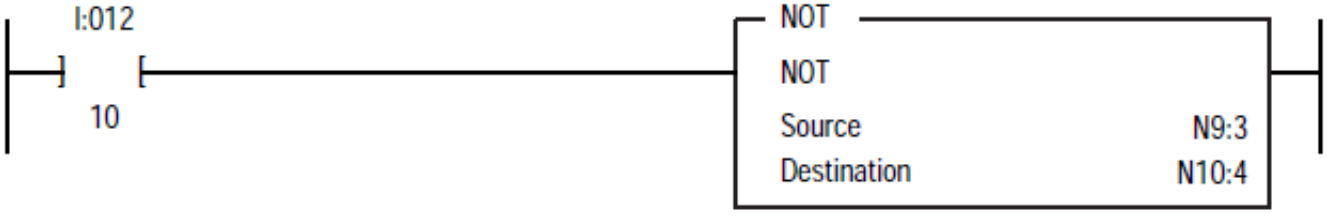
**Description:** Use the NOT instruction to perform a NOT operation using the bits in the source address. This operation is also know as a bit inversion.



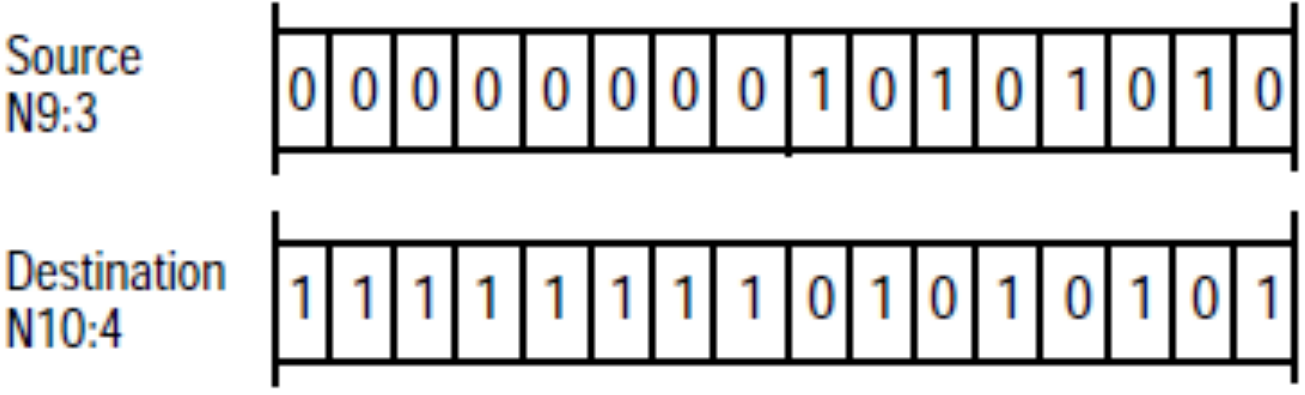

**Truth Table for a NOT Operation**

| Source | Result |
|---|---|
| 0 | 1 |
| 1 | 0 |

**Example:**

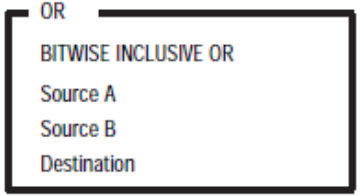



If input word 12, bit 10 is set, the processor performs a NOT operation on N9:3 and stores the result in N10:4


Source
N9:3
0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0
Destination
N10:4
1 1 1 1 1 1 1 1 0 1 0 1 0 1 0 1

# OR Operation (OR)

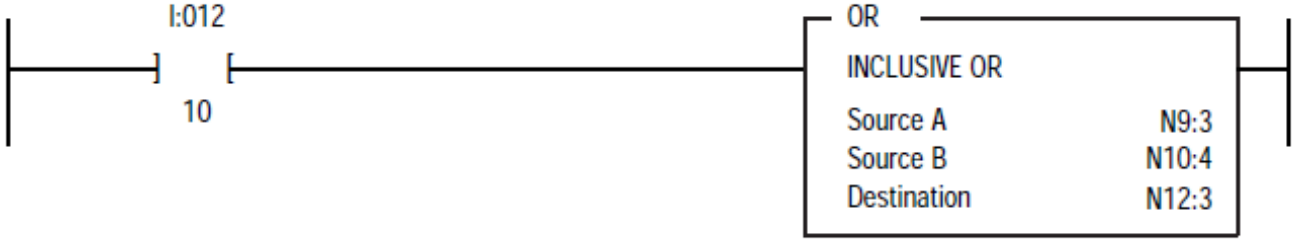
**Description:** Use the OR instruction to perform an OR operation using the bits in the two sources (constants or addresses).



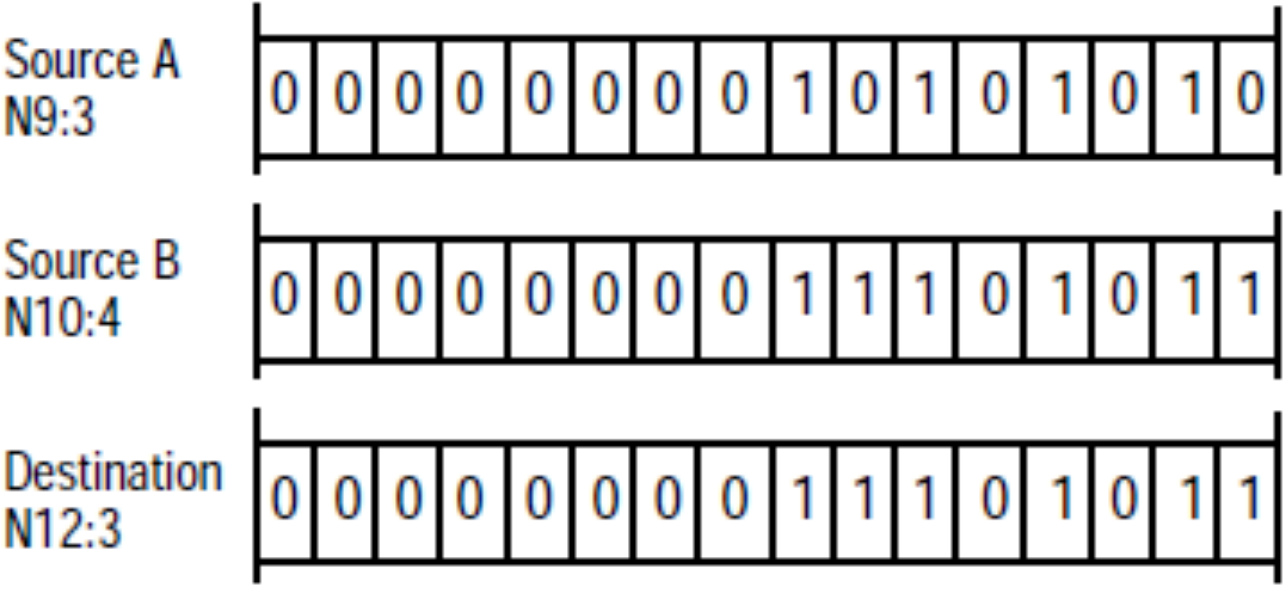

**Truth Table for an OR Operation**

| Source A | Source B | Result |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 1 |

**Example:**

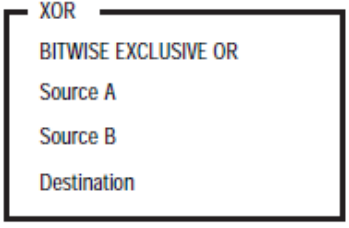



If input word 12, bit 10 is set, the processor performs an OR operation on N9:3 and N10:4 and stores the result in N12:3.


Source A
N9:3
0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0
Source B
N10:4
0 0 0 0 0 0 0 0 1 1 1 0 1 0 1 1
Destination
N12:3
0 0 0 0 0 0 0 0 1 1 1 0 1 0 1 1

# Exclusive OR Operation (XOR)

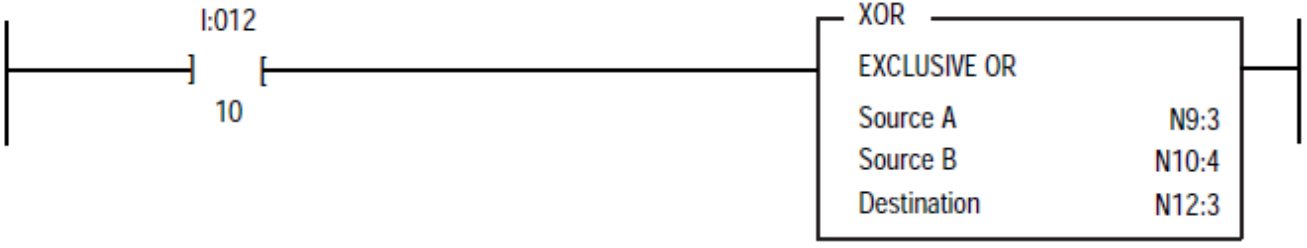
**Description:** Use the XOR instruction to perform an exclusive OR operation using the bits in the two sources (constants or addresses).



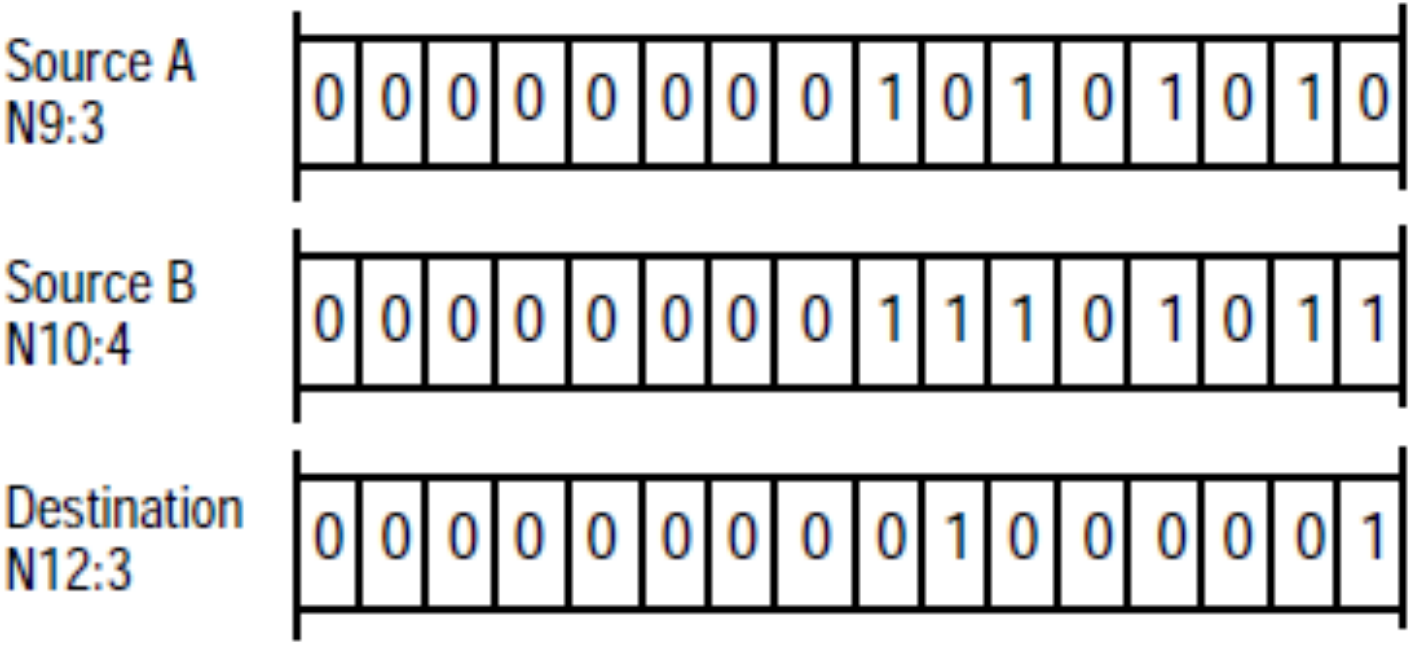

**Truth Table for an XOR Operation**

| Source A | Source B | Result |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 0 |

**Example:**

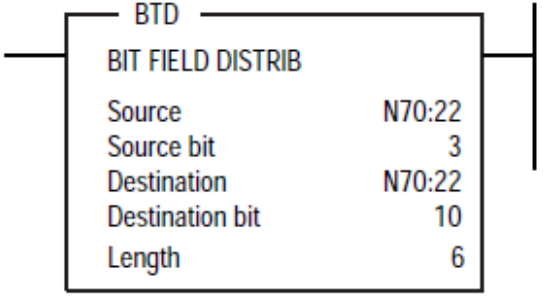


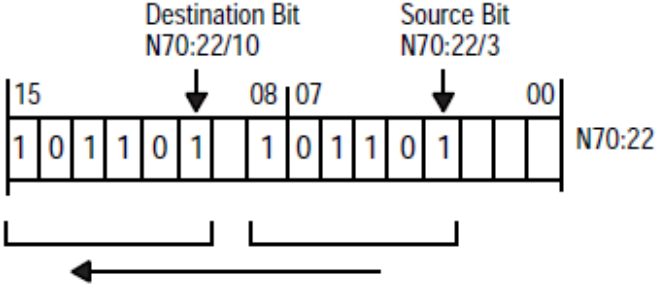

If input word 12, bit 10 is set, the processor performs an XOR operation on N9:3 and N10:4 and stores the result in N12:3.


Source A
N9:3
0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0
Source B
N10:4
0 0 0 0 0 0 0 0 1 1 1 0 1 0 1 1
Destination
N12:3
0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1

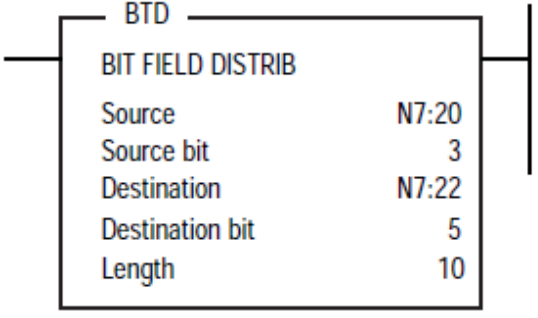
# Move Instructions

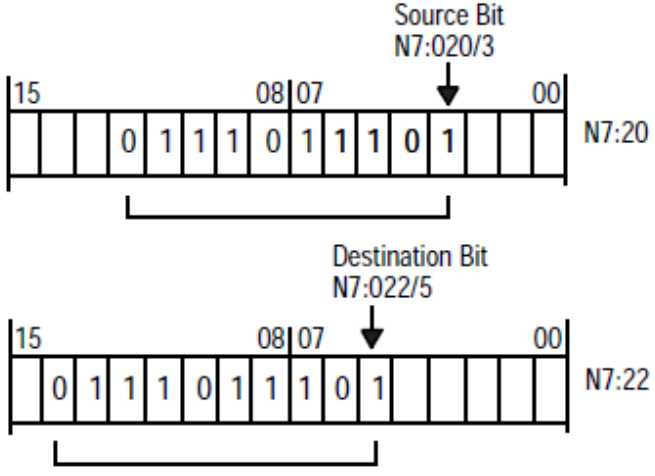
## Moving Bits Within a Word



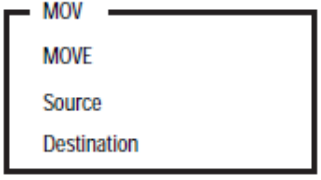
# Moving Bits Between Words



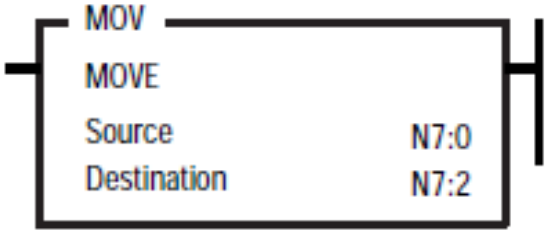
# Move (MOV)

**Description:**




The MOV instruction is an output instruction that copies the Source address to a Destination. As long as the rung remains true, the instruction moves the data each scan.

**Example:**

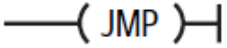


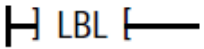

To program this instruction, you must provide the processor with the following:

| Parameter: | Definition: |
|---|---|
| source | is a program constant or data address from which the instruction reads an image of the value.<br>You can also use a symbol, as long as the symbol name is more than 1 character. The source remains unchanged. |
| destination | the data address to which the instruction writes the result of the operation. The instruction writes over any data stored at the destination. |

# Jump (JMP) and Label (LBL)

**Description:** Use JMP and LBL instructions in pairs to skip portions of the ladder program.

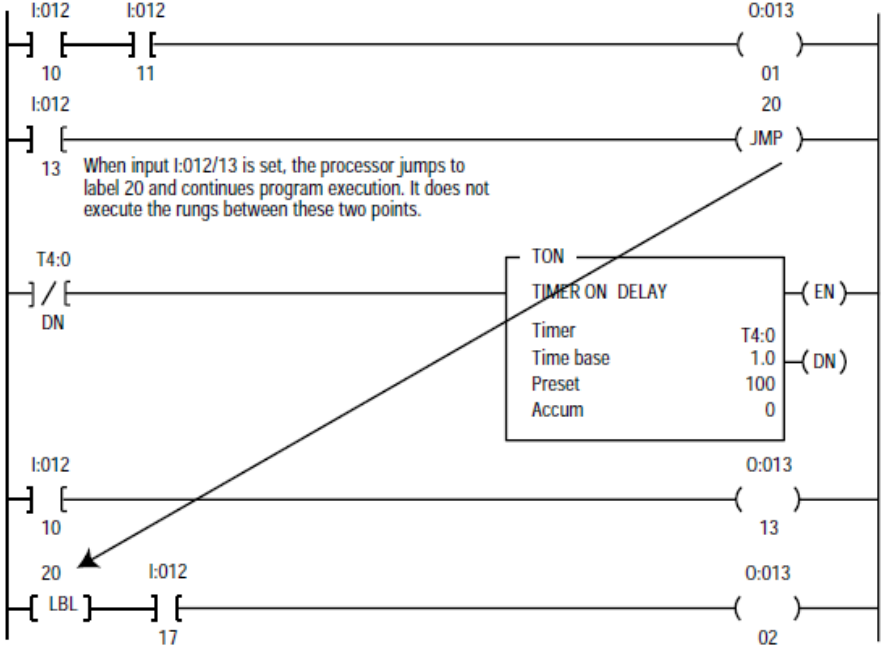



| If the Jump Rung Is: | Then the Processor: |
|---|---|
| True | Skips from the JMP rung to the LBL rung and continues executing the program. You can jump forward or backward. |
| False | Ignores the JMP instruction |

Jumping forward to a label saves program scan time by omitting a program segment until needed. Jumping backward lets the processor repeat iterations through a program segment until its logic is complete.

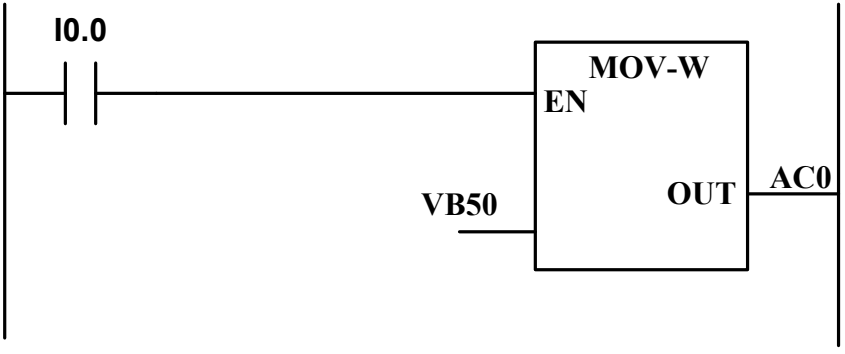
**JMP and LBL Example:** When the rung containing the JMP instruction goes true, the processor jumps over successive rungs until it reaches the rung that contains the LBL instruction with the same number. The processor resumes executing at the LBL rung.



# PLC

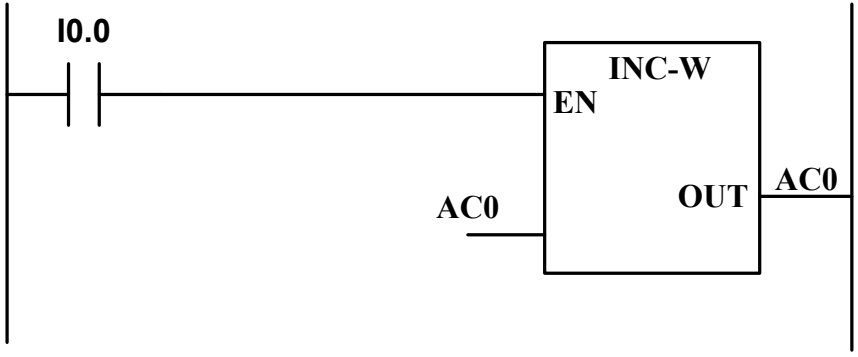
## Math-Comp-Logic-Siemens Instruction

# Move



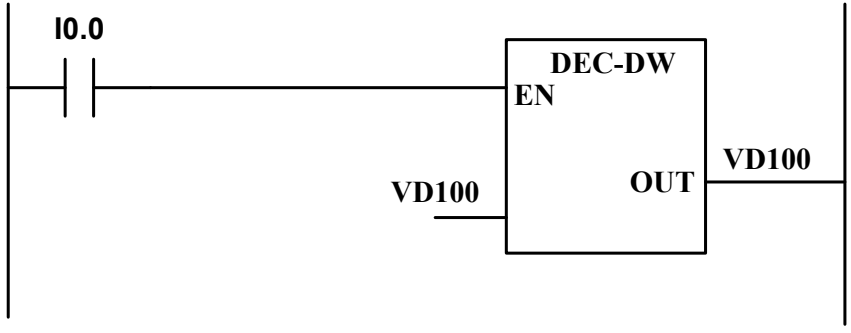

```
LD      I0.0
MOVW    VB50, AC0
MEND
```

# INCREMENT (INC)



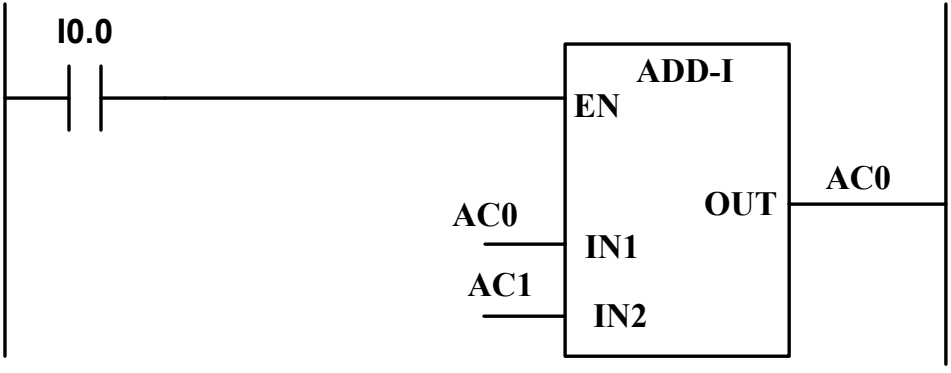

```
LD      I0.0
INCW    AC0
MEND
```

# DECREMENT (DEC)



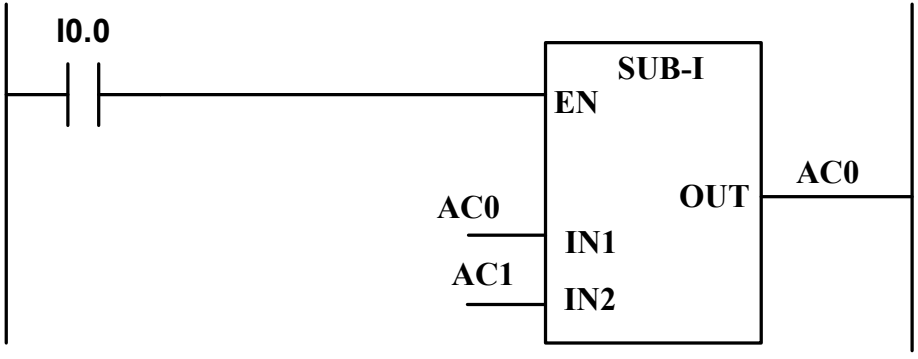

```
LD      I0.0
DECDW   VD100
MEND
```

# ADD



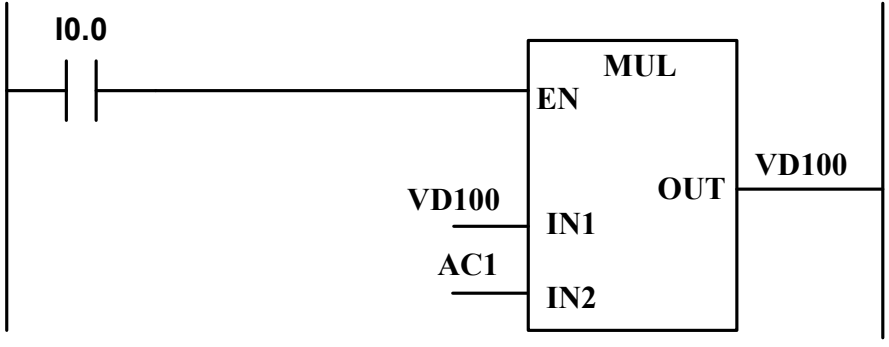

```
LD      I0.0
ADD     AC1,AC0
MEND
```

# SUB



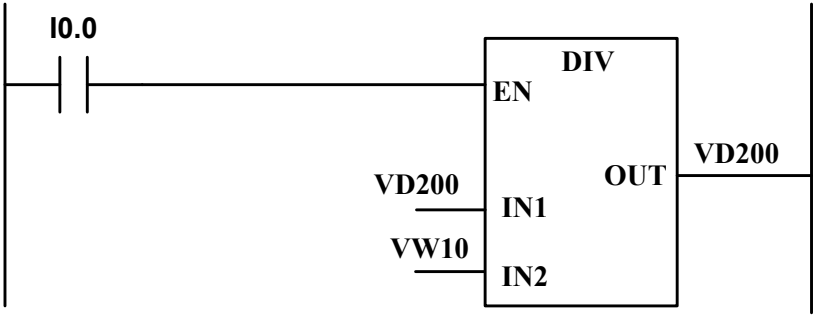

```
LD      I0.0
SUB     AC1,AC0
MEND
```

# MUL



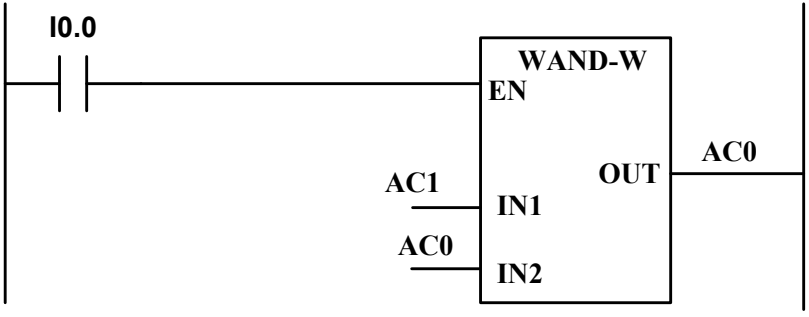

```
LD      I0.0
MUL     AC1,VD100
MEND
```

# DIV



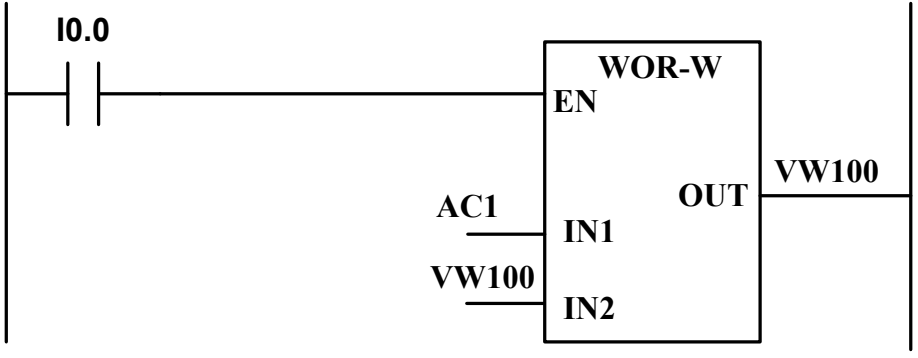

```
LD      I0.0
DIV     VW10,VD200
MEND
```

# Logic Gate AND



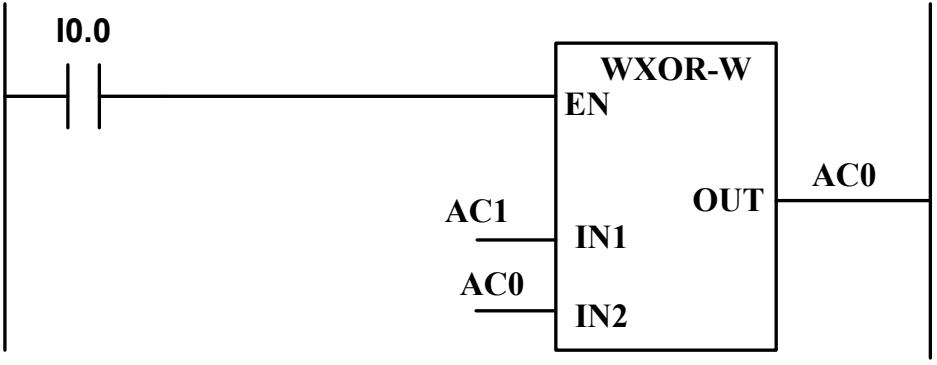

```
LD      I0.0
ANDW    AC1,AC0
MEND
```

# Logic OR



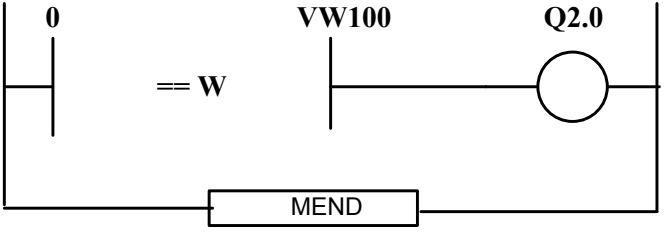

```
LD      I0.0
ORW     AC1,VW100
MEND
```

# Logic Gate XOR



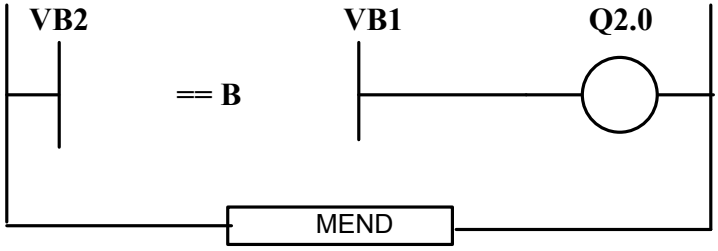

```
LD      I0.0
XORW    AC1,AC0
MEND
```

# Compare Instruction



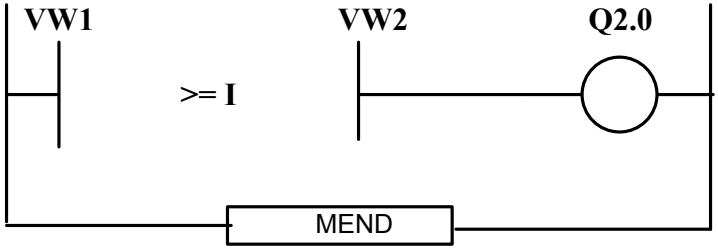

```
LD W==   #0,VW100
=        Q2.0
MEND
```

W  Word



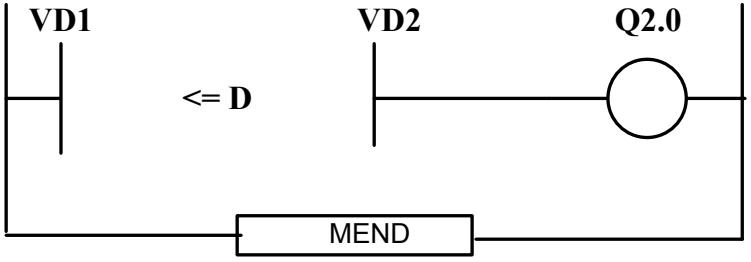

```
LD B==   VB2,VB1
=        Q2.0
MEND
```

B Byte




```
LD I>=    VW1,VW2
=         Q2.0
MEND
```

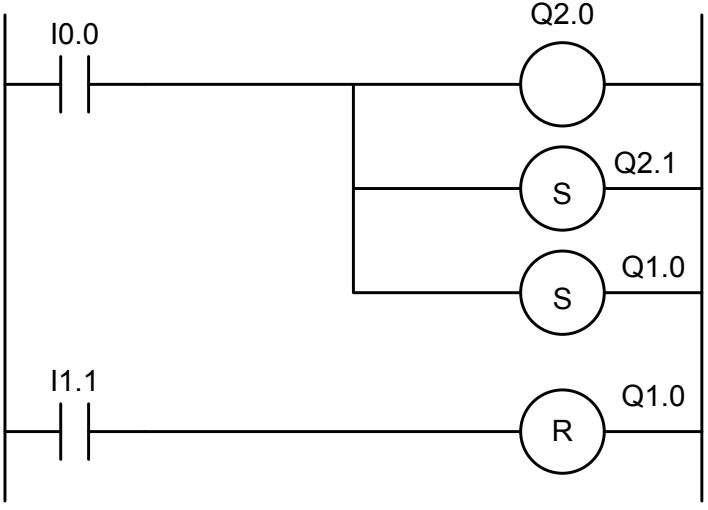
I Integer



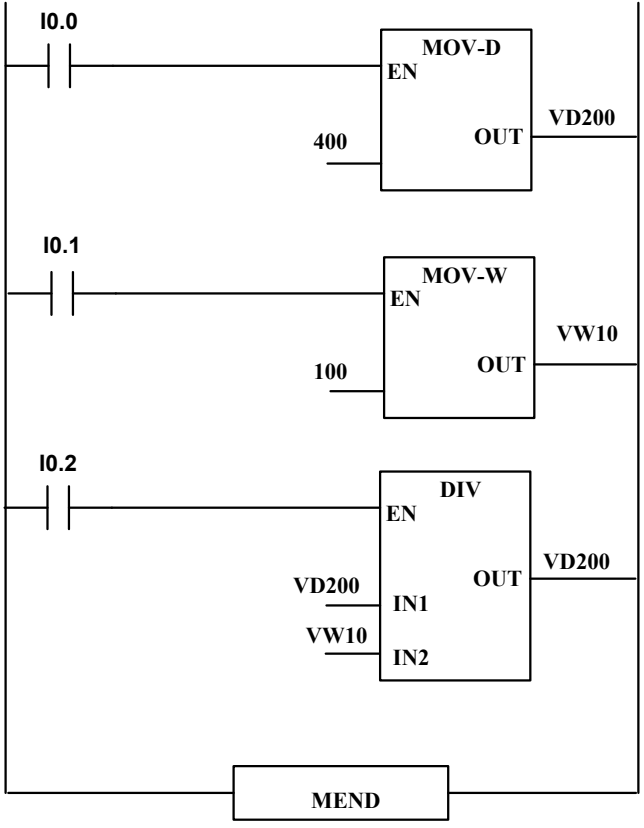

```
LD  D <=    VD1,VD2
=           Q2.0
MEND
```

D  Double Integer

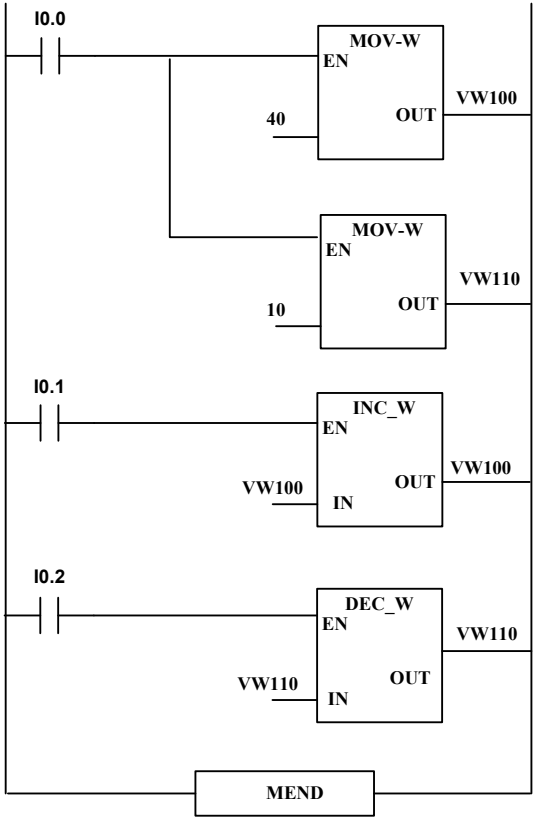
# Set (S) and Rest (R)




```
LD      I0.0
=       Q2.0
S       Q2.1
S       Q1.0
LD      I1.1
R       Q1.0
MEND
```




```
LD      I0.0
MOVD    #400,VD200
LD      I0.1
MOVW    # 100,VW10
LD      I0.2
DIV     VW10,VD200
MEND
```




```
LD      I0.0
MOVW    #40,VW100
MOVW    # 10,VW110
LD      I0.1
INCW    VW100
LD      I0.2
DECW    VW110
MEND
```

# Drilling Process

## Example

Devise a ladder program that could be used to operate the following task.

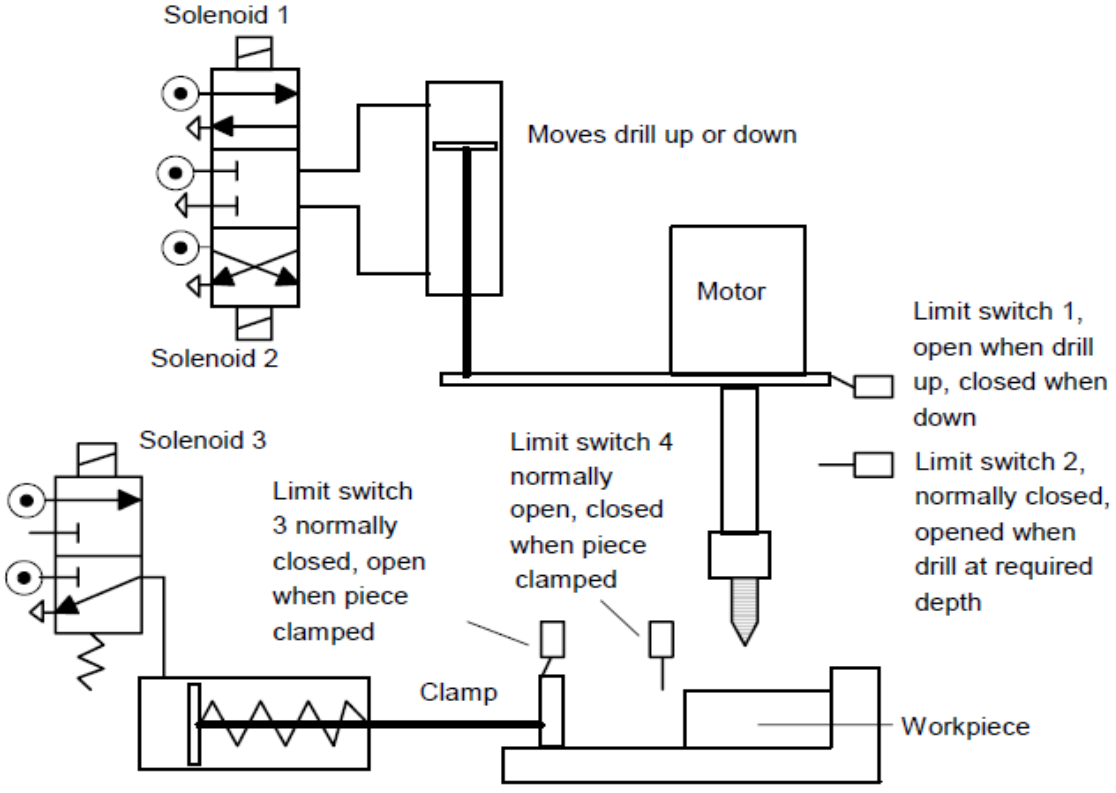
1- Start the drill motor and the pump for the air pressure for the pneumatic valves.

2- Clamp the workpiece .

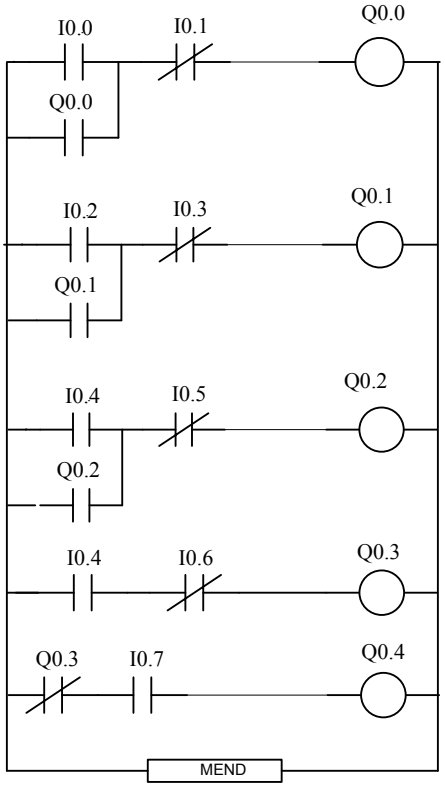
3- Lower the drill and drilling started to the required depth.

4- Then retract the drill and unclamp the workpiece.

The task is shown in the following figure


Solenoid 1
Moves drill up or down
Motor
Solenoid 2
Limit switch 1, open when drill up, closed when down
Limit switch 2, normally closed, opened when drill at required depth
Solenoid 3
Limit switch 4 normally open, closed when piece clamped
Limit switch 3 normally closed, open when piece clamped
Clamp
Workpiece

I0.0 Motor Start P.B
I0.1 Motor Stop P.B
I0.2 Pump Start P.B
I0.3 Pump Stop P.B
I0.4 Limit switch 3
I0.5 Limit Switch 4
I0.6  Limit Switch 2
I0.7 Limit Switch 1
Q0.0 Motor
Q0.1 Pump
Q0.2 Solenoid 3
Q0.3 Solenoid 1
Q0.4 Solenoid 2


I0.0
I0.1
Q0.0
Q0.0
I0.2
I0.3
Q0.1
Q0.1
I0.4
I0.5
Q0.2
Q0.2
I0.4
I0.6
Q0.3
Q0.3
I0.7
Q0.4
MEND

# PLC

Example

# Heating and Mixing of Products

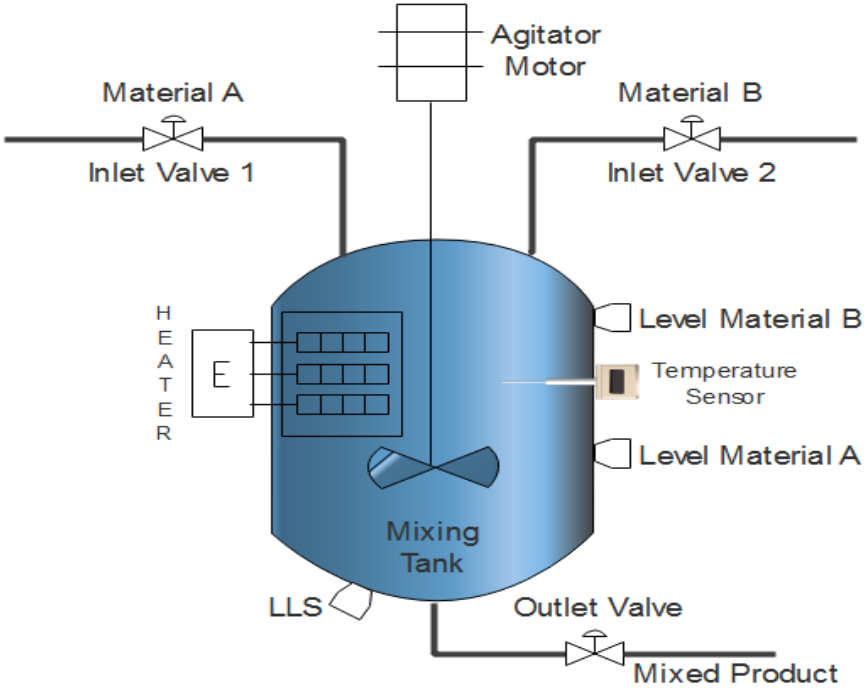
**Description**

Material A and Material B are collected in a tank.

These products are to be mixed and heated till it reaches set point temperature value.

Implement automation of this system in PLC using Ladder Diagram programming language and instruction list for an S7-200 Siemens PLC.

# Diagram


Agitator
Motor
Material A
Material B
Inlet Valve 1
Inlet Valve 2
HEATER
E
Level Material B
Temperature
Sensor
Level Material A
Mixing
Tank
LLS
Outlet Valve
Mixed Product

**Solution**

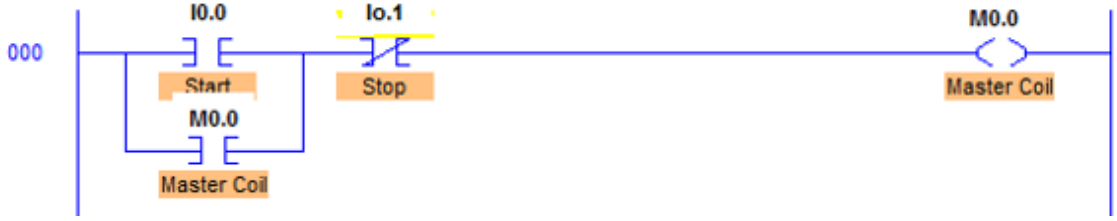
- To detect level of Material A and Material B, two separate level switches are used.
- And to detect low level, one more level switch is used at the bottom of the tank.
- These give output in digital terms that is when corresponding levels are detected.
- To control level of this system, Single Acting Piston valve can be used which has two states, either fully open or fully close.

- To control mixing, motor is used .
- Heater is installed inside the tank and temperature sensor is used to detect the temperature of liquid in the tank.
- This mixer is mixed until set point temperature is achieved.
- Outlet valve is then operated to drain the mixed product.

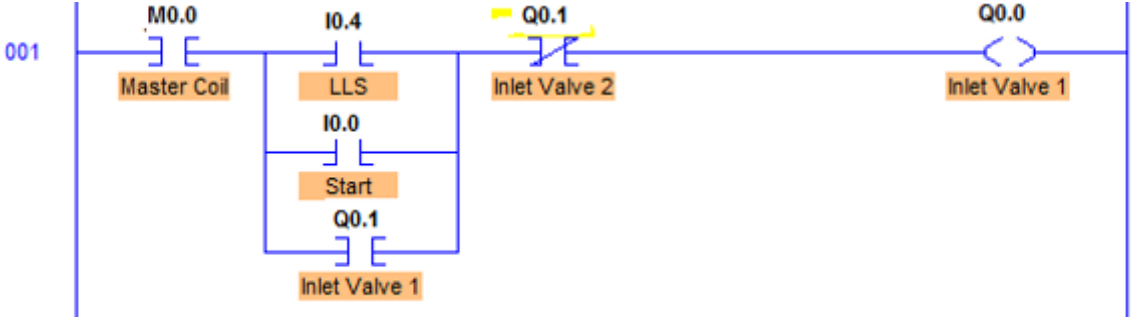
**List of Inputs and Outputs**

| | | |
|---|---|---|
| I0.0 | Start | (Input) |
| I0.1 | Stop | (Input) |
| M0.0 | Master Coil | (Internal) |
| Io.2 | Level of Material B | (Input) |
| I0.3 | Level of Material A | (Input) |
| I0.4 | Low Level Switch (empty tank) | (Input) |
| I0.5 | Temperature detector | (Input) |

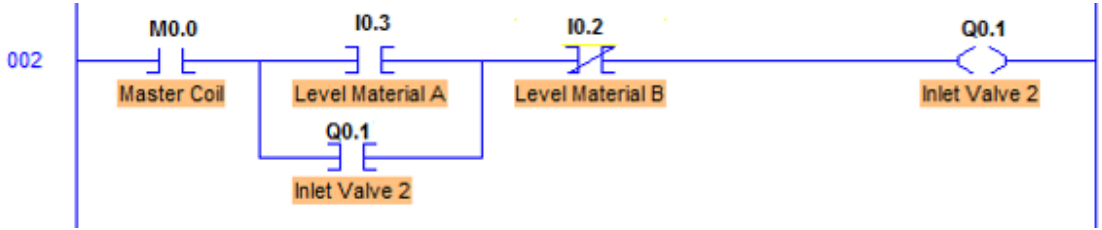
| | | |
|---|---|---|
| Q0.0 | Inlet Valve 1 (Material A Feed) | (Output) |
| Q0.1 | Inlet Valve 2 (Material B Feed) | (Output) |
| Q0.2 | Agitator Motor (Mixing) | (Output) |
| Q0.3 | Heater | (Output) |
| Q0.4 | Outlet valve to drain the product | (Output) |



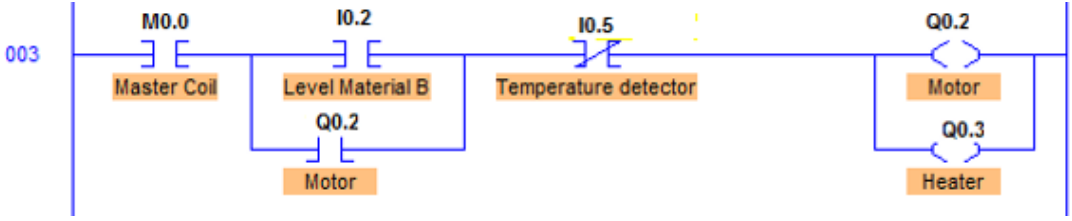

- RUNG000 contain master start/stop with address of Start PB I0.0 and Stop PB I0.1 and internal relay.



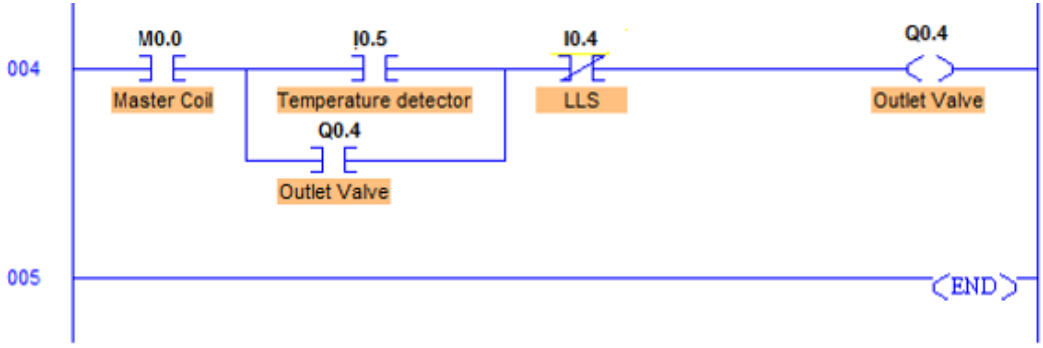

- RUNG001 is to operate Inlet Valve of Material A with address Q0.0. It is operated when Low Level of the tank is detected by Level Low Switch I0.4. And it is closed when Inlet Valve of material B starts that is when Level Material A is detected by a switch with address I0.3.
- Start PB is also connected in parallel with the LLS, it is done so that when LLS or level of Material A is not detected, Inlet Valve is operated by Start PB.



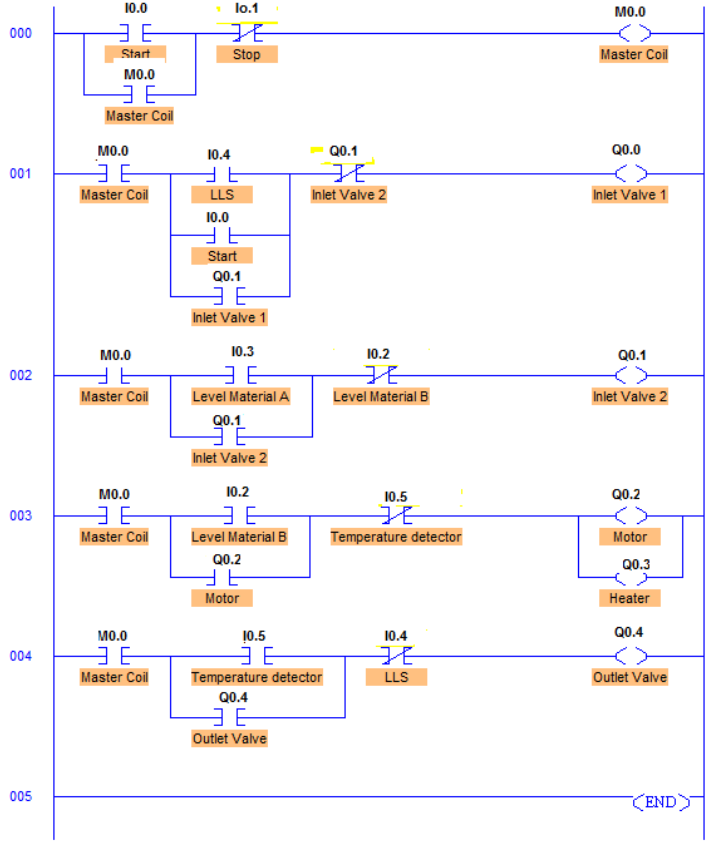

- RUNG002 is to operate Inlet Valve of Material B with address Q0.2. It is opened when Material A is filled to its desired level (Level Material A). In other words, Valve of Material B is opened when Level of Material A is detected by I0.3 and it is closed when Level of Material B is detected or Tank is full.




- RUNG003 operates Motor and Heater with address Q0.2 and Q0.3 respectively.
- When the tank is full with Material A and B, Level High (Level Material B) is detected. Motor is connected to the address Q0.2 and Heater to Q0.3. Motor and Heater are run when I0.2 is set to 1 that is when tank is full.




- RUNG004 is for Outlet Valve with address Q0.3. It is operated when the entire mixing and heating process is completed that is when temperature set point is reached. And this valve is closed when LLS is again detected.


000
I0.0
Start
Io.1
Stop
M0.0
Master Coil
M0.0
Master Coil
M0.0
Master Coil
001
I0.4
LLS
Q0.1
Inlet Valve 2
Q0.0
Inlet Valve 1
I0.0
Start
Q0.1
Inlet Valve 1
002
M0.0
Master Coil
I0.3
Level Material A
I0.2
Level Material B
Q0.1
Inlet Valve 2
Q0.1
Inlet Valve 2
003
M0.0
Master Coil
I0.2
Level Material B
I0.5
Temperature detector
Q0.2
Motor
Q0.3
Heater
Q0.2
Motor
004
M0.0
Master Coil
I0.5
Temperature detector
I0.4
LLS
Q0.4
Outlet Valve
Q0.4
Outlet Valve
005
END

# PLC Example

## Generate Outputs Based on Equations

# Generate Outputs Based on Equations

## Description

The following calculation will be made when input I:1/0 is true.

$$X = 2^{Y} \sqrt{(1 + \sin Y)}$$

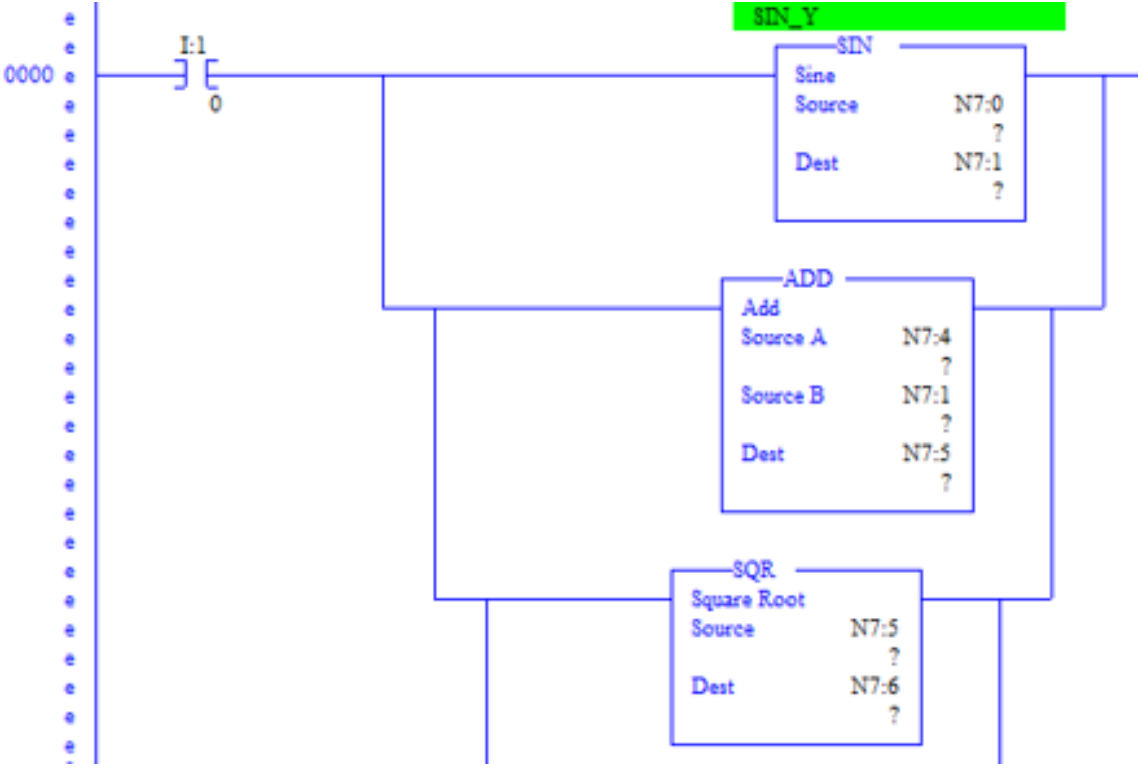
If the result x is between 1 and 10 then the Output 0 will be turned on. The value of x will be output as an Analog voltage. Perform this operation in PLC using Ladder Diagram programming language.

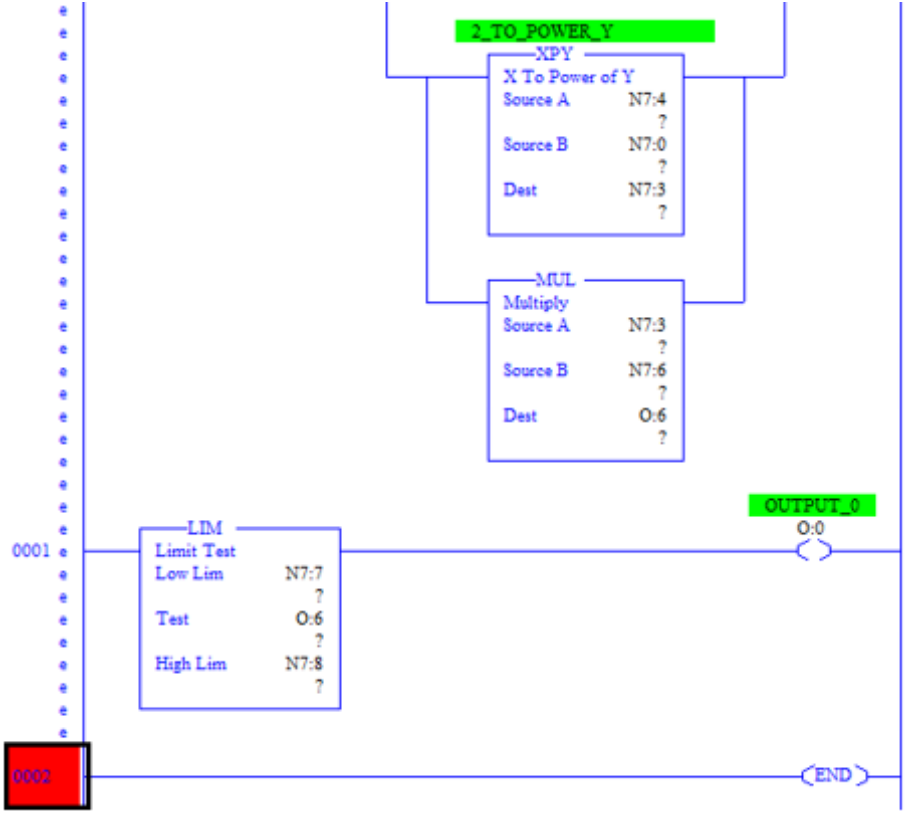
## Solution:

- Perform tasks individually as defined below.
- Assign integer number addresses to store the 16/32bit data into registers.
- Use this addresses in the mathematical instructions as Source and Destination.
- Use (X to power Y) instruction to find the input 2 to power y.
- Use SIN instruction to find Sin y. Add this with 1 by using ADD instruction.
- Square Root this 1 + Sin y by using SQR instruction and multiply the output of XPY and SQR.
- Compare this data with the lower and upper limit and control output.

**List of Inputs and Outputs**

```
I:1/0   = Start Input                                    (Input)
N7:0 = Y input to SIN function                           (Input)
N7:1 = Sin Y operation data                              (Input)
N7:4 = Numeric Value “1” (To perform 1+Sin y)            (Input)
N7:5 = Output of addition 1+Sin Y                        (Input)
N7:6 = Square Root of 1+Sin Y                            (Input)
N7:4 = Numeric Value “2” (To perform 2^y)                (Input)
N7:3 = Output of X= (2^y) Square Root of 1+SinY          (Input)
N7:7 = Lower Limit (01)                                  (Input)
N7:8 = Higher Limit (10)                                 (Input)
O:6  = Output x, multiplication of N7:6 & N7:3           (Output)
O:0  = Output 0                                          (Output)
```


SIN_Y
0000
I:1
0
SIN
Sine
Source
N7:0
?
Dest
N7:1
?
ADD
Add
Source A
N7:4
?
Source B
N7:1
?
Dest
N7:5
?
SQR
Square Root
Source
N7:5
?
Dest
N7:6
?


2_TO_POWER_Y
XPY
X To Power of Y
Source A N7:4
?
Source B N7:0
?
Dest N7:3
?
MUL
Multiply
Source A N7:3
?
Source B N7:6
?
Dest O:6
?
OUTPUT_0
O:0
0001
LIM
Limit Test
Low Lim N7:7
?
Test O:6
?
High Lim N7:8
?
0002
END

## Description

- As we can see, step by step procedure is done.
- Output of individual operations are first obtained.
- Output of all these data are stored into Integer Registers.
- Operation starts from the smallest function as shown in the RUNG0000 that first Sin Y(N7:0) is obtained, then Sin Y is added with the integer 1.
- Output of this 1+Sin Y is then square rooted using SQR instruction and result is stored into the register N7:6.

- Then 2^y is performed. This is obtained by using XPY (X to power Y) instruction where X is N7:4 (digit 2) and Y is N7:0 (y).
- Output of this both are Multiplied to finally obtain the output of the equation X (O:6).
- LIM instruction is then used to compare output of the equation with its lower limit and upper limit. Accordingly output is generated.
- If O:6 is between 1 to 10, output O:0 energizes turning ON the Output 1.

### Tasks for a Sample Mixing Tank Application

Figure 4-2 shows the diagram for a mixing tank. This mixing tank can be used for different applications, such as for making different colors of paint. In this application, two pipelines enter the top of the tank; these supply two different ingredients. A single pipeline at the bottom of the tank transports the finished mixture. The sample program controls the filling operation, monitors the tank level, and controls a mixing and heating cycle. The following tasks describe the process:

Step 1: Fill the tank with Ingredient 1.

Step 2: Fill the tank with Ingredient 2.

Step 3: Monitor the tank level for closure of the high-level switch.

Step 4: Maintain the pump status if the start switch opens.

Step 5: Start the mix-and-heat cycle.

Step 6: Turn on the mixer motor and steam valve.

Step 7: Drain the mixing tank.

Step 8: Count each cycle.

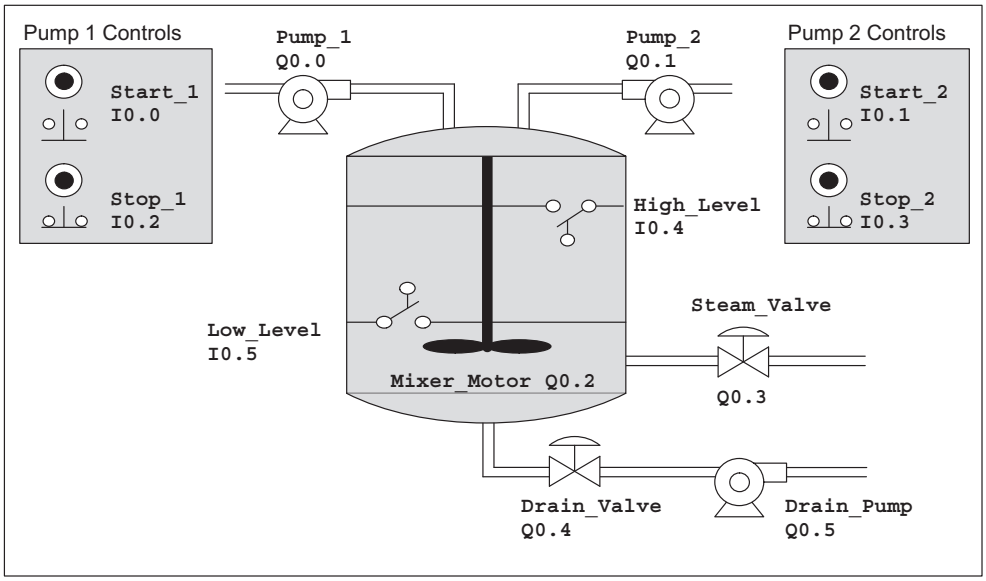



Figure 4-2 Program Example: Mixing Tank

### Sample Program in Statement List (STL) and Ladder Logic

You can enter the sample program in either statement list (STL) or ladder representation. Table 4-1 provides the STL version of the sample program, and Figure 4-3 shows the same sample program in ladder. Sections 4.2 to 4.4 guide you through the tasks required to enter this program in ladder.

Table 4-1 Sample Program in Statement List

```
STL                              Description
NETWORK 1                        //Fill the tank with Ingredient 1.
LD      "Start_1"
O       "Pump_1"
A       "Stop_1"
AN      "High_Level"
=       "Pump_1"

NETWORK 2                        //Fill the tank with Ingredient 2.
LD      "Start_2"
O       "Pump_2"
A       "Stop_2"
AN      "High_Level"
=       "Pump_2"

NETWORK 3                        //Set memory bit if high level is reached.
LD      "High_Level"
S       "Hi_Lev_Reached", 1

NETWORK 4                        //Start timer if high level is reached.
LD      "Hi_Lev_Reached"
TON     "Mix_Timer", +100

NETWORK 5                        //Turn on the mixer motor.
LDN     "Mix_Timer"
A       "Hi_Lev_Reached"
=       "Mixer_Motor"
=       "Steam_Valve"

NETWORK 6                        //Drain the mixing tank.
LD      "Mix_Timer"
AN      "Low_Level"
=       "Drain_Valve"
=       "Drain_Pump"

NETWORK 7                        //Count each cycle.
LD      "Low_Level"
A       "Mix_Timer"
LD      "Reset"
CTU     "Cycle_Counter", +12

NETWORK 8                        //Reset memory bit if low level or time-out.
LD      "Low_Level"
A       "Mix_Timer"
R       "Hi_Lev_Reached", 1

NETWORK 9                        //End the main program.
MEND
```

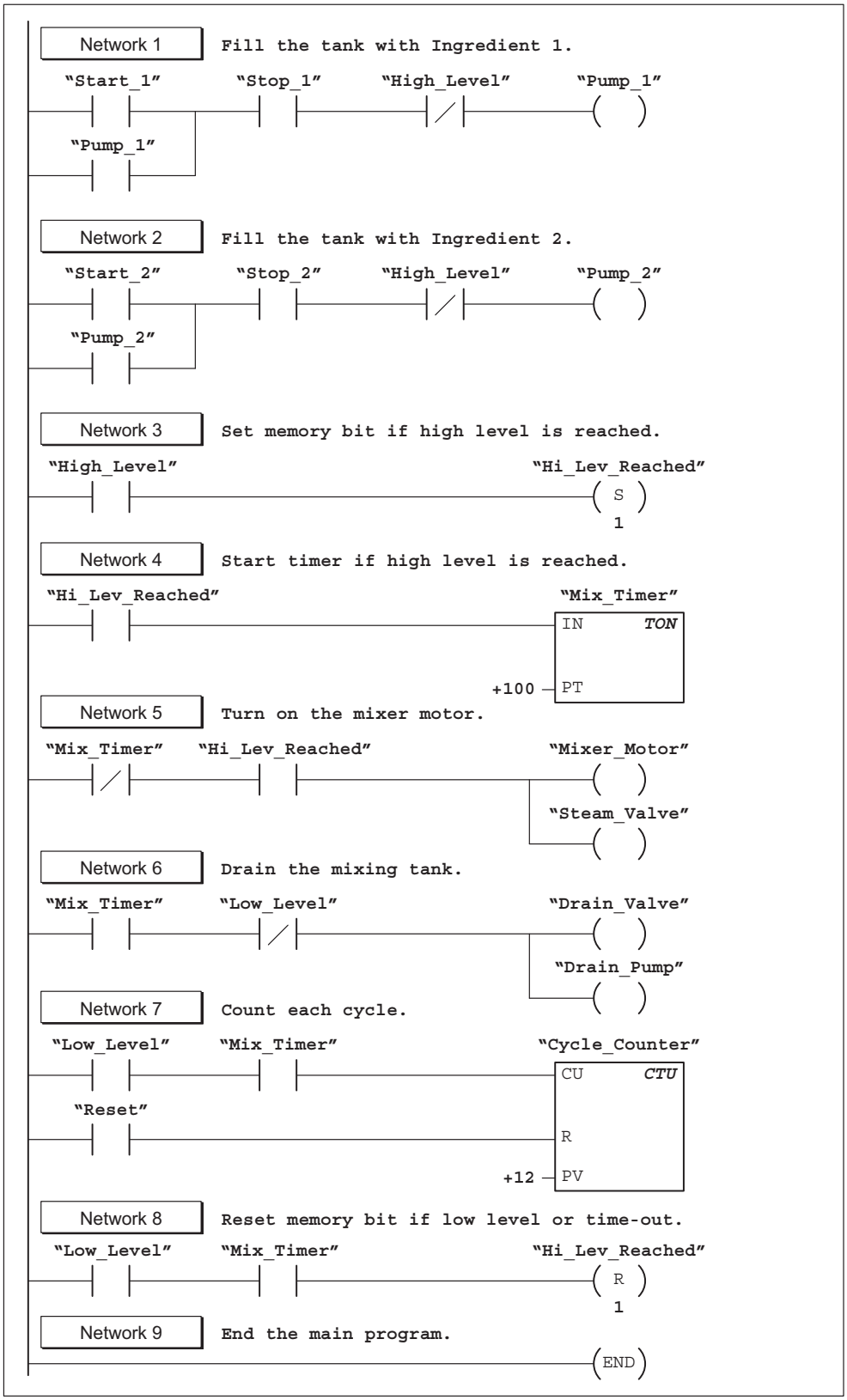



Figure 4-3 Sample Program in Ladder Logic

## 4.2 Task: Create a Project

### Creating a New Project

When you create or open a project, STEP 7-Micro/WIN starts the Ladder or STL Editor (OB1), and depending on your selected preference, the Data Block Editor (DB1), the Status Chart, and the Symbol Table.

To create a new project, select the menu command **Project ▸ New...**, as shown in Figure 4-4, or click the New Project toolbar button. 

The CPU Type dialog box is displayed. Select your CPU type from the drop-down list box.

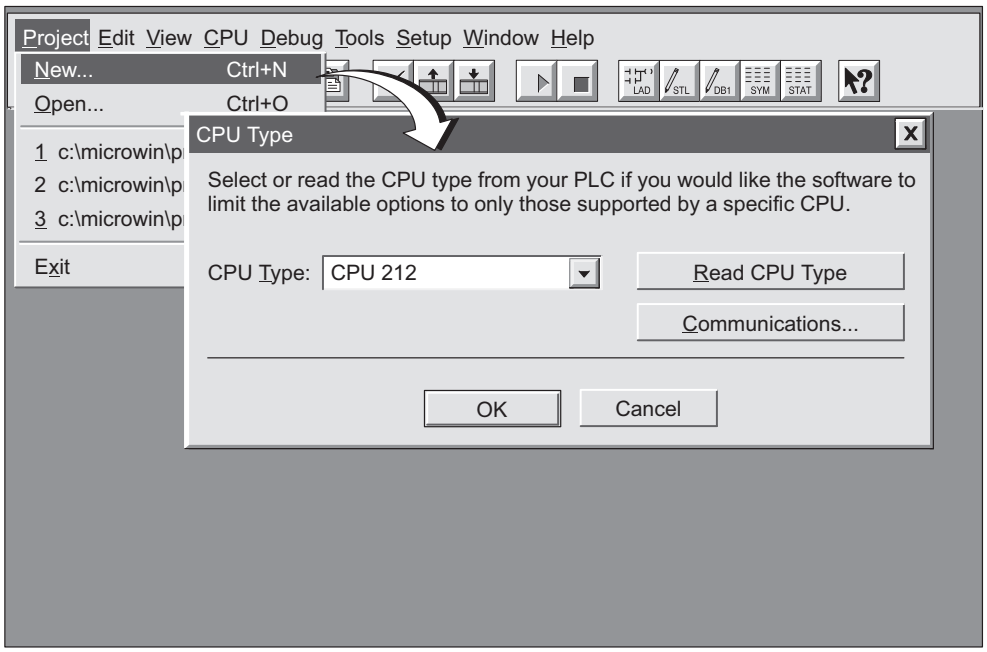



Figure 4-4 Creating a New Project and Selecting the CPU Type

### Naming the Sample Project

You can name your project at any time; for this example, refer to Figure 4-5 and follow these steps to name the project:

1. Select the menu command **Project ▸ Save As...** .
2. In the File Name field, type the following: `project1.prj`
3. Click the "Save" button.

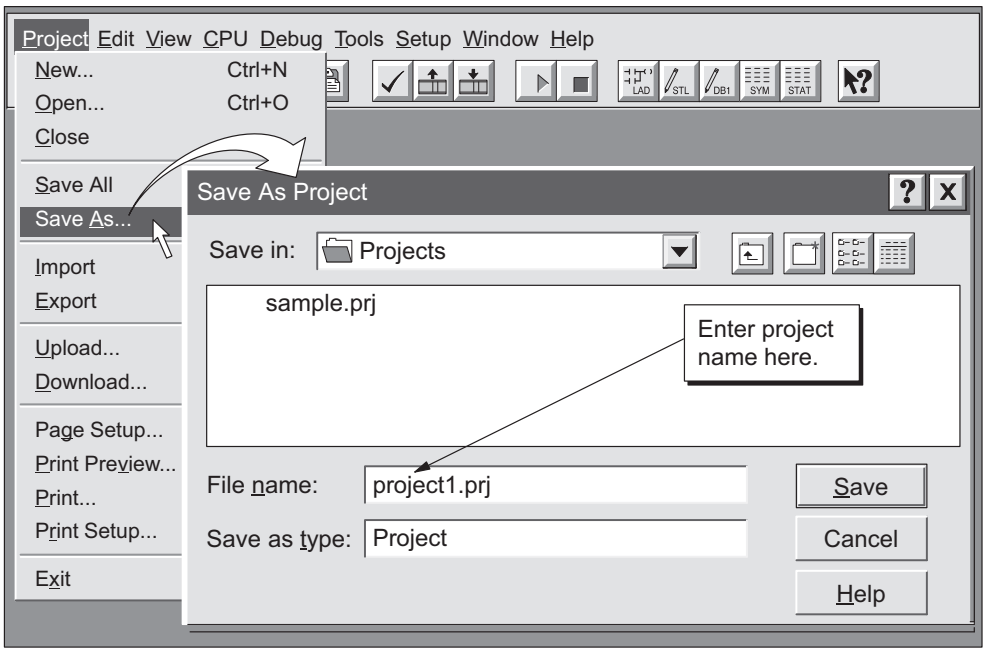



Figure 4-5 Naming the Sample Project

## 4.3 Task: Create a Symbol Table

### Opening the Symbol Table Editor

To define the set of symbol names used to represent absolute addresses in the sample program, open the Symbol Table editor. Double-click the icon, or click the Restore or Maximize button on the icon (in Windows 95). You can also select the menu command **View ▸ Symbol Table**.

### Entering the Symbol Names

Figure 4-6 shows the list of symbol names and the corresponding addresses for the sample program. To enter the symbol names, follow these steps:

1. Select the first cell in the Symbol Name column, and type the following: `Start_1`
2. Press ENTER to move the focus to the first cell in the Address column. Type the address `I0.0` and press ENTER. The focus moves to the cell in the Comment column. (Comments are optional, but they are a useful way to document the elements in your program.)
3. Press ENTER to start the next symbol row, and repeat these steps for each of the remaining symbol names and addresses.
4. Use the menu command **Project ▸ Save All** to save your Symbol Table.

Symbol Table - c:\microwin\project1.sym

| Symbol Name | Address | Comment |
|---|---|---|
| Start_1 | I0.0 | Start switch for Paint Ingredient 1 |
| Start_2 | I0.1 | Start switch for Paint Ingredient 2 |
| Stop_1 | I0.2 | Stop switch for Paint Ingredient 1 |
| Stop_2 | I0.3 | Stop switch for Paint Ingredient 2 |
| High_Level | I0.4 | Limit switch for maximum level in tank |
| Low_Level | I0.5 | Limit switch for minimum level in tank |
| Reset | I0.7 | Reset control for counter |
| Pump_1 | Q0.0 | Pump for Paint Ingredient 1 |
| Pump_2 | Q0.1 | Pump for Paint Ingredient 2 |
| Mixer_Motor | Q0.2 | Motor for the tank mixer |
| Steam_Valve | Q0.3 | Steam to heat mixture in tank |
| Drain_Valve | Q0.4 | Valve to allow mixture to drain |
| Drain_Pump | Q0.5 | Pump to drain mixture out of tank |
| Hi_Lev_Reached | M0.1 | Memory bit |
| Mix_Timer | T37 | Timer to control mixing/heating mixture |
| Cycle_Counter | C30 | Counts total mix & heat cyles completed |

Figure 4-6 Symbol Table for the Sample Program

### Programming with Symbolic Addresses

Before you start entering your program, make sure the ladder view is set for symbolic addressing. Use the menu command **View ▸ Symbolic Addressing** and look for a check mark next to the menu item, which indicates that symbolic addressing is enabled.

**Note**
Symbol names are case-sensitive. The name you enter must match exactly the uppercase and lowercase characters entered in the symbol table. If there is any mismatch, the cursor stays on the element and the message “Invalid parameter” appears in the status bar at the bottom of the main window.

## 4.4 Task: Enter the Program in Ladder Logic

### Opening the Ladder Editor

To access the Ladder Editor, double-click the icon at the bottom of the main window. Figure 4-7 shows some of the basic tools in the Ladder Editor.

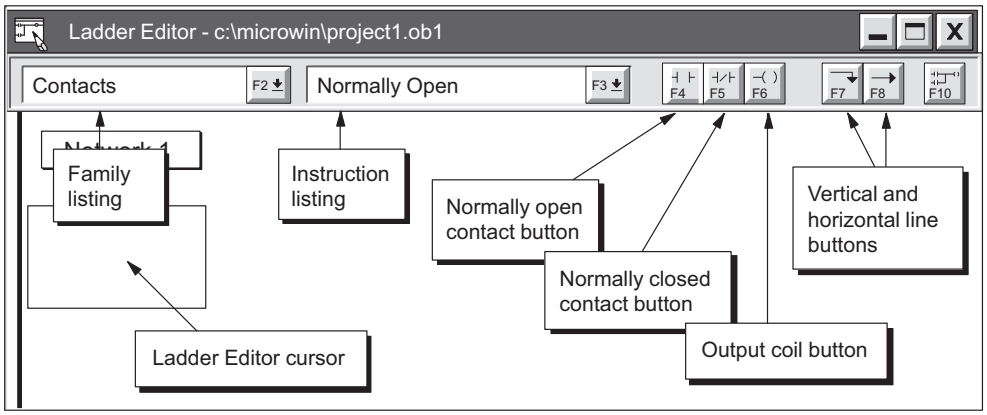



Figure 4-7 Some of the Basic Ladder Editor Tools

### Instruction Toolbar in the Ladder Editor

You can also select **View ▸ Instruction Toolbar** to display the Ladder Instruction Toolbar. See Figure 4-8.

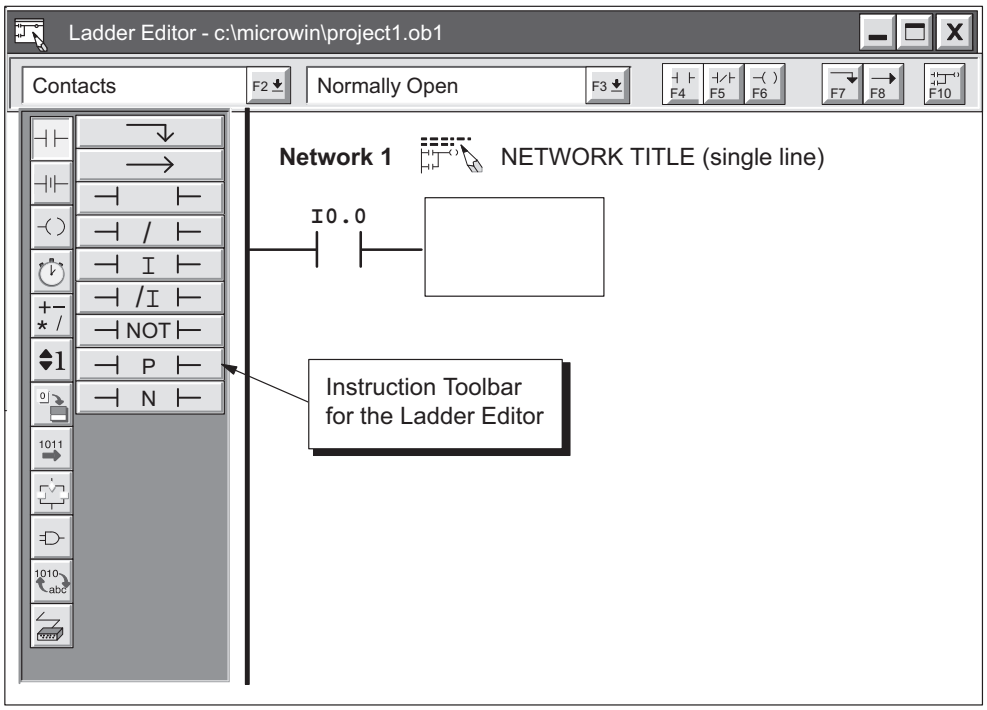



Figure 4-8 Some of the Basic Ladder Editor Tools

### Entering the First Network Element

Follow these steps to enter the first network of the sample program:

1. Double click on or near the numbered network label to access the Title field in the Comment Editor. Type the comment shown in Figure 4-9, and click “OK.”
2. Press the down arrow key. The ladder cursor moves down to the first column position on the left.
3. Select the normally open by selecting “Contacts” from the family listing and then selecting “Normally Open” from the instruction listing.
4. Press ENTER, and a normally open contact appears with the name “Start_1” highlighted above it.

   (Every time you enter a contact, the software displays the default address of **I0.0**, which in this example is defined as Start_1 in the Symbol Table.)
5. “Start_1” is the first element required for Network 1. Press ENTER to confirm the first element and its symbol name. The ladder cursor moves to the second column position.

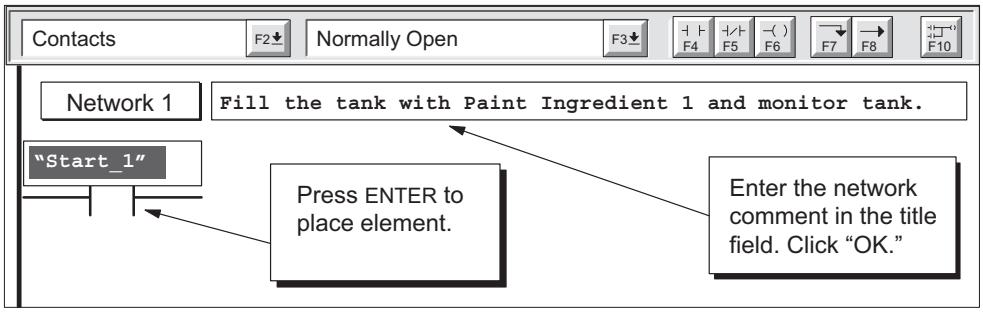



Figure 4-9 Entering the Network Comment and the First Ladder Element

Follow these steps to enter the remaining contacts of the first network:

1. Press ENTER to enter the second element. A normally open contact appears with the default symbol name “Start_1” highlighted above it.
2. Type **Stop_1** and press ENTER. The cursor moves to the next column.
3. Click the normally closed contact button (“F5”). A closed contact appears with the default symbol name “Start_1” highlighted above it.
4. Type **High_Level** and press ENTER.

The ladder network should look like the one shown in Figure 4-10.

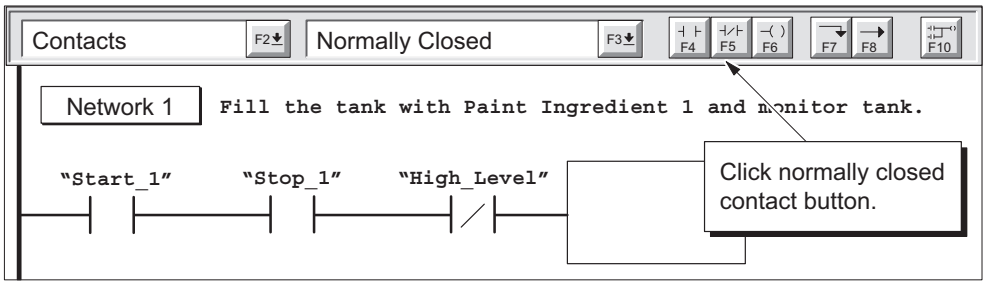



Figure 4-10 Entering the Next Ladder Element

The ladder cursor is now positioned to the right of the normally closed “High_Level” input. Refer to Figure 4-11 and follow these steps to complete the first network:

1. Click the coil button (“F6”) and move the mouse cursor inside the ladder cursor and click. A coil appears with the name “Pump_1” highlighted above it. (Each coil that you enter is given the default address of Q0.0, which in this case has been defined as Pump_1 in the symbol table.)
2. Press ENTER to confirm the coil and its symbol name.
3. Use the mouse or press the left arrow key to move the cursor back to the first element of the current network.
4. Click the vertical line button (“F7”) to draw a vertical line between the first and second contacts.
5. Click the normally open contact button (“F4”) on the toolbar, and press ENTER. A contact with the name “Start_1” appears.
6. Type **`Pump_1`** and press ENTER.

The first network is now complete.

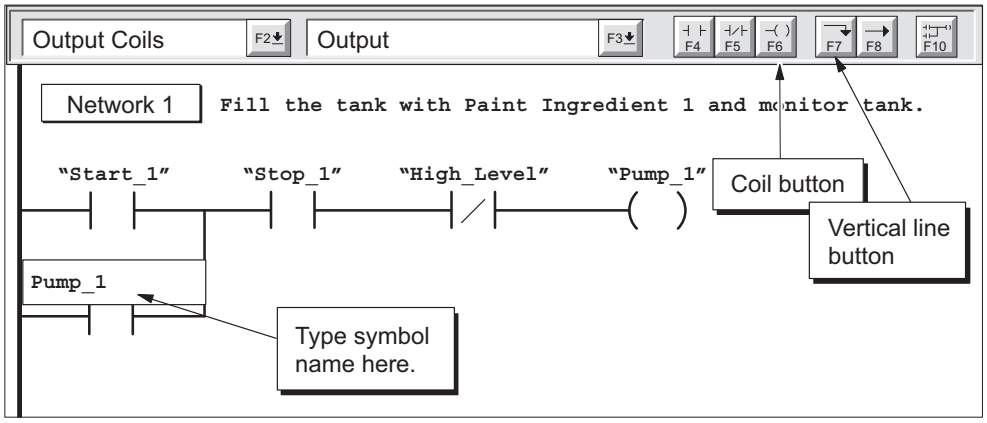



Figure 4-11 Completing the First Network

### Entering the Second Network

Follow these steps to enter the second network of the sample program:

1. Use the mouse or press the down arrow key to move the cursor to Network 2.
2. In the network comment field, type the comment shown in Figure 4-12. (Since the comment for Network 2 is nearly identical to the one for Network 1, you can also select and copy the text from Network 1 and paste it into the comment field for Network 2, then change the paint ingredient number to 2.)
3. Repeat the steps you used to enter the elements of Network 1, using the symbol names shown in Figure 4-12.
4. After you finish Network 2, move the cursor down to Network 3.

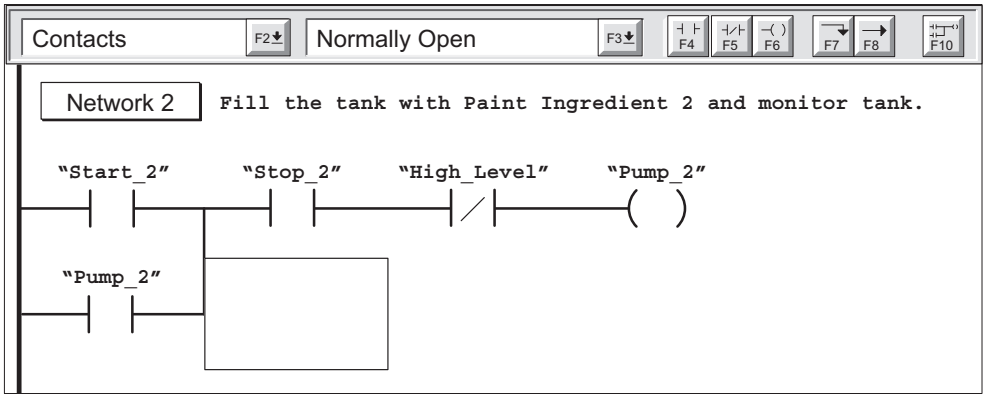



Figure 4-12 Entering the Second Network

### Entering the Remaining Networks

From this point on, you can follow the same general procedures that you have used up to now to enter the remaining networks. Refer to Figure 4-3 for the remaining networks.

### Compiling the Program

After completing the sample program, check the syntax by selecting the menu command **CPU ▸ Compile** or by clicking the Compile button. 

If you have entered all the networks correctly as shown in the sample program, you get a "Compile Successful" message that also includes information on the number of networks and the amount of memory used by the program. Otherwise, the Compile message indicates which networks contain errors.

### Saving the Sample Program

You can save your project by selecting the menu command **Project ▸ Save All** or by clicking the Save All button.  This action saves all the components of your sample project.

## 4.5 Task: Create a Status Chart

### Building Your Status Chart

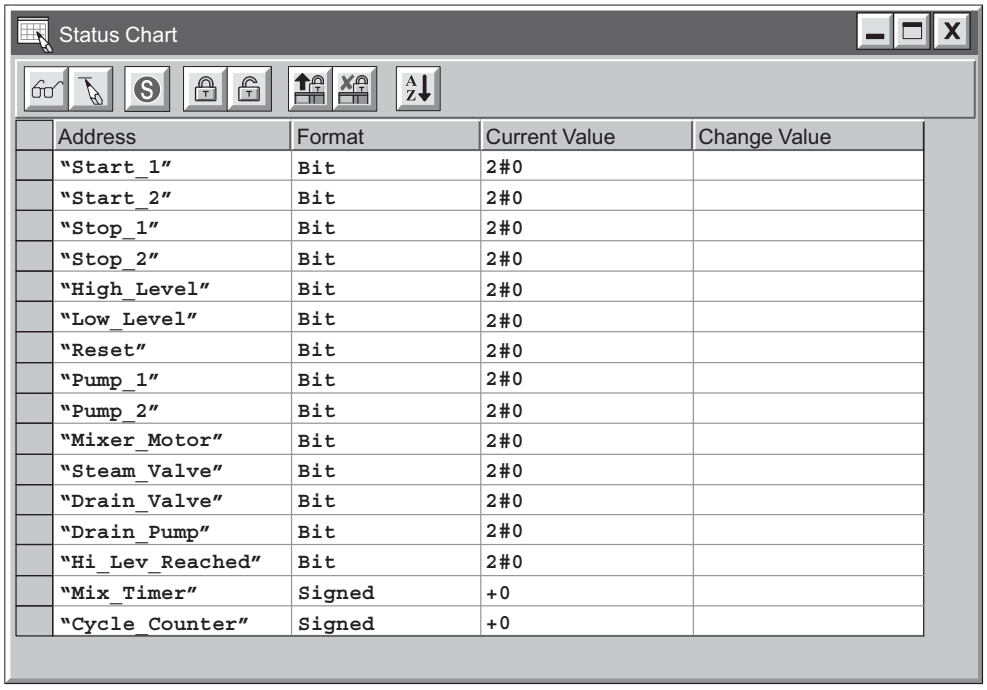
To monitor the status of selected elements in the sample program, you create a Status Chart that contains the elements that you want to monitor while running the program. To access the Status Chart editor, double-click the icon at the bottom of the main window. Then enter the elements for the sample program by following these steps:

1. Select the first cell in the Address column, and type the following: `Start_1`
2. Press ENTER to confirm your entry. This element type can only be displayed in bit format (either 1 or 0), so you cannot change the format type.
3. Select the next row, and repeat these steps for each of the remaining elements, as shown in Figure 4-13.

   When an Address cell is in focus and the row below is empty, pressing ENTER automatically increments the address for each additional row. Refer to the online Help for more information about using the Status Chart.

   You can use the menu command **Edit ▸ Insert Row** (or the INSERT or INS key) to insert a blank row above the row containing the cursor.
4. The timer T37 and the counter C30 can each be displayed in other formats. With the focus in the Format column cell, press the SPACEBAR to cycle through the formats that are valid for these element types. For this example, select Signed for the timer and counter.

Save your Status Chart by selecting the menu command **Project ▸ Save All** or by clicking the Save All button. 




| | Address | Format | Current Value | Change Value |
|---|---|---|---|---|
| | "Start_1" | Bit | 2#0 | |
| | "Start_2" | Bit | 2#0 | |
| | "Stop_1" | Bit | 2#0 | |
| | "Stop_2" | Bit | 2#0 | |
| | "High_Level" | Bit | 2#0 | |
| | "Low_Level" | Bit | 2#0 | |
| | "Reset" | Bit | 2#0 | |
| | "Pump_1" | Bit | 2#0 | |
| | "Pump_2" | Bit | 2#0 | |
| | "Mixer_Motor" | Bit | 2#0 | |
| | "Steam_Valve" | Bit | 2#0 | |
| | "Drain_Valve" | Bit | 2#0 | |
| | "Drain_Pump" | Bit | 2#0 | |
| | "Hi_Lev_Reached" | Bit | 2#0 | |
| | "Mix_Timer" | Signed | +0 | |
| | "Cycle_Counter" | Signed | +0 | |

Figure 4-13 Status Chart for the Sample Program

## 4.6 Task: Download and Monitor the Sample Program

Next you must download your program to the CPU and place the CPU in RUN mode. You can then use the Debug features to monitor or debug the operation of your program.

### Downloading the Project to the CPU

Before you can download the program to the CPU, the CPU must be in STOP mode. Follow these steps to select STOP mode and to download the program:

1. Set the CPU mode switch (which is located under the access cover of the CPU module) to TERM or STOP.
2. Select the menu command **CPU ▸ Stop** or click the Stop button  in the main window.
3. Answer “Yes” to confirm the action.
4. Select the menu command **Project ▸ Download...** or click the Download button in the main window: 
5. The Download dialog box allows you to specify the project components you want to download. Press ENTER or click “OK.”

An information message tells you whether or not the download operation was successful.

**Note**
STEP 7-Micro/WIN does not verify that your program uses memory or I/O addresses that are valid for the specific CPU. If you attempt to download a program that uses addresses beyond the range of the CPU or program instructions that are not supported by the CPU, the CPU rejects the attempt to download the program and displays an error message.

You must ensure that all memory locations, I/O addresses, and instructions used by your program are valid for the CPU you are using.

### Changing the CPU to RUN Mode

If the download was successful, you can now place the CPU in RUN mode.

1. Select the menu command **CPU ▸ Run** or click the Run button  in the main window.
2. Answer “Yes” to confirm the action.

### Monitoring Ladder Status

Ladder status shows the current state of events in your program. Reopen the Ladder Editor window, if necessary, and select the menu command **Debug ▸ Ladder Status On**.

If you have an input simulator connected to the input terminals on your CPU, you can turn on switches to see power flow and logic execution. For example, if you turn on switches `I0.0` and `I0.2`, and the switch for `I0.4` ("High_Level") is off, the power flow for Network 1 is complete. The network looks like the one shown in Figure 4-14.

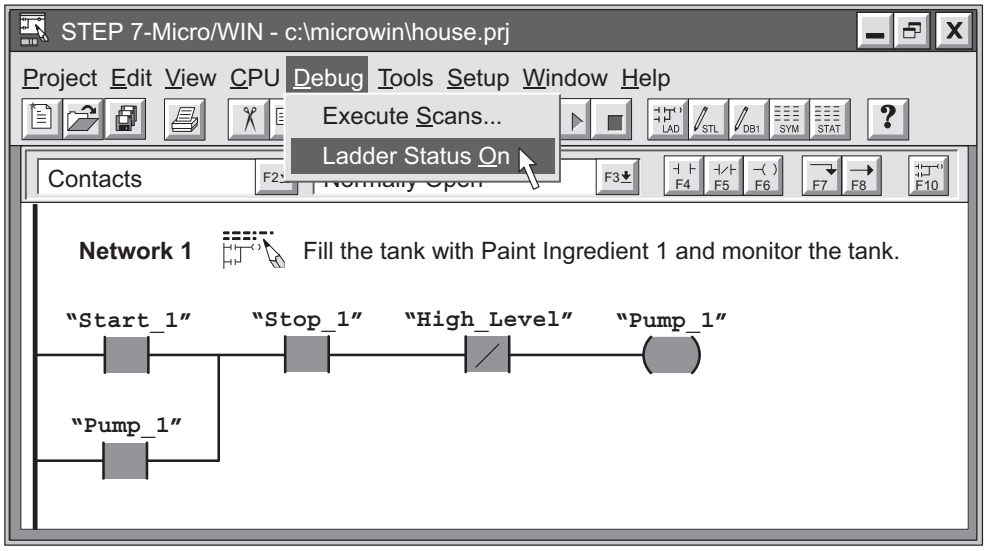



Figure 4-14 Monitoring Status of the First Network

If your STEP 7-Micro/WIN program does not match the CPU program, you are notified by the warning screen shown in Figure 4-15. You are then asked to either compare the program to the CPU, continue this operation, or cancel the operation.

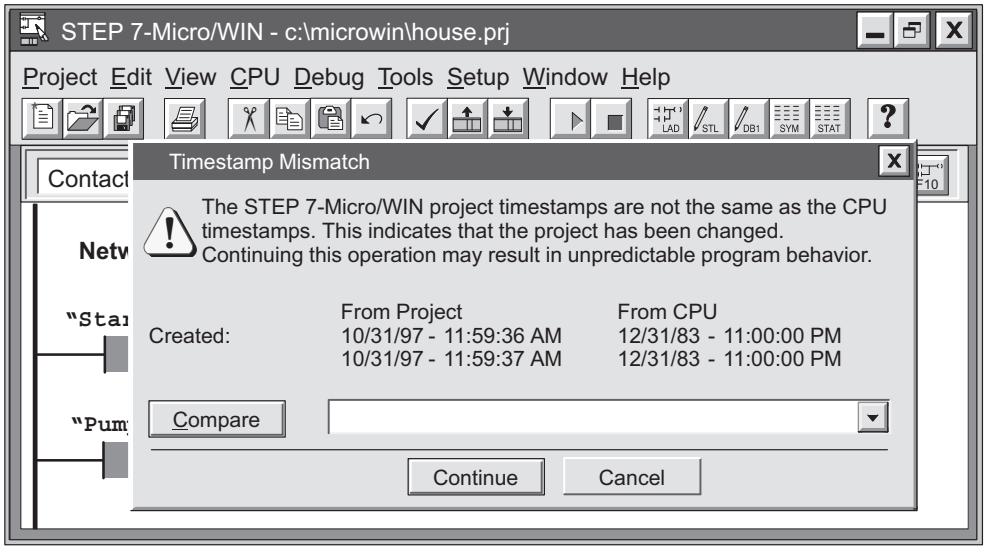



Figure 4-15 Timestamp Mismatch Warning Screen

### Viewing the Current Status of Program Elements

You can use the Status Chart to monitor or modify the current values of any I/O points or memory locations. Reopen the Chart window, if necessary, and select the menu command **Debug ▸ Chart Status On**, as shown in Figure 4-16. As you switch inputs on or off with the CPU in RUN mode, the Status Chart shows the current status of each element.

- To view the current PLC value of the elements in your program, click the Single Read button or the Continuous Read button in the Status Chart window.
- To stop the reading of status, click the Continuous Read button in the Status Chart window.

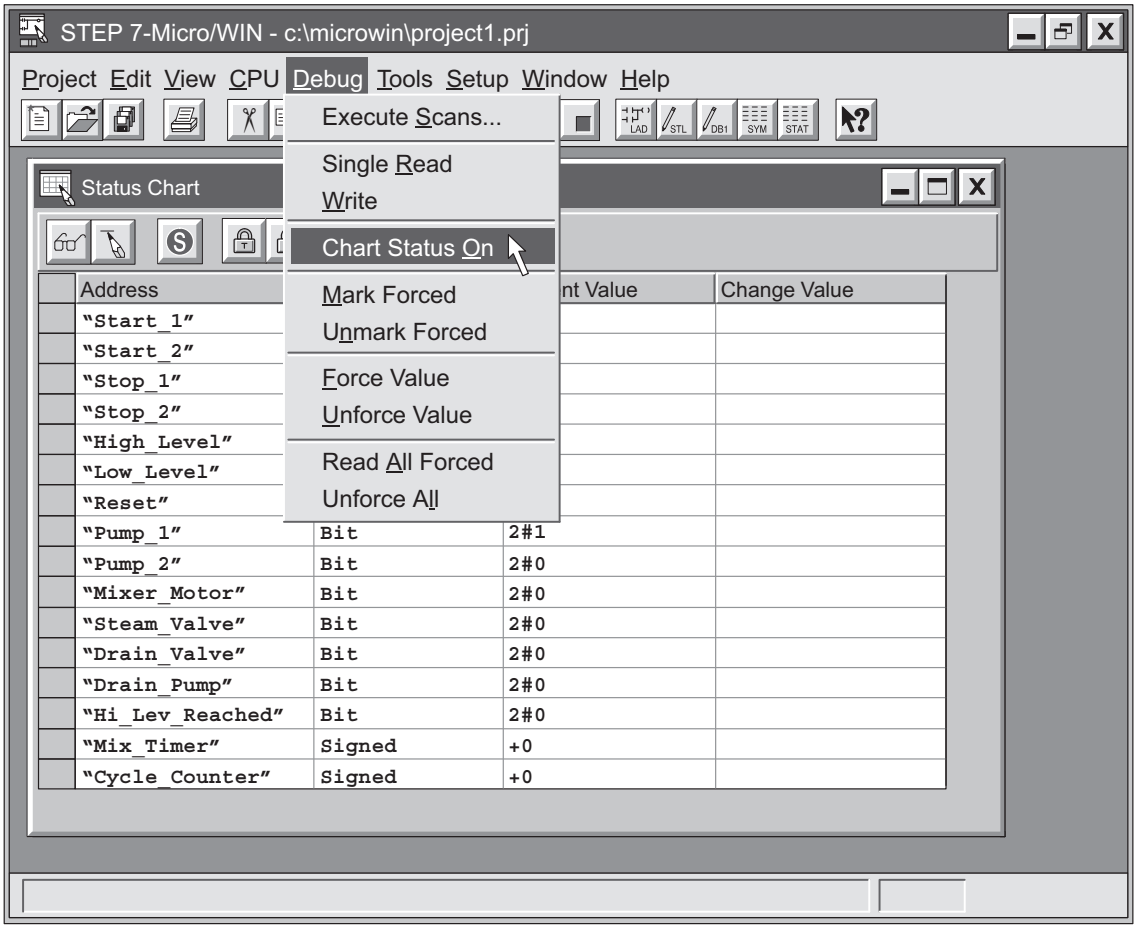



Figure 4-16 Monitoring the Status Chart of the Sample Program